A coloro che hanno il coraggio di credere.

Table of Contents

Chapter 1

Introduction

Humans have always shown an intense desire to understand and replicate the mechanisms of nature. The barter and the merchants were the foundation of how common knowledge and goods were exchanged across the known world. With time and the continuous pursuit of understanding, man were able to create a global means of exchange and interaction: financial markets. These markets are not merely economic engines that fuel our nations with much needed resources, but complex systems where governments, economic actors, and investors associate value to limited resources. The study of these systems lies at the heart of Quantitative Finance, a discipline that builds on the premise that market dynamics can be comprehended through mathematical and statistical frameworks.

However, modeling financial assets remains a central challenge in todays world, since such data are characterized by complex stochastic dynamics that frequently break the assumptions of normality and stationarity. Indeed, these time series frequently exhibit stylized facts [1], or known empirical regularities such as heavy-tailed return distributions, volatility clustering, leverage effects, and sudden, discontinuous jumps. Modeling these non-linear, high-dimensional dynamics has proven complicated. Moreover, with the increase in complexity of financial markets, the need for synthetic data as a source of information for scenario analysis, backtesting, stress test, and data augmentation has soared. Therefore understanding and modeling these patter is more relevant than ever.

Traditionally, academics have relied on statistical tools like ARIMA [2] for trend anal-

ysis and GARCH [3] for modeling heteroskedasticity. Instead, in the continuous-time domain, the Geometric Brownian Motion (GBM) has shown to be a foundational building block for asset price modeling and, particularly, for derivative pricing through Monte Carlo methods. While these models often offer high interpretability and computational efficiency, they often lack the expressivity to capture all characteristics of complex distribution of financial time series (FTS).

With the raise of Machine Learning and subsequently Deep Learning new opportunities came. Two frameworks, in particular, were promising: Generative Adversarial Networks (GANs) [4, 5] and Variational Autoencoders (VAEs) [6, 7]. While both proved exceptional in various domains, they are bound by some core limitations: GANs are notoriously unstable to train and prone to mode collapse, while VAEs produce overly smoothed samples and suffer from posterior collapse, therefore failing to provide the same level of expressivity seen in GANs. In parallel, a diverse new set of architectures has been explored, including Energy-Based Models [8], Normalizing Flows [9], and, most recently, Diffusion Models [10, 11, 12, 13, 14], which became the primary choice in many fields, because they effectively combine expressivity with training stability, reaching state-of-the-art generation for images and videos. Interestingly, while the financial sector has seen a surge in Large Language Models (LLMs) for tasks such as sentiment analysis, classification, and back-end automation, the application of generative diffusion models to FTS is still limited.

To address the gap, this work proposes to combine the Conditional Score-based Diffusion Model for Imputation (CSDI) [15] into the Elucidated Diffusion Model (EDM) [16] framework. This combination is natural, since CSDI utilizes a Transformer architecture, which was originally designed to capture temporal and feature dependencies for imputation, while EDM defines a clear and malleable design environment for diffusion models.

The research is structured in two sequential phases. Phase 1 replicates the work of Kim et al. [17], implementing their VE, VP, and GBM-inspired SDE variants on univari-

ate Close (prices) log-returns, in order to establish a validated baseline and expose the design ambiguities present in the original paper. Phase 2 then extends this foundation by embedding a CSDI backbone within the EDM framework and introducing multivariate conditioning: the model learns to generate Close log-returns given the contemporaneous Open, High, and Low observations. This sequential structure is intentional, with the replication phase first and then a novel approach in the second.

The research questions addressed in the two phases are therefore distinct:

1. **RQ1. Phase 1 — Replication** Can a CSDI-based diffusion architecture, trained under the SDE families proposed by Kim et al. [17], learn the distributional properties of S&P 500 log-returns and reproduce their core stylized facts, including heavy tails, volatility clustering, and the leverage effect?

2. **RQ2. Phase 2 — EDM-CSDI** Does embedding the CSDI backbone within the EDM framework of Karras et al. [16] improve the distributional fidelity of unconditional Close log-return samples over the Phase 1 SDE baseline? And does the further introduction of same-day Open, High, and Low log-returns as conditioning variables yield additional improvements in sample quality and probabilistic sharpness?

In answering the previous question, the following contributions were made:

- Framework integration: the CSDI architecture was adapted within the EDM to ensure mathematical clarity, replicability, and effective control over the reverse diffusion process for financial data.

- Comprehensive evaluation: models were tested on real-world observations, employing extended evaluation measures that combine distributional similarity (e.g. D_{KS}, W_1, tail exponent $\hat{\alpha}$) with probabilistic forecasting measures (CRPS and interval coverage).

- Stylized fact assessment: we evaluate the models' ability to reproduce the core

stylized facts of financial markets across SDE families, conditioning settings, and sampling strategies.

In conclusion, this thesis contributes to the exploration of diffusion models for FTS, empirically validating the SDE-based replication of Kim et al. [17] and showing that embedding the CSDI backbone within the EDM framework, together with same-day Open, High, and Low conditioning, substantially narrows the gap to the empirical distribution in higher moments and tail behaviour. More broadly, the work generalizes the design framework to enable easier replicability and a higher degree of control over training choices.

This thesis commences with a literature review (Ch. 2) covering FTS modeling from classical methods to deep learning ones, and a detailed explanation of EDM and CSDI implementations. Ch. 3 then presents the theoretical framework: the classical GARCH-X and Merton-X baselines, the SDE formulations underlying Phase 1 and Phase 2, the mathematical foundations of diffusion models, and the CSDI architecture, masking strategies, and sampling procedures, concluding with a detailed account of how the EDM framework integrates into this picture. Chapters 4 and 5 cover data preprocessing, training configuration, and the evaluation metrics, shared across both phases. Ch. 6 presents the experimental results sequentially, starting with the classical baselines, followed by Phase 1 and then Phase 2. The thesis continue with a discussion of results, limitations, and future work. Lastly a concluding chapter summarizing the overall contributions.

The two phases differ along four structural dimensions: problem setup, diffusion process, training objective, and sampling strategy. Tab. 1.1 provides a compact side-by-side reference of these differences.

Category	Dimension	Phase 1 — Replication	Phase 2 — EDM-CSDI
Problem setup	Primary objective	Replicate Kim et al.[17]; validate VE, VP, and GBM-inspired variants on unconditional univariate generation	Conditional generation of Close log-returns given observed O, H, L; CSDI backbone integrated within the EDM framework
	Channels K	$K = 1$ (univariate close log-returns)	$K = 4$ (OHLC; all channels as log-returns relative to C_{t-1})
	Conditioning / masking	`unconditional`: $m^{co} = 0$; loss computed on all entries, no side information	`predict_close`: $\{O, H, L\}$ fully observed, $\{C\}$ fully masked; fixed deterministic split
	Sequence length / stride	$L = 2048,\ s = 400$	$L = 512,\ s = 100$
Diffusion process	Forward SDE family	VE; VP; GBM-inspired VE in log-price space	VE-style forward process within EDM preconditioning
	σ_{data}	Implicit; absorbed into the choice of $\sigma_{\max}/\sigma_{\min}$	$\sigma_{data} = 1.0$ by construction (per-channel z-score normalisation)
	Noise schedule parameters	VE: $\sigma_{\max}$=1, $\sigma_{\min}$=0.01; GBM: $\sigma_{\max}$=10, $\sigma_{\min}$=0.1; VP: $\beta_{\max}$=8, $\beta_{\min}$=0.01	P_{mean}=−1.4, P_{std}=1.8 (ablation-tuned)
	Noise level sampling	$t \sim \mathcal{U}[0, 1]$	$\ln \sigma \sim \mathcal{N}(P_{mean}, P_{std}^2)$;
Training objective	Prediction target	Noise $\boldsymbol{\epsilon}$ (DSM)	Clean data x_0 (MMSE)
	Loss function	$\mathbb{E}\big[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(x_t, t)\|^2_{m^{ta}}\big]$	$\mathbb{E}\big[\lambda(\sigma)\|D_\theta(x_t; \sigma) - x_0\|^2_{m^{ta}}\big]$
	Loss weighting λ	$\lambda(\sigma_i) = \sigma_i^2$	$\lambda(\sigma) = c_{out}(\sigma)^{-2}$
	Preconditioning	None; raw network $F_\theta(x_t;\, t)$	$D_\theta = c_{skip}\, x + c_{out}\, F_\theta(c_{in}\, x;\, c_{noise})$, with σ_{data}=1.0
	Diffusion-step embedding	Discrete step index t (sinusoidal basis + two linear layers with SiLU)	$c_{noise}(\sigma) = \frac{1}{4} \ln \sigma$; compresses the dynamic range of σ to a bounded scalar
Sampling	Numerical sampler	Probability flow ODE (first order)	Probability flow ODE (second-order Heun)
	Sampler–training coupling	Coupled	Decoupled

Table 1.1: Design differences between Phase 1 (replication of Kim et al., 2025) and Phase 2 (EDM-CSDI).

Chapter 2

Background and Related Work

2.1 Financial Time Series and Stylized Facts

In order to understand the methodology, it is important to define what a financial time series (FTS) is and which statistical challenges do their modeling pose.

One of the core focus of FTS analysis are log-returns, expressed as:

$$r_t = \log\left(\frac{C_t}{C_{t-1}}\right) \tag{2.1}$$

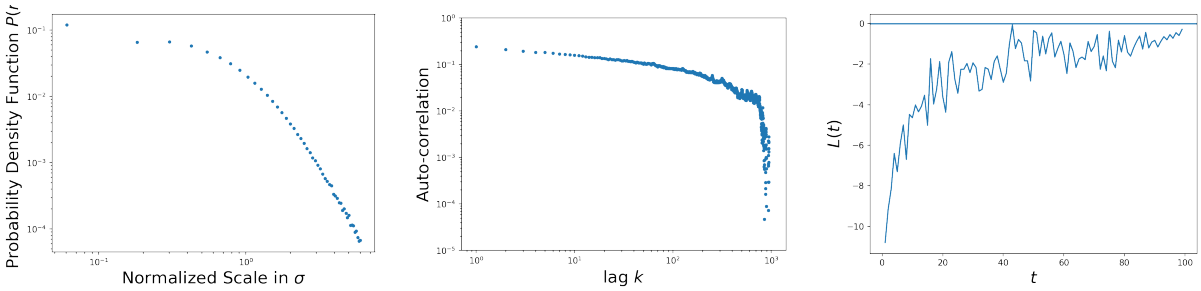
where C_t is the closing price on any given day t. Alongside this, it is useful to consider opening price (O_t), highest price (H_t), and lowest price (L_t), for brevity OHLC values.

In the literature, it is standard to consider *adjusted* versions of prices, where given a nominal price S_t, the adjusted price is $\widetilde{S} = A_t S_t$. Adjustments correct for mechanical price changes caused by corporate actions, such as stock splits, dividend payments, and other. This is done in order to retain price comparability over time. For instance, a 2-for-1 stock split, cause the stock price to be divided by two, while each investor's total economic holding remains unchanged. Without adjustment, this artificial price drop would appear as a large negative return. Adjustments cumulate through time.

FTS log-returns exhibit four main characteristics:

- Not Gaussian IID: returns do not follow a random walk.

- Fat Tails: returns distribution decay according to the power law, $P(|r| > x) \sim x^{-\alpha}$, with the typical exponent $\alpha \in [3, 5]$ for many classes of assets.
- Volatility Clustering: autocorrelation of absolute returns, $|r_t|$, and square returns, r_t^2, decays hyperbolically, respecting $\text{Corr}(|r_t|, |r_{t+k}|) \sim k^{-\beta} \quad \beta \in [0.1, 0.5]$.
- Leverage Effect: negative returns are followed by higher future volatility asymmetrically [18]. This is measure through lead-lag correlation $L(k) = \mathbb{E}\left[r_t r_{t+k}^2\right] / \mathbb{E}\left[r_t^2\right]^2 < 0$.



(a) Heavy-tail distribution (b) Volatility clustering (c) Leverage effect

Figure 2.1: Stylized Facts for FTS, S&P500 log-returns.

It is important to state that financial returns do not follow a Gaussian behavior, indeed large price fluctuations occur more frequently than under Brownian motion, breaking random walk assumption [19, 20]. Moreover, the fact that returns have polynomial-like tail decay [1] implies that, contrary to what was believed before [19], they have finite variance, and often unstable of infinite fourth moment [21]. Therefore, it is modern consensus that returns are not necessarily symmetric Lévy-stable process [22], but that assuming a Gaussian tail distribution would significantly underestimate risk.

2.2 Classical Models

Classical FTS models often address non-Gaussian behaviour by building upon the Brownian motion framework, owing to its mathematical tractability. In particular, GARCH-like models [3] focus on time-varying volatility, where by construction the model is conditionally heteroskedastic, meaning that large shocks are followed by higher volatility and

period of calm by lower ones. Merton's jump diffusion [23] instead build on the continuous Geometric Brownian Motion (GBM) by adding a Poisson jump component. These models are frequently used to capture stylized facts, which is why they would serve as baseline.

In a GARCH(1,1) we have that the conditional variance behaves accordingly to: $\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$, where $\omega > 0$ and $\alpha, \beta \geq 0$, with a persistent condition $\alpha + \beta < 1$. Despite their parsimony, GARCH models remain a competitive benchmark in financial time-series modelling due to their interpretability, tractable likelihood, and well-known statistical properties.

In order to understand Merton's model we have to introduce first the GBM. Given an asset price S_t it is said to follow a GBM if:

$$dS_t = \mu S_t d\tau + \nu S_t dW_t \tag{2.2}$$

where μ is the constant drift, $\nu > 0$ is the constant volatility, and W_t is a standard Brownian motion (Sec. D.1 for more details). The core feature of this process is the multiplicative structure of the noise term, which is scaled by price level. Applying Itô's Lemma to $\log S_t$ it is possible to derive:

$$S_t = S_0 \exp\left[\left(\mu - \frac{1}{2}\nu^2\right)\tau + \nu W_t\right]$$

therefore with the GBM assumption, , Black and Scholes [24], implied that log-returns are normally distributed over any interval, while heteroskedasticity is introduced in price space (proof Sec. D.2). Their model remain highly influential even nowadays, nevertheless it has some fundamental limitations due to the inability of Gaussian's innovations to replicate the heavy tails, excess kurtosis, and sudden discontinuities exhibited by real returns.

Merton addressed this by imposing a compounded Poisson jump process onto the GBM [23], yielding the model that takes his name:

$$dS_t = \mu S_t dt + \nu S_t dW_t + S_{t^-}(J-1)dP_t$$

where P_t is the Poisson process with intensity λ and J is a random jump multiplier $J \sim \mathcal{N}(\mu_j, \sigma_J^2)$. This result in log-returns distribution that is a Poisson's weighted mixture of Gaussians.

Both of these models impose parametric distributions, while this enable easier interpretation and a well understood behavior that makes them valuable baselines, it is also their limiting factor, since it does not enable them to learn complex, highly dimensional joint distribution, which is also why they require re-fitting whenever their estimation regime changes. Limitations are discussed in Sec. D.4.1 and Sec. D.4.2.

2.3 Deep Generative Models

Deep generative models are non-parametric, therefore they learn implicitly the data distribution, $p_{\text{data}}(x)$, and can theoretically learn all the stylized facts simultaneously, including temporal and feature dependencies. Before diffusion models, two families of models mostly dominated the FTS modeling and generation scene: GANs [4, 5, 25, 26] and VAEs [6, 7]. GANs, Generative Adversarial Networks, consist of a non-cooperative minmax game between a Generator $G_\theta(z)$ and a Discriminator $D_\phi(x)$: by learning implicitly the distribution without explicit likelihood constraints they are able to avoid oversmoothing and generate more sharp and diverse samples. On the other hand, GANs often suffer of mode collapse, when the Generator fail to capture fully the target distribution $p_{\text{data}}(x)$ and it maps only to few modes. Moreover, GANs suffer from training instability due to the non-convex nature of the minmax problem, which makes the optimization difficult.

VAEs, Variational Autoencoders, provide a probabilistic framework where input data x are mapped into a latent space through an encoder and reconstructed via a decoder $p_\theta(x|z)$, with the system trained to maximise the Evidence Lower Bound (ELBO), fundamentally balancing a reconstruction term and a KL regularisation term. While theoretically sounding, VAEs suffer mainly from two limitations: posterior collapse and oversmoothing. The former, happens when the KL regularisation term dominates early train-

ing phases and push the approximate posterior towards the prior before any informative encoding is learned, degenerating into an unconditional mean model. The latter, because of the Gaussian decoder assumption, minimise square reconstruction error, which systematically compress variance and suppress tail events. Both model families have limitations that make them difficult to apply to FTS. In contrast, diffusion models provide a probabilistic framework and a stable training objective, at the cost of slower sampling.

2.4 Score-based Diffusion Models

Score-based diffusion models revolve around a simple idea: rather than learning to generate data directly, a model could learn to denoise it. If data are gradually corrupted, up to a point, where they become indistinguishable from a fixed prior, the generation is then reduced to learn how to reverse the corruption process, one step at the time. What makes the reversal possible is the score function,$\nabla_x \log p(x)$, that intuitively points in the direction one should move a noisy sample to make it more likely under the current data distribution.

The score function is chosen to represent the probability distribution of the generative model, because it encodes the geometry of the distribution and it does not require the computation of the normalizing constant, Z_θ, which are often complex and unstable. Therefore, our goal is to learn a function that can approximate the score, $s_\theta(x) \approx \nabla_x \log p(x)$ by minimizing the Fisher divergence between $\mathbb{E}_{p(x)} = [\|\nabla_x \log p(x) - s_\theta(x)\|_2^2]$ the two distributions. This can be achieved without knowing the ground truth through methods called Score Matching (SM) [27, 28].

Once we have trained a score-based model, we can use Langevin dynamics [29] to draw samples from it. Due to the weighting of the Fisher Divergence the score estimate are only reliable in high density region, but Langevin dynamics must traverse low-density regions to move between modes while guided by the score. To address this, data are perturbed with a sequence of noise levels, $\sigma_1 < \sigma_2 < \cdots < \sigma_L$, that populate low-density regions and makes score estimation accurate everywhere. Therefore, we move to estimating a

Noise-Conditional Score-based Network (NCSN) [30], $s_\theta(x, i)$, where $i = 1, 2, \ldots, L$. The chosen noise is often Gaussian $\mathcal{N}(0, \sigma_i^2 I)$ and the sampling is performed through annealed Langevin dynamics. This model is often referred to as Score Matching with Langevin Dynamics (SMLD).

Independently, DDPM's authors Ho et al. [10] reached a similar framework, where the forward process is not motivated through score estimation, but is a fixed Markov chain that gradually injects Gaussian noise into the data. With the advantage that the marginal distribution at any step t has a closed form:

$$q(x_t|x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I\right), \quad q(x_t|x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right)$$

where $\alpha_t := 1-\beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$. They also shows that by reparametrizing $x_t(x_0, \epsilon) = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$, the objective function get simplified to a noise prediction loss:

$$\mathcal{L} = \mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2\right] \tag{2.3}$$

The key connection that makes the score matching tractable was derived by Vincent [28], who showed that the intractable Fisher Divergence over the clean distribution $p(x)$ can be replaced by a noisy one $q_\sigma(\tilde{x}|x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$, yielding the Denoising Score Matching (DSM) objective:

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{x\sim p(x), \tilde{x}\sim q_\sigma(\tilde{x}|x)}\left[\|s_\theta(\tilde{x}, \sigma) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)\|^2\right]$$

Since $q_\sigma(\tilde{x}|x)$ is Gaussian then it is fully tractable with $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x) = -\epsilon/\sigma$, where $\tilde{x} = x + \sigma\epsilon$ and $\epsilon \sim \mathcal{N}(0, I)$ is the added noise. The score therefore points from the corrupted sample $\tilde{x}$ back towards the clean data x, scaled by σ^2. This reveals that the DDPM noise-prediction objective (Eq.2.3) is not conceptually different from score matching up to a scaling factor, since $\epsilon_\theta(x_t, t) \approx -\sqrt{1-\bar{\alpha}_t}s_\theta(x_t, t)$. By consequence, DDPM and NCSN can be understood as discrete instantiations of a more general principle:

corrupting data with a noise process and learning to reverse it by estimating scores. When training over multiple noise levels, the DSM objective becomes a weighted sum:

$$\mathcal{L} = \sum_{i=1}^{L} \lambda(\sigma_i)\, \mathbb{E}_{x\sim p(x),\, \tilde{x}\sim q_{\sigma_i}(\tilde{x}|x)} \left[\| s_\theta(\tilde{x}, \sigma_i) - \nabla_{\tilde{x}} \log q_{\sigma_i}(\tilde{x}|x) \|^2 \right]$$

Since the score satisfies $\nabla_{\tilde{x}} \log q_{\sigma_i}(\tilde{x}|x) = -\epsilon/\sigma_i$, its magnitude varies strongly across noise levels, and the choice of weighting $\lambda(\sigma_i)$ is non-trivial. Song et al. [30] propose $\lambda(\sigma_i) = \sigma_i^2$ to approximately equalise the contribution of each noise level, but this choice is heuristic rather than principled.

Song et al. [11] formalize the diffusion model process by lifting DDPM and SMLD into a continuous framework. He showed that the diffusion process $\{x_t\}_{t=0}^{T}$ indexed by a continuous time variable $t \in [0, T]$, where $x_0 \sim p_0$ is composed by i.i.d. samples of the dataset, and $x_T \sim p_T$, for which we have a tractable form, can be modeled as the solution to an Itô SDE:

$$dx_t = f(x_t, t)dt + g(t)dW_t \tag{2.4}$$

where W is the standard Wiener process, $f(\cdot, t) : \mathbb{R}^d \to \mathbb{R}^d$ is a vector valued function called the drift coefficient of x_t, and $g(\cdot) : \mathbb{R} \to \mathbb{R}$ is a scalar function known as the diffusion coefficient of x_t. The SDE admits a unique strong solution provided the coefficients are globally Lipschitz [31], and p_T is chosen to be a tractable prior, typically a Gaussian with fixed mean and variance, that contains no information about p_0.

The key insight is that the forward process admits a corresponding reverse-time SDE, specifically it has been shown by Anderson [32] that the reverse of a diffusion process is a diffusion process itself. By starting from samples $x_T \sim p_T$ and reversing the process from $t = T$ to $t = 0$, we can obtain samples $x_0 \sim p_0$ according to the relationship:

$$dx_t = \left[f(x_t, t) - g(t)^2 \nabla_x \log p_t(x_t) \right] dt + g(t)d\bar{W}_t \tag{2.5}$$

where $\bar{W}_t$ is a standard Wiener process that flows back from T to 0. Since $\nabla_x \log p_t(x_t)$

is unknown then we estimate it by using $s_\theta(x_t, t)$. Importantly, Song et al. [11] showed that for any SDE (Eq.2.4) there exists a corresponding deterministic Ordinary Differential Equation (ODE), called probability flow ODE, that produces the same family of marginal distributions $\{p_t\}_{t\in[0,T]}$:

$$dx_t = \left[f(x_t, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x_t) \right] dt \tag{2.6}$$

substituting the intractable score with the learned one yields a Neural ODE [33], where given starting conditions $x_T \sim p_T$ one can deterministically integrate backward to obtain an approximate sample from p_0. This deterministic formulation is generally more computationally efficient than its SDE counterpart, as it does not require noise injections at each step.

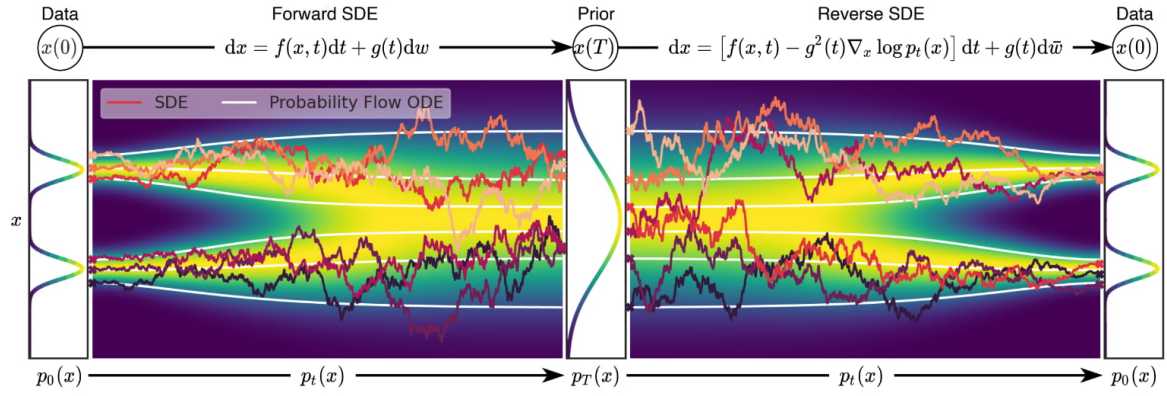


Figure 2.2: Data can be mapped to a noise distribution (the prior) with an SDE (Eq. 2.4), and reverse this SDE for generative modeling (Eq. 2.5). We can also reverse the associated probability flow ODE (Eq. 2.6), which yields a deterministic process that samples from the same distribution as the SDE. Both the reverse-time SDE and probability flow ODE can be obtained by estimating the score $\nabla_x \log p_t(x_t)$. Source: Song et al. [11].

Two specific SDE formulation that matched the SMLD and DDPM ones, in continuous time, hence when we assume that the number of noise steps $N \to \infty$, are: Variance Exploding (VE) and Variance Preserving (VP). The former sets $f(\cdot, t) = 0$ and is defined as:

$$dx_t = \sqrt{\frac{d[\sigma^2(t)]}{dt}} dW_t \tag{2.7}$$

so that $x_t = x_0 + \sigma\epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$, therefore the marginal's variance grows unbounded

as $\sigma(t)$ increases. VP instead is:

$$dx_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)}dW_t$$

with drift $f(x_t, t) = -\frac{1}{2}\beta(t)x_t$ and $g(t) = \sqrt{\beta(t)}$. This setting preserve unit variance when x_0 has it as well with the marginal converging to $\mathcal{N}(0, I)$ as $t \to T$.

The reverse SDE can be solved numerically through different samplers. Song et al. [11] employs the Euler-Maruyama method, a first-order discretisation of the reverse-time SDE:

$$x_{t+dt} \leftarrow x_t + \left[f(x_t, t) - g(t)^2 s_\theta(x_t, t)\right] |dt| + g(t)\sqrt{|dt|}\,\epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{2.8}$$

though any suitable numerical solver can in principle be substituted.

This continuous formulation has the advantage of decoupling the noise schedule from the choice of sampler, admitting common schedules such as linear, exponential, and cosine. However, both DDPM [10] and the SDE-based framework [11] require hundreds of Neural Function Evaluations (NFEs) to produce a single sample, and neither provides a principled answer to how noise levels should be chosen, nor how network inputs and outputs should be scaled.

2.5 Elucidating the Design Space of Diffusion Models

In Elucidated Diffusion Models (EDM), Karras et al. [16] identify that prior formulations, including DDPM and Song's SDEs, choose in a dependable way four conceptually distinct design choices: the noise schedule, the parametrisation of network inputs and outputs, the weighting of the loss across noise levels, and the numerical solver used at sampling time. EDM separates these into four independent components and optimises each separately.

Preconditioning EDM parametrises the denoiser as:

$$D_\theta(x;\sigma) = c_{\text{skip}}(\sigma)\, x + c_{\text{out}}(\sigma)\, F_\theta(c_{\text{in}}(\sigma)\, x;\; c_{\text{noise}}(\sigma)) \tag{2.9}$$

where F_θ is the raw neural network and $c_{\text{skip}}, c_{\text{out}}, c_{\text{in}}, c_{\text{noise}}$ are scalar functions of σ chosen so that the inputs to F_θ have unit variance, the training targets have unit variance, and errors in F_θ are amplified as little as possible. For data with standard deviation σ_{data}, EDM sets:

$$c_{\text{skip}}(\sigma) = \frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}, \quad c_{\text{out}}(\sigma) = \frac{\sigma \cdot \sigma_{\text{data}}}{\sqrt{\sigma^2 + \sigma_{\text{data}}^2}}$$

$$c_{\text{in}}(\sigma) = \frac{1}{\sqrt{\sigma^2 + \sigma_{\text{data}}^2}}, \quad c_{\text{noise}}(\sigma) = \frac{1}{4}\ln(\sigma)$$

The skip connection $c_{\text{skip}}(\sigma)$ has a simple interpretation: at low σ, $c_{\text{skip}} \to 1$ and the network mostly copies the input, a sensible prior when the noisy observation is already close to the data; at high σ, $c_{\text{skip}} \to 0$ and the network must reconstruct the clean signal from scratch. This mechanism helps stabilizing the training across the full noise range without requiring separate tuning for each noise level. Its connection with the Wiener Filter is discussed in Sec. E.2.1.

Loss weighting The EDM training objective is defined with respect to the denoiser $D_\theta(x;\sigma)$. Given a clean sample $x_0 \sim p_0$ and noise $\epsilon \sim \mathcal{N}(0, I)$, the noisy observation is $\tilde{x} = x_0 + \sigma\epsilon$, as in VE, and the loss is:

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{\sigma,\, x_0,\, \epsilon}\left[\lambda(\sigma)\; \|D_\theta(x_0 + \sigma\epsilon;\; \sigma) - x_0\|^2\right] \tag{2.10}$$

The effective weight for each sample on the raw network prediction F_θ is $\lambda(\sigma)\, c_{\text{out}}(\sigma)^2$. EDM sets $\lambda(\sigma) = c_{\text{out}}(\sigma)^{-2}$, which cancels out this factor and equalises loss contribution across all noise levels. Without this correction, high-noise steps would dominate the gradient signal, producing a model that denoises well at large σ but fails to recover fine structure poorly, which is precisely the failure mode left unaddressed by NCSN.

Noise distribution Rather than sampling σ uniformly in t as in DDPM, EDM

concentrates training effort on the noise range that is most informative. At very low σ the noisy input is nearly identical to the clean data, and at very high σ the target collapses to the dataset mean. The informative region is in the middle, therefore EDM targets it by drawing:

$$\ln \sigma \sim \mathcal{N}(P_{\text{mean}},\, P_{\text{std}}^2) \tag{2.11}$$

with default values $P_{\text{mean}} = -1.2$ and $P_{\text{std}} = 1.2$, calibrated on ImageNet [34].

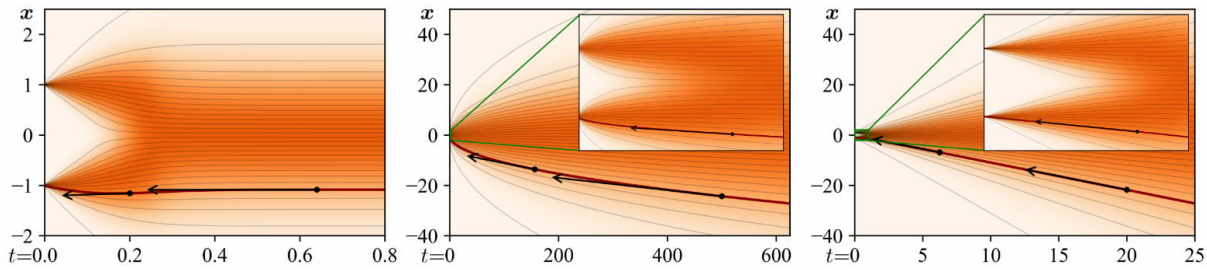


Figure 2.3: A sketch of ODE curvature in 1D where p_{data} is two Dirac peaks at $x \pm 1$ Horizontal t axis is chosen to show $\sigma \in [0, 0.25]$ in each plot, with insets showing $\sigma \in [0, 1]$ near the data. Example local gradients are shown with black arrows. *Left.* Variance preserving ODE of Song et al. [11] has solution trajectories that flatten out to horizontal lines at large σ. Local gradients start pointing towards data only at small σ. *Center.* Variance exploding variant has extreme curvature near data and the solution trajectories are curved everywhere. *Right.* With the schedule used by DDIM [12] and us, as σ increases the solution trajectories approach straight lines that point towards the mean of data. As $\sigma \to 0$, the trajectories become linear and point towards the data manifold. Source: Karras et al. [16].

Sampler design At inference time, EDM employs a deterministic second-order Heun ODE solver with a ρ-power noise schedule:

$$\sigma_i = \left(\sigma_{\max}^{1/\rho} + \frac{i}{N-1}\left(\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}\right)\right)^{\rho}, \quad i = 0, \ldots, N-1 \tag{2.12}$$

with $\rho = 7$. The power-law schedule concentrates discretisation steps at small σ, where the ODE curvature is highest and denoising errors have the greatest impact on sample quality, which is why instead of $\rho = 3$ that equalize the truncation error at each step they selected higher values. This resulted in equivalent sample quality to DDPM with approximately 7× fewer NFEs, and better convergence as shown in Fig. 2.3. Moreover, the sampler is independent of the training procedure, therefore a model trained with the

EDM loss could be evaluated with any compatible ODE or SDE solver.

Stochastic Heun Sampler Karras et al. also propose a stochastic variant of the Heun solver in which a controlled amount of noise is reinjected before each denoising step. Before applying the standard second-order Heun update, the current noise level σ_i is temporarily inflated to

$$\hat{\sigma}_i = \sigma_i \sqrt{1 + \gamma_i^2}, \qquad \gamma_i = \min\left(\frac{S_{\text{churn}}}{N},\ \sqrt{2} - 1\right), \tag{2.13}$$

and the sample is perturbed as $\hat{x}_i = x_i + \sqrt{\hat{\sigma}_i^2 - \sigma_i^2}\ \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, I)$, before the Heun step proceeds from $(\hat{x}_i, \hat{\sigma}_i)$ to σ_{i+1}. The scalar $S_{\text{churn}} \geq 0$ is the sole stochasticity parameter: when $S_{\text{churn}} = 0$ every $\gamma_i = 0$ and the algorithm collapses to the deterministic ODE solver, while positive values allow the trajectory to briefly re-enter a higher-noise regime at each step, counteracting discretisation errors at the cost of additional noise injections.

2.6 Conditional Score-based Diffusion Models for Time Series Imputation (CSDI)

Standard diffusion models, as described in the previous sections, learn to generate unconditionally from p_{data}. For time series imputation, or more generally for conditional generation, given a subset of conditioning observed values related to the target entries we can compute the conditional distribution $p(x^{\text{ta}}|x^{\text{co}})$. CSDI [15] make this possible via a masking mechanism that is applied both at training and inference time.

Architectural lineage To understand the architectural choices made in CSDI, it is useful to look at evolution of CSDI's backbone. PixelCNN [35] established the core ideas of modeling high-dimensional data through local conditional information by using masked convolutions, residual connections, and gated activations, enabling the understanding of local and global relationships. WaveNet [36] transferred this logic from images to raw audio by replacing 2D masked image convolutions with 1D causal dilated convolutions,

allowing the receptive field to grow exponentially with depth while maintaining causality. DiffWave [37] then kept the WaveNet-style residual architecture but abandoned the autoregressive component entirely, since it moves to a diffusion model. Because during denoising the target is the entire signal, future context is accessible to the model, and causal convolutions are replaced by bidirectional dilated convolutions. The key inherited components from WaveNet are the stacks of residual blocks, the diffusion-step conditioning, gated activations, and skip aggregation. CSDI [15] then adapts this DiffWave backbone to multivariate time-series imputation, replacing the convolutional modeling with two-dimensional attention and introducing an explicit conditioning mechanism for observed values. The evolution therefore follows a clear progression: firstly autoregressive local dependency modelling, secondly dilated temporal convolutions, thirdly diffusion denoising with WaveNet-style residual blocks, and then CSDI's conditional time-series denoiser with attention.

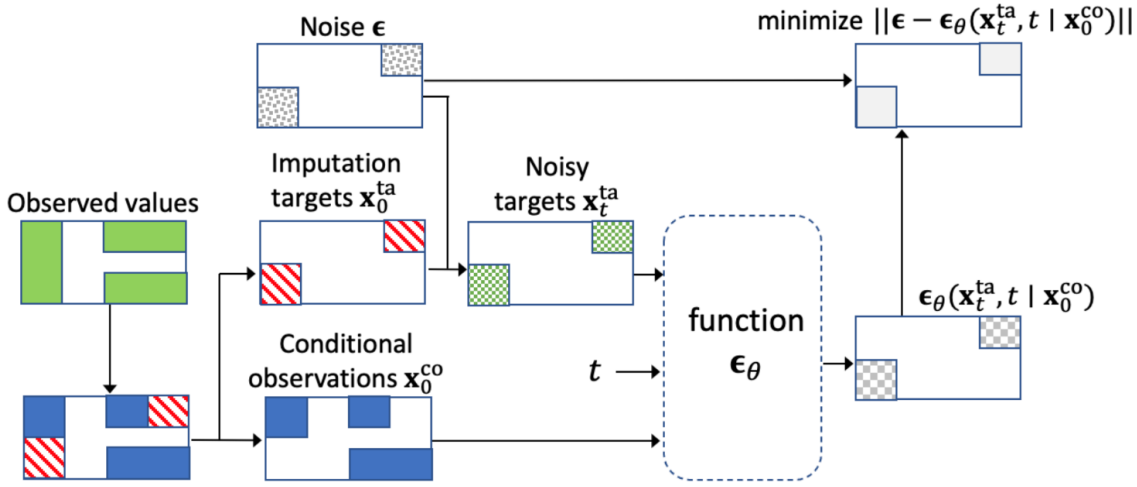


Figure 2.4: The self-supervised training procedure of CSDI for implementation of time series imputation. The colored areas in each rectangle represent the existence of values. The green and white areas represent observed and missing values, respectively, and white areas are padded with zeros to fix the shape of the inputs. Zero padding is also applied to all white areas. As with Figure 2, the observed values are separated into red imputation targets x_0^{ta} and blue conditional observations x_0^{co}. For the extended targets $\hat{x}_0^{ta}$, the area of value 0 shows dummy values. Source: CSDI [15].

Conditional denoising mechanism Looking at Fig. 2.4, we notice that at each denoising step t, the CSDI model takes as input three quantities: the noisy target x_t^{ta},

which gets zero-padded at conditioning indices; the clean conditioning observations x_0^{co}, which gets zero-padded at target indices; and a binary mask $m^{\text{co}} \in \{0,1\}^{K\times L}$ indicating which entries are observed, where K is the number of channels and L is the sequence length. The conditional score network is therefore $\epsilon_\theta(x_t^{\text{ta}}, t \mid x_0^{\text{co}}, m^{\text{co}})$, and the training objective is:

$$\min_\theta \ \mathbb{E}\left[\left\|\epsilon - \epsilon_\theta\left(x_t^{\text{ta}}, t \mid x_0^{\text{co}}, m^{\text{co}}\right)\right\|_{m^{\text{ta}}}^2\right] \tag{2.14}$$

where $\|\cdot\|_{m^{\text{ta}}}^2$ denotes that the squared error is evaluated only at target indices. The conditioning information x_0^{co} and m^{co} are kept fixed throughout the reverse process, while only the target entries are denoised.

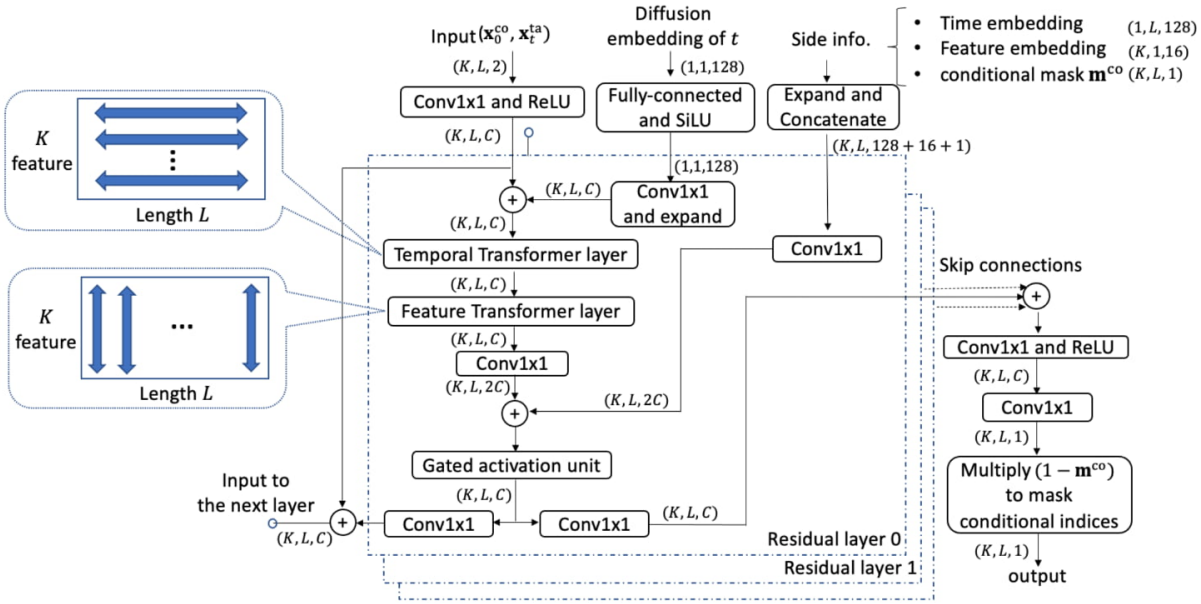


Figure 2.5: Architecture of ϵ_θ in CSDI for multivariate time series imputation. Source: CSDI [15].

2D attention over time and features The central innovation of CSDI over DiffWave is the replacement of dilated temporal convolutions with a two-dimensional attention mechanism (Fig. 2.5. Two Transformer encoders [38] operate in sequence within each residual block: a temporal Transformer which operates along the time axis, capturing short- and long-range temporal dependencies within each channel, and a feature Transformer operating along the channel axis, capturing cross-channel dependencies between variables. This 2D attention allows the model to learn both the temporal dynamics

of each channel and the contemporaneous relationships between channels, without the quadratic cost of attending jointly over the full time-feature product space.

Positional and diffusion-step embeddings Three types of embeddings are added to the latent representation to allow the model to simultaneously distinguish temporal position, noise level, and channel identity: a sinusoidal diffusion-step embedding encoding the current noise level t; a temporal embedding encoding absolute position along the time axis; and a feature embedding encoding the identity of each channel. The diffusion-step embedding plays a similar role as the noise conditioning $c_{\text{noise}}(\sigma)$ in the EDM preconditioning, ensuring that the denoiser is aware of the current noise level.

Self-supervised training strategy and limitations CSDI introduces several masking strategies for training (random, historical, mixed, and test-pattern) that simulate different conditioning and target splits during training, effectively providing a sort of data augmentation through varied masking.

2.7 GBM–style SDE

The idea of using CSDI for FTS modeling was previously explored by Kim et al. in *A diffusion-based generative model for financial time series via geometric Brownian motion* [17]. While conceptually motivated, this work suffers from reproducibility limitations and lacks precise specification of the formulation being proposed.

They showed that starting from the Black–Scholes equation (Eq. 2.2) and applying Itô's lemma to $X_t = \log S_t$, one obtains:

$$dX_t = \left(\mu_t - \frac{1}{2}\nu_t^2\right) dt + \nu_t \, dW_t,$$

where $\nu_t > 0$ is the instantaneous volatility. Choosing $\mu_t = \frac{1}{2}\nu_t^2$ cancels the drift term, yielding a VE-type SDE in log-price space (Eq. 2.7), with the forward process reduced to:

$$x_t = x_0 + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \tag{2.15}$$

where σ denotes the noise level at diffusion step t, distinct from the instantaneous volatility.

This drift cancellation is a deliberate modelling choice, not a consequence of the GBM dynamics: the resulting process is no longer a pure GBM but rather a VE-type diffusion in log-price space inspired by it. The financial interpretation is that, while noise remains additive in log-price space, the exponential map back to price space induces effective heteroskedasticity with volatility that scales with the price level.

However, an ambiguity in the original paper requires clarification. In Section 3.1, 'Geometric Brownian motion in score-based Models' is stated:

> *[...] the GBM-based formulation naturally injects larger noise into higher–value regimes, thereby preserving economically meaningful dynamics in the generated trajectories. It inherently scales noise with the underlying price level, enhancing the models sensitivity to large market movements. As a result, the diffusion process can more accurately capture stylized facts such as volatility clustering and heavy tails.*

Nevertheless, in Section 3.1.2, the authors state:

> *For each selected stock, we calculated daily log-returns, defined as* $r_t = \log(p_t/p_{t-1})$, *where* p_t *represents the adjusted closing price on day* t*. To create training samples suitable for diffusion modeling, we applied a sliding window approach to the log-return sequences.*

This indicates that the model is trained on log-returns, not on log-prices. Log-prices $X_t = \log S_t$ are non-stationary, making them undesirable as direct training targets. The GBM-style diffusion derivation above operates on log-prices in theory, but practical training on log-returns.

2.8 Heavy–Tailed Diffusion Models

A natural question follows from the heavy–tailed nature of financial time series: if the target distribution exhibits rare but economically important extreme events, should the

forward perturbation process also be heavy–tailed, so that both the noised data and the terminal prior better reflect the structure of the underlying data distribution?

This question remains open in literature, but in order to narrow down the problem we should consider the following points:

- **Q1.** Can a diffusion model based on Gaussian perturbations learn a heavy–tailed target distribution at all?
- **Q2.** If it can, how much modelling capacity should be explicitly allocated to tail behaviour rather than to the bulk of the distribution?

Recent theoretical work suggests that Gaussian perturbations are not automatically invalid in the presence of heavy tails. Yu et al. [39] study standard Gaussian score-based diffusion models when the target distribution is heavy–tailed. Under a tractable kernel-based score estimator, they derive minimax guarantees showing a distinction between tail regimes: for exponentially decaying tails, Gaussian diffusion remains statistically robust, while for polynomially decaying tails the score-estimation and sampling rates depend explicitly on the tail index. Since financial log-returns are commonly modelled as having polynomial-like tail decay, this result is directly relevant to financial time series. This does not imply that Gaussian diffusion is infeasible, but rather that heavy tails increase the statistical difficulty of score learning (Q1).

A complementary line of work modifies the perturbation distribution itself. Pandey et al. [40] propose heavy–tailed diffusion models based on multivariate Student-t perturbation kernels, leading to t-EDM. In this framework, the degrees-of-freedom parameter ν controls tail heaviness: smaller values of ν produce heavier tails, while the Gaussian case is recovered in the limit $\nu \to \infty$. Because the Kullback–Leibler divergence between Student-t distributions does not generally admit a convenient closed form, they replace the usual KL-based objective with a γ-power divergence. This objective is not an ELBO in the standard sense, but it recovers the KL divergence in the appropriate limited regime (Q2).

A more radical alternative is to replace Brownian motion itself with a heavy–tailed jump process. Yoon et al. [41] propose the Lévy-Itô Model, which uses isotropic α-stable Lévy processes instead of Brownian motion. They derive the corresponding reverse-time SDE and train the model through fractional denoising score matching. Such an approach is conceptually appealing for financial data, where jumps and rare events are structurally important, though the isotropic α-stable forward process ($\alpha < 2$, infinite variance) sits in some tension with the finite-variance, $\alpha \in [3, 5]$ characterisation of returns from Sec. 2.1. However, its substantially different mathematical framework would not have allowed comparability with Kim et al. [17], and applying CSDI as a denoiser within it would have been of doubtful outcome (Q2).

Consequently, this thesis study a Gaussian EDM–CSDI baseline for conditional financial time-series generation, with explicit tail-sensitive evaluation. Heavy–tailed perturbation kernels and Lévy-driven score models may improve tail fidelity, but they also introduce additional modelling choices and may alter the trade-off between tail accuracy, bulk distributional fit, training stability, and computational simplicity.

Chapter 3

Theoretical Framework

The following chapter builds on the previous one, detailing the theoretical structure and motivation for the implemented model.

3.1 Adapting the Framework to OHLC Financial Time Series

Phase 1 Data are formally represented as $X \in \mathbb{R}^{K \times L}$, where K denotes the number of channels and L the number of time steps per window. In Phase 1, $K = 1$ and the input reduces to a univariate sequence of Close log-returns.

Phase 2 OHLC have $K = 4$ which denotes the number of channels (Open, High, Low, Close), or conditioning features. This partition is financially motivated: the opening price O_t is fixed at the start of the trading session, while the intraday extremes H_t and L_t are continuously observed throughout it, encoding both the direction of price movements and the microstructure of intraday volatility. The closing price C_t, by contrast, is determined only at the end of the session. Two derived quantities are particularly informative as conditioning signals: the overnight return O_t/C_{t-1}, which captures the opening gap with respect to the previous close, and the intraday range $H_t - L_t$, a well-established realised volatility proxy [42]. In addition, log-returns are often preferred to nominal prices because of their stationarity [1], which makes them comparable across time and assets,

removing underlying price trends.

The data are therefore partitioned into observed conditioning variables $X^{\text{co}} = \{O, H, L\}$ and a generation target $X^{\text{ta}} = \{C\}$, and the model is trained to learn the conditional distribution:

$$p\big(X^{\text{ta}} \mid X^{\text{co}} = x^{\text{co}}, \tau\big)$$

where τ denotes the temporal conditioning provided by `observed_tp`. Unlike X^{co}, which is masking-strategy dependent, τ is always present (Sec. 3.7).

Daily prices are constrained by the ordering relationship:

$$L_t \leq \min(O_t, C_t) \leq \max(O_t, C_t) \leq H_t$$

When the series is expressed as log-returns (Eq.2.1), this constraint no longer admits a closed form, since it involves nonlinear inequalities across cumulative sums of returns rather than individual observations. Nevertheless, it remains a fundamental structural property of the data-generating process. On the other hand, because the Gaussian diffusion kernel has unbounded support, there is no guarantee that generated Close values will respect the implied bounds $[L_t, H_t]$. Moreover, enforcing such constraints is not always desirable: confining generated price paths strictly within the High and Low bounds would severely restrict the diversity of the generated samples. No constraint-preserving mechanism is applied in this work; the ordering relationship is further disrupted by feature-wise global normalisation, which rescales each channel independently and therefore does not preserve the relative ordering between channels.

3.2 Classical Baselines with OHLC Conditioning

As introduced in Sec. 2.2, GARCH and Merton models serve as parametric benchmarks. Both are extended with exogenous OHLC conditioning, denoted by the suffix -X, so that they receive the same per-day information as the diffusion model's conditioning set. Each model is fitted per ticker by maximum likelihood via L-BFGS-B [43].

Log-GARCH-X with Student-t Innovations The close log-return follows an AR(1) mean with log-GARCH variance: $r_t = \mu + \phi\, r_{t-1} + \varepsilon_t$ with $\varepsilon_t = \sigma_t z_t$ and $z_t \sim t_\nu$, therefore we are modeling:

$$\log \sigma_t^2 = \omega + \alpha \log(\varepsilon_{t-1}^2 + c) + \beta \log \sigma_{t-1}^2 + \boldsymbol{\delta}^\top \mathbf{q}_t \tag{3.1}$$

with $|\phi| < 0.5$, $\alpha+\beta < 0.98$, and $\nu > 2.01$. The log-GARCH formulation keeps σ_t^2 positive by construction without any sign constraints on (ω, α, β). The Student-t innovations is used to better capture fat tails. The OHLC conditioning vector is:

$$\mathbf{q}_t = \left[o_t^2,\ \tfrac{(\log H_t/L_t)^2}{4\log 2}\right]^\top,$$

where o_t^2 is the squared overnight log-return and the second term is the Parkinson [42] realised variance estimator (Sec. D.5 for more details).

Merton-X The close log-return is decomposed into a diffusive and a compound Poisson component:

$$r_t = \mu + \nu_t Z_t + \sum_{i=1}^{P_t} Y_{t,i}, \quad P_t \sim \text{Poisson}(\lambda), \quad Y_{t,i} \sim \mathcal{N}(\mu_J, \sigma_J^2),$$

where the diffusive volatility ν_t is driven by observations:

$$\log \nu_t^2 = a_0 + \mathbf{a}^\top \mathbf{q}_t, \qquad \mathbf{q}_t = \left[o_t^2,\ (\log H_t/L_t)^2\right]^\top. \tag{3.2}$$

Unlike GARCH-X, there is no sequential recursion: ν_t depends only on same-day observables, making the inference structure directly analogous to the diffusion model's use of $\{O_t, H_t, L_t\}$ to generate C_t. It is well established that marginalising over Poisson counts results in a Gaussian mixture likelihood (Sec. D.5).

Both baselines are parametric and fitted separately on each ticker's full history, and are therefore ticker-specific. Moreover Merton-X captures a complementary stylized fact

to GARCH-X: discontinuities. Nevertheless, none of the two learns the conditional distribution $p(r_t^C \mid r_t^O, r_t^H, r_t^L)$ non-parametrically. The diffusion model is the non-parametric alternative, at the cost of higher complexity.

3.3 VE Instantiation for OHLC

As discussed in Sec. 2.4, diffusion models were originally developed in distinct formulations before being unified under the EDM framework. These formulation are relevant for Phase 1, which aims to replicate the results of Kim et al.[17]. Meanwhile, for Phase 2 we will move towards the EDM framework.

Phase 1 Phase 1 is identical in its implementation to Song et al. in [11], but in a simplified version.

Phase 2 For Phase 2, by looking at the forward VE formulation (Eq. 2.15), we can derive the reverse SDE

$$dx = -\sigma \nabla_x \log p_\sigma(x)$$

when defined in σ-space instead of t-space and with σ decreasing, as by EDM framework. The deterministic counterparts, the probability flow ODE, is:

$$\frac{dx}{d\sigma} = -\frac{\sigma}{2} \nabla_x \log p_\sigma(x)$$

As discussed in Sec. 2.4 we can use DSM to brigde the gap between denoising and score matching. It turned out that DSM is a special case of a statistical result known as Tweedies formula, which provides a relationship between score functions and conditional expectations. For noisy observation Eq. 2.15 the optimal MMSE denoiser is:

$$D^*(x_t; \sigma) = \mathbb{E}\left[X_0 | X_t = x_t\right]$$

Therefore, the denoiser and score function are connected by:

$$\nabla_{x_t} \log p_\sigma(x_t) = \frac{D^*(x_t; \sigma) - x_t}{\sigma^2}$$

implying that rather than learn the score field directly, which is hard and an ill-conditioned regression target in high dimensions, we train a denoiser D_θ to approximate D^*, and we later recover the score analytically.

3.4 GBM–style SDE and CSDI (Kim et al.)

In Sec.2.7 we highlighted that the GBM derivation operates on log-prices $X_t = \log S_t$ in theory, while training is performed on log-returns. The cumulative interpretation of log-returns closes this gap, since summing returns recovers the log-price path the GBM dynamics act upon. This stands from the fact that training diffusion models on raw log-prices is inadvisable for two reasons. First, log-prices are non-stationary, therefore their mean grows approximately linearly and with different scales across tickers and time periods. Second, the sampling process relies on a Gaussian prior $\mathcal{N}(0, \sigma_{\max}^2 I)$; for this prior to be well-matched to the training distribution, the input data must be approximately centred at zero with a scale commensurate with $\sigma_{\max}$. Raw log-prices violate both conditions significantly. Daily log-returns $r_t = \log(S_t/S_{t-1})$ solve non-stationarity, however, working with cumulative log-returns $\mathcal{C}_{\tau+k} = \sum_{j=1}^{\tau+k} r_j$ is motivated by the GBM theoretical motivations of Sec. 2.7 and by the experimental results shown in Figures 4–6 of Kim et al. [17], that otherwise wouldn't have any reason to differentiate VE from GBM. How non–stationarity of cumulative log-returns is handled is addressed in Sec. 4.3.

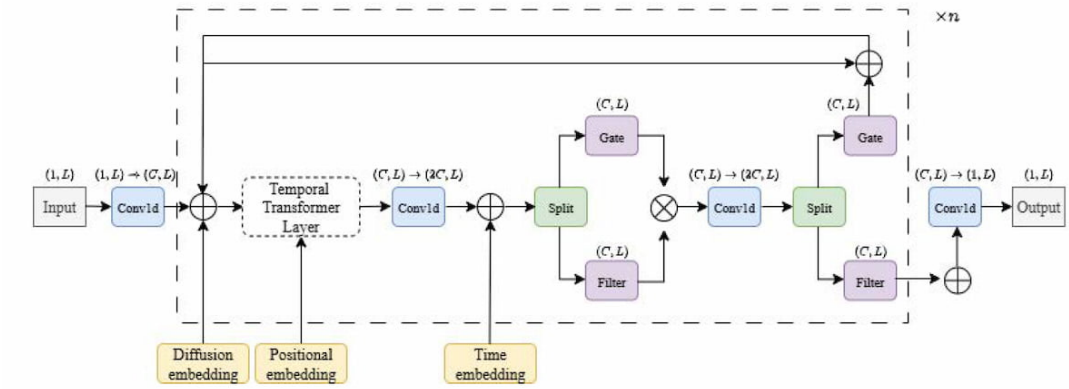


Figure 3.1: CSDI-based diffusion architecture. Source: Kim et al. [17].

Another inconsistency in Kim et al. [17] concerns the use of time, positional, and

diffusion-step embeddings. The text states that

> *These embeddings are added to the latent representation and passed through a Transformer block, which models global dependencies across time.*

However, Fig. 3.1 clearly shows that the time embeddings is provided only after the Transformer layer. This last interpretation was used. Moreover, based on the model architecture presented in Fig. 3.1, the diffusion-step, temporal, and positional embeddings were interpreted as being injected independently at every residual block, consistently with the approach adopted in CSDI [15]. Lastly, Kim et al. [17] describe the time embedding as combining sinusoidal functions with learnable vectors, while the positional embedding relies solely on fixed sinusoidal functions with no learnable parameters; this distinction suggests that the positional embedding is the only one among the three that contains no trainable weights.

3.5 EDM

The components described in this section apply exclusively to Phase 2.

Preconditioning Training a raw network $F_\theta(x_t; \sigma)$ directly as a denoiser is impractical because according to the VE forward process (Eq. 2.15) the inputs have variance $\sigma^2_{\text{data}} + \sigma^2$, which means that the network sees input that varies by orders of magnitude across the distribution of σ (different noise levels), as discussed in Sec. 2.5. EDM addresses both problems introducing preconditioned denoiser D_θ (Eq. 2.9).

Follows the role of each preconditioning parameter:

- $c_{\text{in}}(\sigma)$, input normalization: F_θ receives as input $c_{\text{in}}(\sigma) \cdot x_t$, and because, according to Eq. 2.15, $\text{Var}(x_t) = \sigma^2_{\text{data}} + \sigma^2$, then we scale the input to unit variance.

- $c_{\text{skip}}(\sigma)$, skip connection: is the Wiener filter coefficient (Sec. E.2.1), or the optimal linear estimator of x_0 given x_t. This is dervied from the MMSE argument, and it controls how much of the noisy input is passed directly to the output.

- $c_{\text{out}}(\sigma)$, output scaling: rescales the network output so that the effective training targets also have unit variance, similarly to $c_{\text{in}}(\sigma)$.

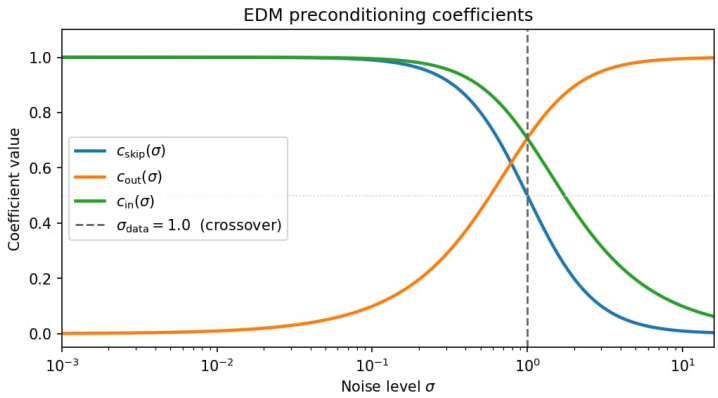


Figure 3.2: Preconditioning coefficients for $\sigma_{\text{data}} = 1$, with $\sigma_{\text{min}} = 0.002$ and $\sigma_{\text{max}} = 8.0$.

In summary, $c_{\text{skip}}(\sigma)\cdot x$ is the linear prior, while $c_{\text{out}}(\sigma)\cdot F_\theta(\cdot)$ is the non-linear correction on top of the prior. It is important to pay attention at the crossover point where $\sigma = \sigma_{\text{data}}$ and $c_{\text{skip}}(\sigma) = 0.5$, meaning both terms contributes equally. For instance, with $\sigma_{\text{data}} = 1$ then if $\sigma < 1$, the network makes tiny corrections and if $\sigma > 5$, the network reconstruct almost entirely from noise.

$c_{\text{noise}}(\sigma)$ is not derived from a requirement, but it is empirically chosen for numerical stability, and it is used to compresses the large dynamic range of σ into a bounded scalar, where the $\frac{1}{4}$ prevents saturation of the network's embedding layer. This actually is connected to CSDI's diffusion–step embedding (Sec.3.6), since it encodes the noise level for the network.

Sigma distribution The EDM framework was originally calibrated on ImageNet [34] with $\sigma_{\text{data}} = 0.5$, $P_{\text{mean}} = -1.2$, and $P_{\text{std}} = 1.2$. These defaults reflect the pixel-value statistics of natural images after unit-variance normalisation. OHLC log-returns differ in a relevant respect: after global standardisation, each channel has unit variance by construction, so $\sigma_{\text{data}} = 1.0$ is the appropriate choice. It is important to notice that P_{mean} shift the distribution, where lower values concentrate training on fine-grained reconstruction, and viceversa. P_{std} control the spread, where wider spread gives more uniform coverage. It is important that the σ_{max} used for the prior sampling in the reverse process, covers

approximately 95% or more of the distribution $\ln(\sigma) \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$. Whether P_{mean} and P_{std} also have empirically optimal values for this domain remains an open calibration question only partially addressed in this work.

Loss function EDM varies also the loss function characterization by moving away from noise prediction as in Phase 1 (Eq. 2.14), to a "clean" data prediction as in Eq. 2.10 and with different weighting, as discussed in Sec. 2.5.

3.6 Architecture

The CSDI-based backbone described in this section is shared by both phases.

U-Net architectures [44] have been widely adopted in diffusion models for image generation, where hierarchical downsampling and convolutional inductive biases provide strong local structure and multi-scale spatial representations. Financial time series, however, are not characterised by local spatial regularity but by long-range temporal dependencies and contemporaneous cross-channel relationships, for which the locality assumptions of convolutions is a poor fit. Transformers, by contrast, attend globally over the input, making them theoretically better suited to this domain.

As discussed in Sec. 2.6, CSDI factors the full $K \times L$ attention into a temporal Transformer operating over $L \times L$ and a feature Transformer operating over $K \times K$, reducing complexity from $\mathcal{O}((KL)^2)$ to $\mathcal{O}(L^2 + K^2)$. Beyond computational savings, the factored design is also structurally motivated: temporal attention captures *when* patterns occur within each channel, while feature attention captures *how* channels covary contemporaneously. Attending jointly over the time-feature product space would mix these two distinct dependency types, making the learned representations harder to interpret and potentially trickier to optimise.

Fig. 3.3 illustrates the overall flow of the denoiser. The noise level σ, the noised data x_t, the observed (conditioning) data, the `cond_mask` indicating which entries serve as conditioning inputs, and the `observed_tp` time indices are all injected into the network at the stages depicted in Fig. 3.5. These inputs are processed through a stack of n

`ResidualBlock` layers, producing the preconditioned output $F_\theta\big(c_{\text{in}}(\sigma)x;\, c_{\text{noise}}(\sigma)\big)$. The internal structure of each `ResidualBlock`, shown in Fig. 3.4, is described in detail below.

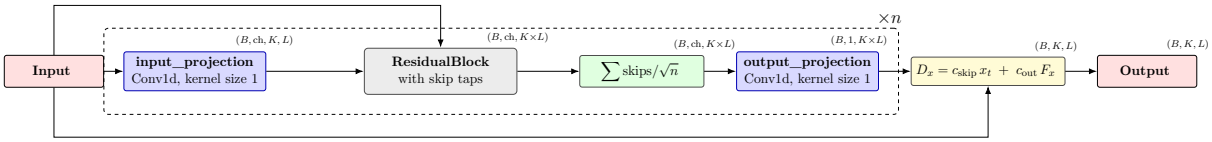


Figure 3.3: High-level overview of the CSDI denoising network. The two-channel input (details in Fig. 3.5) is projected into a latent space, processed by n stacked residual blocks with skip connections, and decoded back to the predicted clean signal.

Input projection A pointwise 1×1 convolution projects the two-channel input (signal and mask channels) into a d-dimensional latent space, where d is a free hyperparameter, lifting the two input channels into a richer representation.

Temporal and feature Transformer layers A new temporal (K, L, d) and feature (L, K, d) Transformer is initialised per residual layer rather than shared across them, Fig. 3.4. Weight sharing would constrain all layers to operate at the same level of abstraction, limiting the model's ability to build hierarchical representations, while independent weights allow each layer to specialise; consistently with CSDI implementation [15]. Multi-head attention [38] allows the model to attend to multiple dependency patterns simultaneously within each axis, for instance it could modeled short-range trend and longer-range volatility clustering along the temporal axis, or spread relationships along the feature axis. The structure is represented in Fig. 3.6.

Gated residual blocks Following the WaveNet structure [36], each block applies the element wise multiplication of $\sigma(\cdot) \odot \tanh(\cdot)$. The sigmoid $\sigma(\cdot)$ acts as the selection gate, producing a value in $[0, 1]$ that controls *how much* of the candidate update is passed forward. The $\tanh(\cdot)$ acts as the content filter, producing a bounded, signed activation in $[-1, 1]$ that encodes *what* information to write. Their product suppresses uninformative information while preserving significant features. This could be useful for FTS, since Signal-to-Noise Ratio (SNR) is often low and depends on the regime.

Embeddings Three embedding mechanisms operate at distinct levels of the architecture, each with a well-defined scope. The *diffusion-step embedding* (`diffusion_emb`: si-

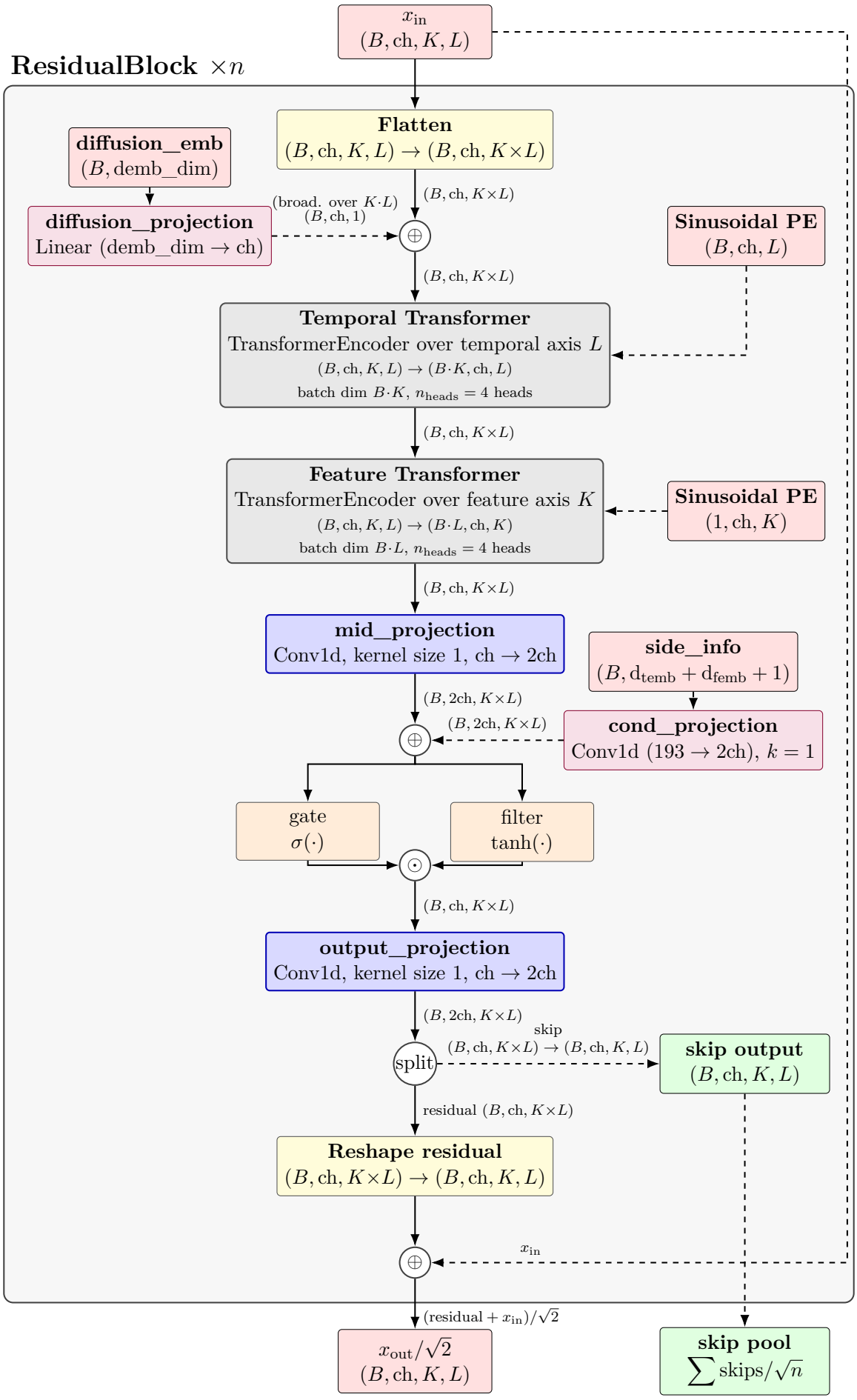


Figure 3.4: `ResidualBlock` architecture, CSDI–inspired. Each block emits a skip connection aggregated across all n_{layers} blocks (see Fig. 3.3).

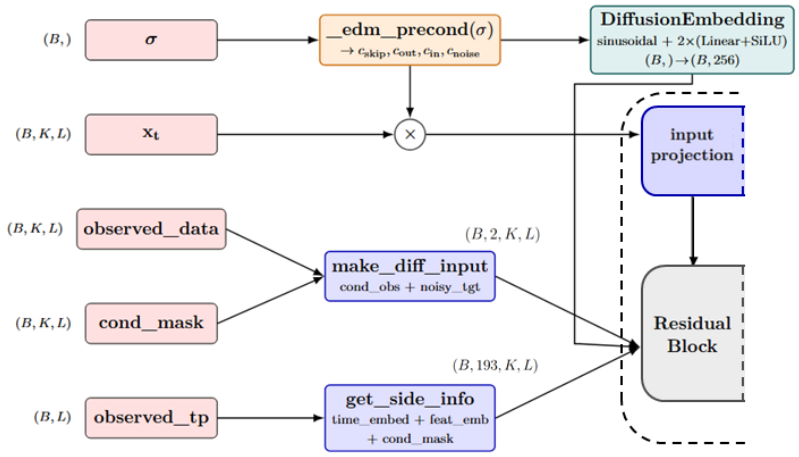


Figure 3.5: Representation of model inputs and how they are feeded into the Denoiser. `input projection` and `ResidualBlock` are those represented in Fig. 3.3.

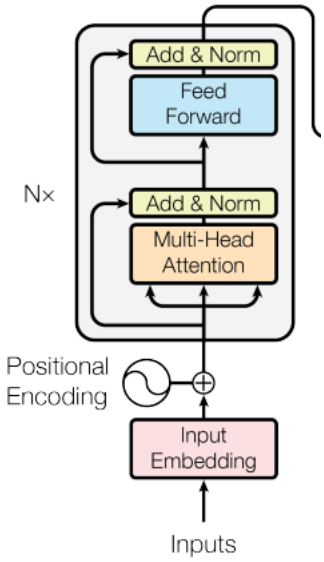


Figure 3.6: Transformer model architecture for the Encoder. Source: Vaswani et al. [38].

nusoidal basis projected through two linear layers with SiLU activations) encodes $c_{\text{noise}}(\sigma)$ in the EDM phase and the discrete step index t in the replication phase, telling the network the current noise level. The *side information embeddings* (`side_info`) carry two complementary signals: a time embedding (sinusoidal basis followed by a learnable linear projection) encodes the absolute calendar position of each time step, allowing the model to associate positions with recurring temporal patterns such as month-end or seasonal effects that are persistent in the training data; a feature embedding is a learned lookup table over channel indices $\{O, H, L, C\}$, giving the model an explicit representation of each channel, enabling channel-specific denoising. Finally, the *sinusoidal positional encoding* (`Sinusoidal PE`), following Kim et al. [17], encodes the relative position of each element within the sequence and is injected immediately before each Transformer call, both along the temporal and feature axes. Without this, the Transformer would be permutation-invariant, which would destroy the temporal order, which is essential for financial return sequences.

Residuals and skip connections Each `ResidualBlock` produces two outputs: a residual branch, added back to the block's input, and a skip branch, collected and

pooled across all n blocks before the final output projection. Following WaveNet [36] and CSDI [15], each `ResidualBlock` produces two outputs: a residual branch, added back to the block's own input, and a skip branch, aggregated across all n blocks before the final output projection. This design allows every block to contribute directly to the prediction, irrespective of its depth in the stack. The residual addition and the skip aggregation are rescaled by $1/\sqrt{2}$ and $1/\sqrt{n}$, respectively, to maintain stable signal variance as representations accumulate through successive blocks. The `mid_projection` is a pointwise convolution that doubles the channel dimension immediately before the side-information conditioning is added; this provides the extra capacity needed to later split the combined representation into the gate and filter branches of the gated activation unit.

3.7 Masking Strategies

This work employs two masking strategies: `unconditional` and `predict_close`.

Phase 1 The `unconditional` setting is used for Phase 1, where $K = 1$ and the model is trained on univariate log-returns with no conditioning information. The conditioning mask is $m^{\text{co}} = 0$ everywhere, and the target mask is $m^{\text{ta}} = 1$ everywhere, so the loss reduces to the standard denoising objective evaluated on all entries without any side information.

Remark on the `unconditional` label It is worth stressing that the `unconditional` mask eliminates value-level conditioning, since the OHL channels are zeroed in the model input, but it does not eliminate temporal conditioning. The time embedding (`observed_tp`), encoding the absolute calendar position of each time step as described in Sec. 3.6, is passed to the model regardless of mask mode and contributes a form of conditioning on calendar time. In this sense, both training phases mdoel a temporally conditioned distribution $p_\theta(x \mid \tau)$, where τ denotes `observed_tp`; the `unconditional` mask differs from `predict_close` only in not providing observed OHLC values.

Phase 2 When $K > 1$ we use the `predict_close` strategy that defines a determin-

istic mask:

$$m^{\text{co}}_{k,l} = 1 \quad \text{for } k \in \{O, H, L\}, \qquad m^{\text{co}}_{k,l} = 0 \quad \text{for } k = C, \qquad \forall\, l,$$

and correspondingly:

$$m^{\text{ta}}_{k,l} = 0 \quad \text{for } k \in \{O, H, L\}, \qquad m^{\text{ta}}_{k,l} = 1 \quad \text{for } k = C, \qquad \forall\, l.$$

This differs from the random masking used in the original CSDI [15], where conditioning and target sets are drawn stochastically at each training step. The `predict_close` mask is consistent with the OHLC information structure discussed in Sec. 3.1: the conditioning set $\{O, H, L\}$ is always observed before C within a trading session.

As introduced in Sec. 3.6, the two-channel input to the backbone concatenates a signal channel and a mask channel:

$$\underbrace{m^{\text{co}} \odot x^{\text{co}}_0 + (1 - m^{\text{co}}) \odot \big(c_{\text{in}}(\sigma) \cdot x_t\big)}_{\text{signal channel}}, \qquad \underbrace{m^{\text{co}}}_{\text{mask channel}}.$$

The scaling $c_{\text{in}}(\sigma)$ is applied only to the noisy target entries and not to the clean conditioning observations, since the latter carry no noise and no variance drift, rescaling them would destroy the conditioning signal. The conditional EDM loss therefore (Eq. 2.10) becomes:

$$\mathcal{L}_{\text{cond}}(\theta) = \mathbb{E}_{\sigma, x_0, \epsilon}\left[\lambda(\sigma)\, \frac{\sum_{k,l} m^{\text{ta}}_{k,l} \Big(D_\theta\big(x_t; \sigma \mid x^{\text{co}}_0, m^{\text{co}}, \tau, f\big) - x_0 \Big)^2_{k,l}}{\sum_{k,l} m^{\text{ta}}_{k,l}}\right],$$

where the denominator normalises by the number of active target entries, which in the `predict_close` setting is fixed and equal to L. The additional arguments τ (observed time positions `observed_tp`) and f (feature indices `feature_id`) are passed to D_θ to construct the side information embeddings, consistent with the CSDI implementation.

3.8 Sampling

Phase 1 As discussed in Sec. 2.4, the Euler–Maruyama method performs a first-order discretisation of the reverse SDE, at each step removing noise while injecting a small stochastic perturbation. Because it is first-order, the local truncation error is $\mathcal{O}(h^2)$, and acceptable sample quality typically requires hundreds to thousands of steps, implying a high number of NFEs. Consistently with Song et al. [11] the probability flow ODE (Eq. 2.6) is used in Phase 1, where the learned score $s_\theta(x_t, t)$ is substituted for the intractable $\nabla_x \log p_t(x_t)$.

Phase 2 In the EDM phase, the Heun second-order predictor–corrector sampler (Eq. 2.12) is used, which integrates the probability flow ODE rather than the reverse SDE. The corrector step requires double the number of NFEs per denoising step, but reduces the local truncation error to $\mathcal{O}(h^3)$.

A key advantage of the EDM framework is the decoupling of the sampler from the training objective: a model trained with the EDM loss can be evaluated with any compatible ODE or SDE solver without retraining. This is in contrast to DDPM [10], where the sampler and the noise schedule are tightly coupled to the training procedure.

Chapter 4

Data and Training Setup

This section specifies the dataset characteristics and training choices. A summary table is made available in Tab. 4.1.

4.1 Data

The dataset follows the source and structure reported by Kim et al. [17]. Specifically, the list of current S&P 500 members was taken as the starting universe[1] and tickers with non-standard symbols (e.g., BRK.B, BF.B) were excluded. For each remaining ticker, the maximum available daily price history was downloaded. The dataset was then filtered to retain only those stocks whose available history extends at least 40 years back. According to the authors:

> *This selection criterion ensures that the dataset encompasses diverse market regimes, including various boom and bust cycles, thus providing a rich context for training the generative model.*

For each selected stock, daily log-returns were computed as in Eq. 2.1, using the adjusted closing price on day t.

This selection is subject to some limitations. First, survivorship bias: only companies that remained listed for a period substantially longer than the average tenure in the

[1] The starting universe is taken as of the 10^{th} of April 2026.

S&P 500 index, which is 15 years [45], are included. The resulting universe therefore may understate the frequency of adverse market events associated with delistings and corporate failures. The second aspect regards the pre-decimalization process, which was mandated by the Securities and Exchange Commission in January 2000 and completed by the 9th of April 2001 [46]. Before this period, US stock prices were quoted in fractions, specifically 1/16th of a dollar, meaning that the minimum price movement needed to record an effective price change was 0.0625 dollars. After decimalization, this changed to 1 cent. This manifests as 7.1928% of all returns displaying an exact 0% daily return. A more in depth analysis is presented in Sec. A.1.

4.2 Preprocessing

Phase 1 Only univariate log-returns are used, as defined in Eq. 2.1, and no preprocessing is applied to them since no mention of it is made by Kim et al. [17], and the average log-return $\mu = 0.0005$ and standard deviations $\sigma = 0.020867$ do not conflict with prior sampling assumptions.

Phase 2 All four OHLC channels are expressed as log-returns relative to the previous closing price:

$$\hat{O}_t = \log\left(\frac{O_t}{C_{t-1}}\right), \qquad \hat{H}_t = \log\left(\frac{H_t}{C_{t-1}}\right), \qquad \hat{L}_t = \log\left(\frac{L_t}{C_{t-1}}\right) \tag{4.1}$$

This formulation is consistent with the Close log-return defined in Eq. 2.1, expressing all channels relative to the same reference price C_{t-1}.

Phase 2 A per-channel z-score normalisation is then applied independently to each of the four distributions while standardising it to zero mean and unit variance, so that $\sigma_{\text{data}} = 1$ holds by construction and network inputs remain numerically stable across channels. However, because each channel is scaled by its own standard deviation, the ordering constraint $L_t \leq \min(O_t, C_t) \leq \max(O_t, C_t) \leq H_t$ is not preserved after normalisation. This is a structural limitation acknowledged throughout the evaluation.

Four tickers were excluded from the dataset prior to training: BEN, HUBB, NVR,

and WMB. Each exhibits absolute log-returns exceeding 100%, with values ranging from -157% to $+330\%$. Returns below -100% are theoretically inadmissible, because they violate price non-negativity. Such observations are therefore artifacts of the split- and dividend-adjustment procedure described in Sec. 2.1, rather than genuine market events. Exclusion was preferred over imputation or smoothing, both of which would introduce their own distortions. This choice is consistent with the likely preprocessing of Kim et al. [17]: after removing these four tickers, the empirical distribution aligns more closely with Figure 8 of that paper, which is used here as a reference baseline.

4.3 Sliding Window Construction

Training sequences are constructed via a sliding window over the full dataset. Two configurations are used across the two experimental phases:

- **Phase 1, replication:** window length $L = 2048$ and stride $s = 400$, yielding 5,568 training windows over the full dataset.
- **Phase 2, EDM-CSDI:** window length $L = 512$ and stride $s = 100$, yielding $25,226$ maximum possible windows if no validation is considered `val_split_ratio` $= 0$.

A window of length $L = 2048$ spans approximately ten years of daily returns and is therefore well-suited to capturing long-term trends. However, the temporal Transformer incurs an $\mathcal{O}(L^2)$ cost in sequence length, making it computationally expensive. The shorter $L = 512$ setting was adopted for Phase 2 due to hardware constraints imposed by MIG partitioning of the A100 GPU by the university's High-Performance Computing (HPC) policy. The full memory analysis is provided in Sec. F.3.

Phase 1 Given the window length ($L = 2048$), the number of non-overlapping windows per ticker is small, making a strict train–validation split undesirable. Validation windows were therefore extracted by random holdout, supposedly consistent with the implementation of Kim et al. [17], since such design choice was not specified. Due to the stride, partial overlap between training and validation windows exists.

For the GBM-inspired SDE variant, each window was additionally mean-centred prior to training by subtracting the sample mean of the window. This transformation does not conflict with the generative setting, because no forecasting is performed. In addition, any daily log-returns can be easily recovered by $r_{\tau+k} = C_{\tau+k} - C_{\tau+k-1}$, except for the first element.

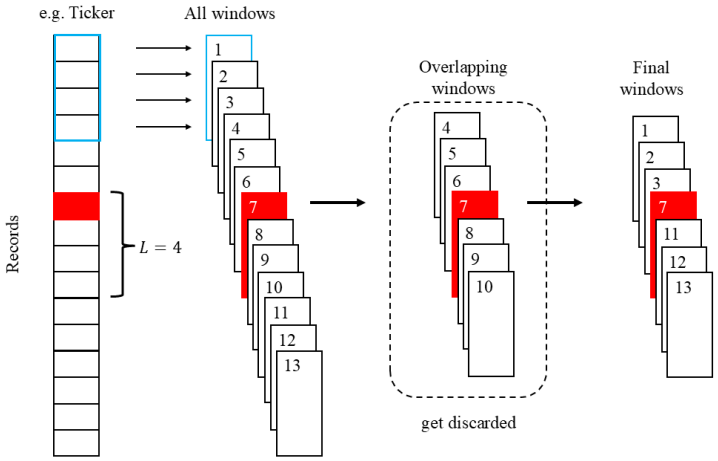


Figure 4.1: Phase 2 selection of Validation windows process. Example with window length $L = 4$, stride $s = 1$, and `val_split_ratio`$= 0.25$, where the 7^{th} ticker index (red block) is drawn at random.

Phase 2 In order to assess the discriminative power of the conditioning information, validation windows were fully held out: for each ticker a contiguous block of records amounting to a `val_split_ratio` fraction was reserved starting at a random index, and every window straddling this interval was excluded from the subsequent training (Fig. 4.1). This per-ticker approach enables the model to see diverse market conditions across all stocks without concentrating only on the longer-lasting tickers. The validation split ratio was set to 5% (239 windows), because, since windows overlap accordingly to the stride, each additional validation window required roughly 10 training windows to be discarded. For 5% validation split, $2,129$ windows are discarded, leaving $22,858$ for training.

4.4 Training Configuration

This section summarises the most relevant training configuration and design choices. Explicit formulations for the optimiser and learning rate schedule are given in Sec. F.1,

and full hyperparameter details in App. F.

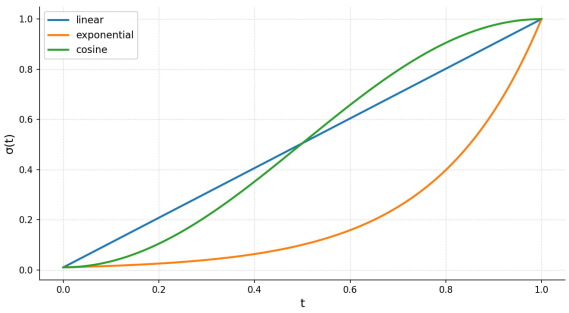
Optimiser The optimiser used throughout is `AdamW` [47], following the choice of CSDI [15]. It applies weight decay directly to the parameters rather than through the gradient, preventing the interaction between ℓ_2 regularisation and the adaptive scaling of standard Adam. This decoupling results in more consistent regularisation across parameters with different gradient magnitudes.

Learning Rate Scheduler The learning rate follows cosine annealing [48], allowing larger updates early in training and progressively smaller updates near convergence. Reduce-on-Plateau was also evaluated but yielded consistently worse training dynamics and was not adopted. For Phase 1, $\eta_{\max} = 5 \times 10^{-4}$ and $\eta_{\min} = 1 \times 10^{-6}$; for Phase 2, $\eta_{\max}$ is lowered to 1×10^{-4} to reflect the smaller effective gradient variance under EDM loss weighting.

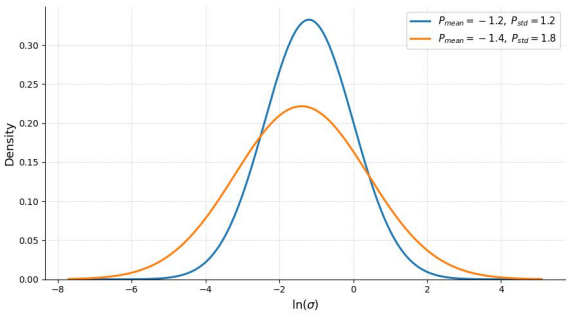
Model Architecture Both phases share the same backbone: 128 `channels`, 6 `layers`, a 256-dimensional diffusion-step and time embedding, and a 64-dimensional feature embedding, following Kim et al. [17]. This configuration was selected empirically: a 64-channel variant was less expressive, while a 256-channel variant offered no measurable improvement in generation quality at substantially higher compute cost.

The number of input/output channels (K) also varies across configurations: (K=1) for the unconditional models (Close log-returns only), and (K=4) for the conditional EDM-CSDI model, which jointly processes Open, High, Low, and Close log-returns with `predict_close` masks. This increase is reflected in the total parameter count, which grows from approximately 2.2 million in Phase 1 to approximately 3.4 million in Phase 2 (Sec. F.2).

Noise Schedule The noise hyperparameters differ between phases. In Phase 1, they define the range of the forward-process noise: VE-style diffusion uses $\sigma_{\max} = 1$ and $\sigma_{\min} = 0.01$; GBM-style diffusion, operating on cumulative log-returns, uses $\sigma_{\max} = 10$ and $\sigma_{\min} = 0.1$; and the VP formulation is parameterised by $\beta_{\max} = 8$ and $\beta_{\min} = 0.01$. These values were determined empirically by testing the forward process to achieve prior



(a) Sigma Schedule ($\sigma_{\min} = 0.01$, $\sigma_{\max} = 1.0$) for $t \sim \mathcal{U}(0, 1]$ and noise $\sigma(t)$



(b) $\ln(\sigma) \sim \mathcal{N}\left(P_{\text{mean}}, P_{\text{std}}^2\right)$

Figure 4.2: Noise schedules: Phase 1 (left), Phase 2 (right).

matching while preserving as much signal as possible, with emphasis on the former.

In Phase 2, the EDM log-normal sampling distribution $\ln \sigma \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$ replaces the forward-process schedule entirely. The values $P_{\text{mean}} = -1.4$ and $P_{\text{std}} = 1.8$ were selected via grid search over $P_{\text{mean}} \in \{-0.8, -1.0, -1.2, -1.4, -1.6\}$ and $P_{\text{std}} \in \{1.0, 1.2, 1.4, 1.6, 1.8, 2.0\}$ based first on validation loss and later on generative samples. This was influenced by Phase 1, where small $t-$regimes were identified as the hardest to predict. Lower P_{mean} concentrates more training samples on low-σ regions and higher P_{std} helps cover the tails. Above $P_{\text{std}} = 1.8$, no significant generation gains were observed.

Training parameters The batch size is 64 across both phases. Phase 1 trains for 1000 epochs, as in Kim et al. [17], while Phase 2 uses half that budget, which reflects the hardware constraints and the larger number of available training windows, which increases the number of gradient steps per epoch.

Sampling parameters For Phase 2, $\sigma_{\max}$ and $\sigma_{\min}$ were selected to cover the support of $\ln \sigma \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$, requiring $\sigma_{\max} \geq \exp(P_{\text{mean}} + n\, P_{\text{std}})$ for a chosen coverage multiplier n. With $n = 5$, $\sigma_{\max} = 8.0$ with $\sigma_{\min} = 0.002$ fixed throughout.

Category	Parameter	Phase 1 — Replication	Phase 2 — EDM-CSDI
Data & preprocessing	Channels K	$K = 1$ (close log-return)	$K = 4$ (OHLC)
	Preprocessing	Raw close log-return	OHLC channels as log-returns relative to C_{t-1}; per-channel z-score normalisation
	Conditioning	Unconditional; $m^{co} = \mathbf{0}$	`predict_close`: $\{O, H, L\}$ observed, $\{C\}$ masked
	Window length L	2048 (≈8 years)	512 (≈2 years)
	Stride s	400	100
Architecture	Model channels	128	
	Layers	6	
	Diffusion emb.	256	
	Time emb.	128	
	Feature emb.	64	
Noise schedule	$\sigma_{\max}$ / $\sigma_{\min}$	VE: $\sigma_{\max}$=1, $\sigma_{\min}$=0.01; GBM: $\sigma_{\max}$=10, $\sigma_{\min}$=0.1	—
	$\beta_{\max}$ / $\beta_{\min}$	VP: $\beta_{\max}$=8, $\beta_{\min}$=0.01	—
	P_{mean} / P_{std}	—	P_{mean}=−1.4, P_{std}=1.8
Training	Validation wind.	Partly overlapping	Fully held-out
	LR schedule	cosine annealing	
	$\eta_{\max}$ / $\eta_{\min}$	5×10^{-4} / 1×10^{-6}	1×10^{-4} / 1×10^{-6}
	Epoch	1000	500
	Batch size	64	
	Optimizer	`AdamW`	
	Early-stopping	—	validation-loss; 200 epochs
Sampling	Method	Euler–Maruyama SDE Probability-flow ODE	Heun ODE Stochastic Heun SDE
	$\sigma_{\max}$ / $\sigma_{\min}$	from *Noise schedule*	$\sigma_{\max} = 8.0$, $\sigma_{\min} = 0.002$

Table 4.1: Training setup for Phase 1 (replication of Kim et al. [17]) and Phase 2 (EDM-CSDI). Parameters shared across both phases appear in merged cells. *emb.* is short for embeddings and *wind.* for windows.

Chapter 5

Evaluation Methodology

This chapter presents the evaluation methodology applied to both unconditional and conditional generated data. Multiple metrics are used to assess distributional similarity, moment reproduction, path-wise behaviour, and conditional predictive quality, combining quantitative metrics with visual insights.

5.1 Distributional Similarity

To assess the similarity between the real and generated data distributions, two complementary metrics are reported: the Wasserstein distance [49], which captures global distributional shifts, and the Kolmogorov–Smirnov (KS) test [50], which provides a non-parametric hypothesis test for distributional differences. Both metrics are computed on the log-return datasets $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{gen}}$.

Kolmogorov–Smirnov The KS statistic measures the maximum distance between the empirical cumulative distribution functions of two samples. Given two empirical cumulative distributions $F(x)$ and $G(x)$, it is defined as:

$$D_{\text{KS}} = \sup_x |F(x) - G(x)|$$

where $\sup_x$ denotes the supremum over all values of x. The KS test is advantageously nonparametric and sensitive to both location and shape differences; however, its maximum

deviation is dominated by the bulk of the distribution and is relatively insensitive to tail discrepancies.

Wasserstein Distance The Wasserstein distance measures the minimal effort required to transform one distribution into another. For two cumulative distributions $F(x)$ and $G(x)$, it is defined as:

$$W_1(F, G) = \int_{-\infty}^{\infty} |F(x) - G(x)| \, dx$$

Unlike the KS statistic, which reports only the maximum deviation, W_1 integrates the discrepancy across the entire support, providing a global measure of distributional mismatch.

5.2 Aggregate Statistics

To obtain a more comprehensive view of generative quality, the pathwise mean, variance, skewness, and kurtosis are computed for each window in $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{gen}}$. Scalar metrics such as KS and W_1 flatten temporal structure; pathwise higher-order moments instead provide direct evidence of whether the model reproduces non-Gaussian behaviour and the stylized facts of financial time series, with particular attention to skewness and excess kurtosis. The latter must be interpreted with caution, since, as discussed in Sec. 2.1, the fourth moment is likely unbounded for FTS. Hence, even if computationally finite on a limited dataset, it may be infinite for the true underlying population.

5.3 Visualizations

Complementary with the statistics we plotted different sets of graphs.

Quantile–Quantile (QQ) Plots Quantile comparison between pooled generated and real returns are useful to check whether a model who can match the bulk if it still misses the tails. By plotting quantiles of D_{gen} again the quantiles of D_{train}.

ECDF Plots It is a visual complement to the KS test. Plot the Empirical Cumulative Distribution Functions $\hat{F}_{\text{gen}}$ and $\hat{F}_{\text{data}}$ together with the KS gap marked. This

makes the KS statistic more interpretable.

Unnormalized Paths Invert the normalisation to recover approximate price-level trajectories. This serves primarily as a sanity and plausibility check of the model's ability to generate coherent price levels, where "unnormalize" denotes applying the inverse of the global z-score.

5.4 Financial Time Series Evaluation

For financial time series, a set of tools is used to compute the tail distribution, the autocorrelation function (ACF), the leverage effect, and a power-law tail exponent α. These metrics are implemented following the functions provided in [51], which was also adopted by Kim et al. [17], enabling a direct comparison with their results. For consistency, the power-law coefficient is computed using the same Python package: `powerlaw` by Alstott et al. [52]. The definitions of the tail distribution, α, volatility clustering, and the leverage effect are provided in Sec. 2.1. Following Kim et al. [17], the heavy-tail diagnostic and the exponent α are computed on the positive (right) tail only.

Following FinGAN [51] and Kim et al. [17], in Phase 1 the maximum lag for the leverage effect is set to 100 and for volatility clustering to 1000. In Phase 2, the maximum lag for volatility clustering is reduced to 256 for estimation stability. The ACF at lag k is estimated over $L - k$ observation pairs; for a window of length $L = 512$, lags approaching L leave too few pairs to produce reliable estimates, rendering a maximum lag of 512 impractical. Setting the upper bound to $L/2 = 256$ ensures that at least half the window's values contributes to each estimate.

5.5 Conditional Evaluation

When evaluating the conditional generation we are evaluating the quality of the conditional generation by focusing on CRPS (Continuous Rank Probability Score) and interval coverage.

CRPS This metrics is measured as:

$$\mathrm{CRPS}(\hat{F}, y) = \mathbb{E}_{\hat{F}}\left[|X - y|\right] - \tfrac{1}{2}\,\mathbb{E}_{\hat{F}}\left[|X - X'|\right]$$

where X, X' are independent draws from the predictive ensemble $\hat{F}$ and y is the observed Close. CRPS [53] is a proper scoring rule: it penalises inaccuracy (the first term) and rewards spread (the second term lowers CRPS as ensemble diversity grows). It reduces to MAE when the distribution collapses to a point forecast, so it can be interpreted as a calibration-adjusted MAE, where $0 \leq \mathrm{CRPS} \leq \mathrm{MAE}$ (proof in Sec. E.3). This score is also used in CSDI [15]. Nevertheless, a raw CRPS value is difficult to interpret in isolation, so it was constructed a *climatology* baseline whose ensemble is formed by drawing, with replacement, from the full pool of observed Close log-returns, ignoring the OHL conditioning. A second baseline fits an OLS regression of Close on the same-day Open, High, and Low log-returns, with its ensemble formed by adding bootstrapped residuals to the OLS point forecast. Relative skill is summarised by the CRPS skill score $\mathrm{SS}_{\mathrm{CRPS}} = 1 - \overline{\mathrm{CRPS}}_{\mathrm{model}}/\overline{\mathrm{CRPS}}_{\mathrm{baseline}}$, where 1 is perfect, 0 means no improvement over the uninformative baseline.

Interval Coverage This metrics is measure for nominal coverage levels, $1 - \alpha \in \{0.5, 0.8, 0.9, 0.95, 0.99\}$. Once defined the nominal level investigated the empirical coverage is:

$$\mathrm{coverage}_{1-\alpha} = \frac{1}{N}\sum_{t=1}^{L}\sum_{n=1}^{N_t} \mathbf{1}\left[y_{n,t} \in \left[q_{\alpha/2}(n,t),\ q_{1-\alpha/2}(n,t)\right]\right]$$

where q are ensemble quantiles, which means they are quantiles computed from the set of multiple forecasts the model produces for the same target value, or set of conditioning information. A well-calibrated model has the empirical value approximately identical to nominal. Positive gap means underconfidence (intervals too wide), negative gap means overconfidence (intervals too narrow).

Chapter 6

Experiments and Results

6.1 Overview

This chapter presents the experimental evaluation of the two modeling phases introduced in Ch. 3 and 4, assessed with the metrics defined in Ch. 5. Two research questions guide the analysis:

1. **RQ1. Phase 1 — Replication** Can a CSDI-based diffusion architecture, trained under the SDE families proposed by Kim et al. [17], learn the distributional properties of S&P 500 log-returns and reproduce their core stylized facts, including heavy tails, volatility clustering, and the leverage effect?

2. **RQ2. Phase 2 — EDM-CSDI** Does embedding the CSDI backbone within the EDM framework of Karras et al. [16] improve the distributional fidelity of unconditional Close log-return samples over the Phase 1 SDE baseline? And does the further introduction of same-day Open, High, and Low log-returns as conditioning variables yield additional improvements in sample quality and probabilistic sharpness?

The chapter is organised in four sequential parts. The first part discusses two baseline models, GARCH-X and Merton-X, which serve as a comparison against parametric models that are widely accepted in industry, fully understood, and probabilistically tractable.

The second part covers Phase 1, which concerns the replication of the work of Kim et al. [17] on unconditional univariate generation of S&P 500 Close log-returns. The third part investigates the contribution of the EDM framework when applied to Kim et al.'s CSDI-based architecture, retaining unconditional generation while introducing EDM preconditioning and loss weighting. The fourth and final part presents the full EDM-CSDI model, which extends this architecture with a feature-wise Transformer to incorporate same-day Open, High, and Low conditioning.

Subsequently, other, smaller experiments were carried out to compare the models' behavior under different sampling procedures.

A design choice of particular relevance to Phase 2 is that validation windows are strictly held out: 239 windows, corresponding to approximately 1% of all windows and 5% of unique rows. While this distinction is less critical for unconditional generation, since date ranges hidden in the validation set may still appear across overlapping training windows, it is fundamental for assessing conditional learning. The reasons behind these choices are discussed in Sec. 3.7 and Sec. 4.3.

Moreover, because of the way the data are processed to build windows, the 'Reference' values vary slightly according to the specific configuration being analyzed: raw reference, windows with length $L = 2048$ and stride $s = 400$, or windows with $L = 512$ and stride $s = 100$.

All power-law exponents $\hat{\alpha}$ are estimated on a subsample of 300,000 observations.[1] Both D_{KS} and W_1 are computed on a subsample matched to the smaller of the Reference dataset and the generated set, using a fixed seed for reproducibility.[2] For the `Validation` split, this amounts to 611,840 observations [3] For all other cases, the full Reference dataset of 11,825,664 observations is used.[4] Excess kurtosis (Kur.) is reported relative to the Gaussian baseline. Throughout this chapter, the heavy-tail stylized fact is assessed on

[1] `powerlaw` by Alstott et al. [52] was used for estimating $\hat{\alpha}$.

[2] For D_{KS} and W_1, the respective `scipy.stats` functions were used: `ks_2samp` and `wasserstein_distance` [54].

[3] The product of the number of validation windows, the sequence length, and the number of samples per window ($239 \times 512 \times 5$).

[4] The product of the total number of windows (training and validation combined) and the sequence length ($23{,}097 \times 512$).

the positive (right) tail only, following Kim et al. [17] (Sec. 5).

6.2 Baseline Models: GARCH-X and Merton-X

GARCH-X and Merton-X models, defined in Sec. 3.2, were trained on the log-transformed data described in Sec. 4.1, following Eq. 4.1. A GARCH-X and a Merton-X model were fit specifically for each ticker. Details about the design choices and constraints applied are available in Sec. D.5.4.

Tab. 6.1 reports distributional statistics (full table in the appendix, Tab. B.1):

Model	**Mean**	**Std**	**Skew.**	**Kur.**	$q_{0.01}$	$q_{0.99}$	$D_{\mathrm{KS}} \downarrow$	$W_1 \downarrow$	$\hat{\alpha}$
Reference	0.0005	0.0232	−0.5773	38.5	−0.0626	0.0649	—	—	4.532
GARCH-X	0.0009	0.0230	−7.5635	11 021.7	−0.0577	0.0597	0.0540	0.0012	4.188
Merton-X	0.0005	0.0364	1.2102	3736.4	−0.0584	0.0598	0.0423	0.0018	3.344

Table 6.1: Aggregate distributional statistics for Reference, GARCH-X, and Merton-X over pooled log-returns. Reference uses the full dataset; GARCH-X and Merton-X use a 10M-observation subsample out of 50M total.

From the statistics, it can be observed that these models, which have been refined within working windows, can match the central moments and bulk quantiles (Mean, Std, $q_{0.01}$, $q_{0.99}$) reasonably well, but higher moments remain severely misestimated, with excess kurtosis that is overstated by roughly two to three orders of magnitude (11021.7 and 3736.4 vs. 38.5). This is mostly because, even if strongly bounded by construction (Sec. D.5.3), the generation still tends to drift away for a few samples (less than 0.00% for GARCH-X and 0.02% for Merton-X have $|r_t| > 1$, see Sec. B.1). The same instability surfaces in the skewness, where the two models fail in opposite directions: GARCH-X amplifies the reference's negative asymmetry by an order of magnitude (−7.56 vs. −0.58), whereas Merton-X reverses its sign entirely (+1.21). Moreover, Fig. 6.1 shows that heavy-tail distribution and volatility clustering are partially reproduced by both baselines, suggesting these stylized facts are within reach of parametric models calibrated per ticker. For Merton-X, however, this apparent persistence is attributable to the autocorrelation of the exogenous OHLC conditioning regressors $\mathbf{q}_t$ (Eq. 3.2), rather than to any endogenous

variance-feedback mechanism. The model remains structurally incapable of generating volatility persistence without this conditioning (Sec. D.4). Nevertheless, the leverage effect is clearly missed: the symmetric variance specifications of both models (Sec. 3.2) make this a structural impossibility.

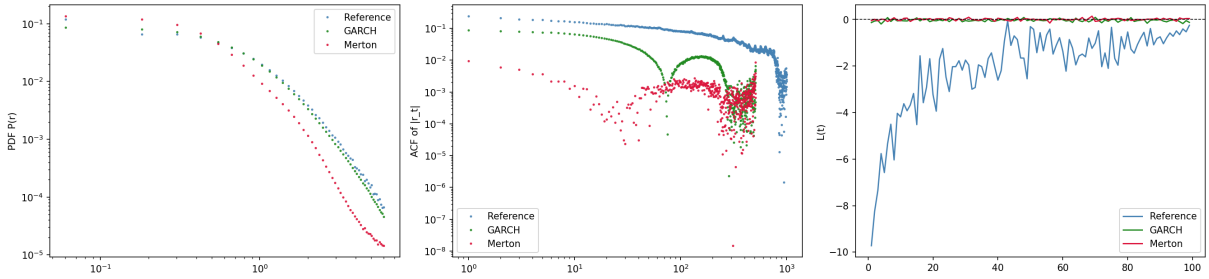
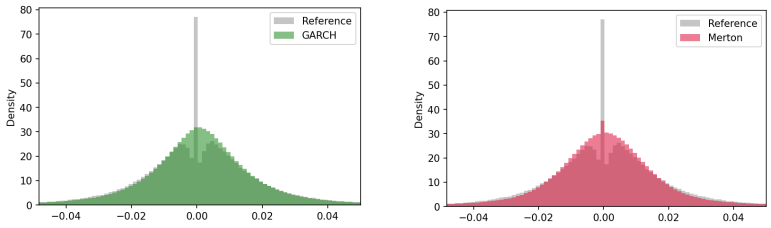


Figure 6.1: Stylized facts for Reference, GARCH-X, and Merton-X. From left to right, heavy-tail distribution (positive tail); volatility clustering (ACF); and leverage effect. Sec. 2.1 for details.

One key feature of the Reference data distribution is the atypical spike of returns at 0%, visible in Fig. 6.2. This is due to the pre-decimalization effect described in Sec. 4.1 and Sec. A.1. GARCH-X and Merton-X smooth out this spike.



(a) GARCH-X distribution $q_{0.02} - q_{0.98}$

(b) Merton-X distribution $q_{0.02} - q_{0.98}$

Figure 6.2: Marginal distribution against Reference (in gray).

6.3 Phase 1: Replication

Phase 1 aims to replicate the generative approach of Kim et al. [17] as detailed in the column "Phase 1" of Tab. 4.1. For each model, a dataset of 512 windows of length $L = 2048$ ($1,048,576$ observations for model) was generated using the probability-flow ODE (Eq. 2.6) with $2,000$ discretization steps. The number of samples was chosen for computational feasibility. Full results are presented in the appendix, Sec. B.2.

Aggregate Statistics and Distributional Similarity

Configuration	**Mean**	**Std**	**Skew.**	**Kur.**	$q_{0.01}$	$q_{0.99}$	$D_{\mathrm{KS}} \downarrow$	$W_1 \downarrow$	$\hat{\alpha}$
Reference	0.0005	0.0208	−0.5387	36.0	−0.0562	0.0583	—	—	4.378
VE + Linear	0.0004	0.0270	−0.1712	7.0	−0.0649	0.0659	0.121	0.007	5.429
VE + Exponential	0.0005	0.0190	−0.0860	14.3	−0.0479	0.0492	0.037	0.002	4.461
VE + Cosine	0.0008	0.0221	−0.5979	21.4	−0.0558	0.0575	0.062	0.003	4.448
VP + Linear	0.0023	0.0446	6.3270	529.8	−0.0739	0.0891	0.142	0.011	4.883
VP + Exponential	0.0011	0.0546	0.1530	5.6	−0.1365	0.1539	0.206	0.025	5.535
VP + Cosine	0.0003	0.0235	−0.2985	15.7	−0.0607	0.0627	0.060	0.003	4.368
GBM + Linear	0.0001	0.2797	−0.0023	0.0	−0.6519	0.6516	0.419	0.210	13.055
GBM + Exponential	0.0001	0.1453	−0.0122	0.0	−0.3385	0.3379	0.370	0.101	10.935
GBM + Cosine	0.0000	0.1437	−0.0130	0.0	−0.3350	0.3342	0.371	0.102	11.134

Table 6.2: Aggregate distributional statistics for Phase 1 generated close log-returns and the training reference, computed over pooled window-level returns ($L = 2048$, stride= 400).

The statistics of Tab. 6.2 illustrate the ability of the CSDI model to learn the bulk of the FTS distribution. Specifically, for the VE family and VP+Cosine, quantiles between 1% and 99% are well reproduced and the standard deviation remains close to the reference. VP+Linear, VP+Exponential, and all GBM configurations instead show substantial deviations in scale (Std up to $\sim$13$\times$ the reference for GBM). Nevertheless, across all models two limitations still exist: the inability to learn the peculiar zero-return spike and a failure to capture the heavy-tailed distribution of FTS. Indeed, 'Kur.' and 'Skew.' are underestimated or unstable across all configurations. The extreme 'Kur.' of 'VP + Linear' (529.8) is driven by a small number of generated windows with extreme values (Min = −3.79, Max = 5.30, Tab. B.3), an order of magnitude beyond any other configuration, and since excess kurtosis is highly sensitive to such outliers, this statistic should not be read as representative of VP+Linear's typical sample quality.

Most importantly, interpreting the log-returns as cumulative did not yield any useful advantage. On the other hand, for the GBM family the learning appeared rudimentary: the generated values display near-zero excess kurtosis and skewness, indicating they are statistically close to Gaussian rather than to the heavy-tailed reference distribution, and casting doubt on the model's ability to discern signal from noise in these configurations.

The inflated tail exponents corroborate this ($\hat{\alpha} \approx 11$–13 vs. reference 4.38), and sit closer to the values Kim et al. [17] report for the VE SDE family ($\alpha = 8.96, 8.49, 4.14$) than to empirical data, despite markedly different stylized facts (Figs. B.5, B.6, B.7).

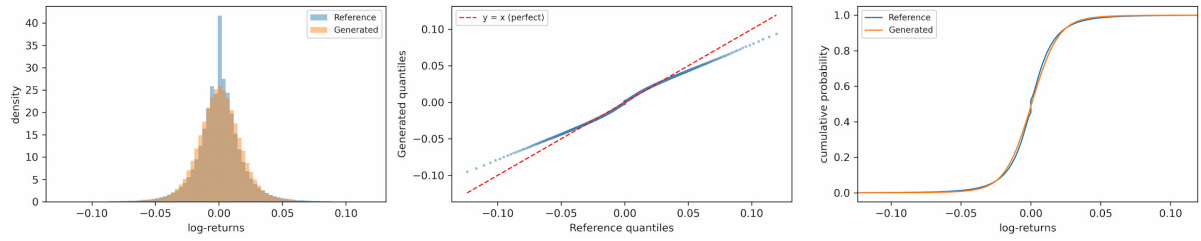


Figure 6.3: From left to right, marginal distribution, QQ-plot, and ECDF plot. All three graphs are of VE SDE with Exponential noise schedule and the displayed quantiles are $q_{0.001} - q_{0.999}$.

From Tab. 6.2, the VE family achieves the best overall D_{KS}/W_1 (VE+Exponential), and within VE the linear schedule clearly underperforms its cosine and exponential counterparts. More generally, the linear schedule is never the best-performing choice on D_{KS}/W_1 in either SDE family, suggesting it is comparatively the least useful noise schedule for this architecture, even though its effect is less pronounced within VP. Nevertheless, even the VE approach fail in tail reconstruction (as visible in the QQ-plot of Fig. 6.3) and in matching the zero-return spike (as shown by the ECDF in Fig. 6.3), cutting through the central vertical jump.

Stylized Facts The estimate of the α component is consistent with Kim et al., who reported $\alpha \approx 4.35$, while here it was estimated at 4.37. An estimate below this benchmark indicates heavier tails in the data, and viceversa, since α is defined as $P(|r| > x) \sim x^{-\alpha}$.

Although the parametric baselines remain competitive on pooled W_1 (0.0012 and 0.0018 vs. 0.002 for the best Phase 1 model, Tabs. 6.1 and 6.2), the core advantage of Phase 1 models is their ability to recover the stylized facts of the training windows, especially the leverage effect, even if this can be reproduced only at a lower scale than the reference, as visible in Fig. 6.4.

Summary In conclusion, the models appear to work under certain conditions and provide a good basis for further extensions, especially for VE SDE. Nevertheless, the GBM approach must be discarded in its current form and it requires further clarification

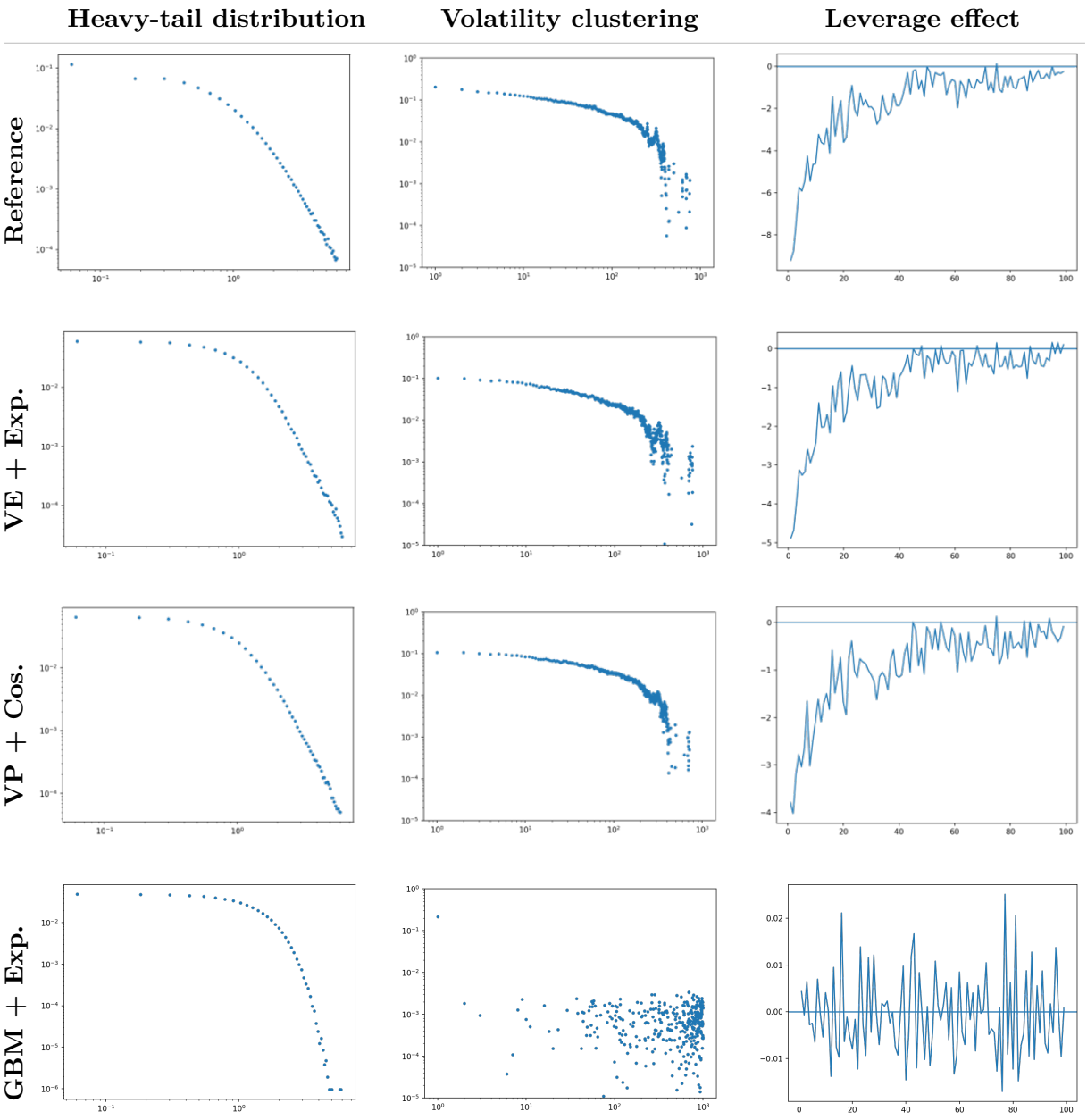


Figure 6.4: Stylized facts across Phase 1 SDE families and the empirical training reference on windows with $L = 2048$. *Columns* (left to right): heavy-tail distribution on a log–log scale with estimated power-law exponent $\hat{\alpha}$, Tab. 6.2; autocorrelation of absolute returns on a log–log scale (volatility clustering) with lag = 1000; lead–lag correlation $L(k)$ (leverage effect), with lag = 100. The lag choices match Stakahashy et al. [51] for FinGAN and Kim et al. [17].

by the authors Kim et al. [17].

6.4 Phase 2: EDM-CSDI

This section explores the results regarding the EDM-CSDI framework. The first part evaluates unconditional generation, analysing the generalization of Phase 1 models, and then training another model, EDM+CSDI (Unc.), with the parameters defined in Tab. 4.1 (Phase 2 column), but using the `unconditional` mask instead of `predict_close` (Sec. 3.7) and with the number of features equal to $K = 1$ [5]. For a fair comparison with Phase 1, the two most promising models, VE+Exponential and VE+Cosine, were retrained with the same settings as in Phase 1 (Tab. 4.1), but with window length $L = 512$ and stride $s = 100$. The second part of the section explores conditional generation.

6.4.1 Phase 1 Champion Models

A total of $20,000$ windows each were generated for VE+Exponential and VE+Cosine. In this case, a second-order Heun ODE probability-flow sampler (Eq. 2.12) with 200 steps was used to match EDM sampler. The following aggregate statistics are reported for the entire generated set. A complete table is presented in the appendix, Tab. B.4.

Aggregate Statistics and Distributional Similarity

Configuration	Mean	Std	Skew.	Kur.	$q_{0.01}$	$q_{0.99}$	$D_{\mathrm{KS}} \downarrow$	$W_1 \downarrow$	$\hat{\alpha}$
Reference	0.0005	0.0209	−0.5864	39.5	−0.0563	0.0583	—	—	4.418
VE + Exponential	0.0003	0.0199	−0.2382	14.3	−0.0508	0.0514	0.0436	0.0019	4.634
VE + Cosine	0.0004	0.0209	−0.2368	16.6	−0.0535	0.0553	0.0458	0.0020	4.577

Table 6.3: Aggregate distributional statistics for the Phase 1 champion models (VE+Exponential, VE+Cosine) generalized to $L = 512$, stride= 100.

As visible in Tab. 6.3, generalizing the Phase 1 champions does not produce a systematic improvement: D_{KS}/W_1 remain in a similar range to before (e.g., VE+Exponential 0.0436/0.0019 vs. 0.037/0.002 in Tab. 6.2), and the excess kurtosis of both models remains

[5] As clarified in Sec. 3.7, the `unconditional` label refers only to the absence of value-level OHLC conditioning; the time embedding still encodes the calendar position of each window, so the model strictly learns $p_\theta(x \mid \tau)$ rather than the fully marginal $p(x)$. App. C.2 confirms this empirically: EDM-CSDI (Unc.) recovers the historical ordering of volatility regimes via this temporal embedding alone (correlation is 0.69).

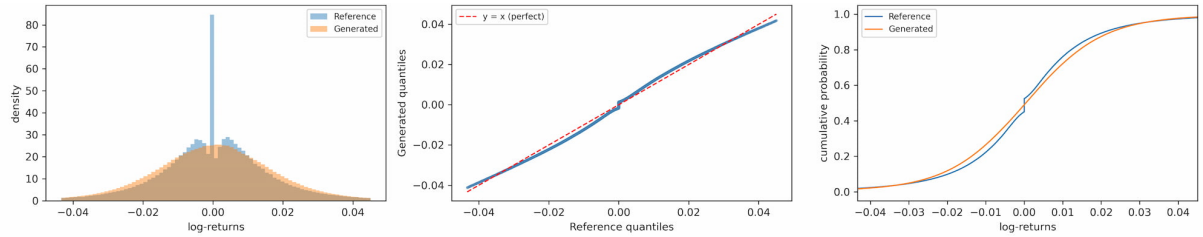


Figure 6.5: Marginal distribution for VE+Exponential ($L = 512$) against Reference (blue).

substantially below the reference (39.5): VE+Exponential reaches 14.3, about 64% lower, while VE+Cosine reaches 16.6, about 58% lower. Moreover, as can be noted from Fig. 6.5, the zero-return spike is still oversmoothed, with the model showing no significant ability to learn this pattern.

Stylized Facts The stylized facts of FTS windows are fairly well captured by the model with heavy-tail prediction that is reasonably close, while volatility clustering and the leverage effect are still underestimated (Fig. 6.6).

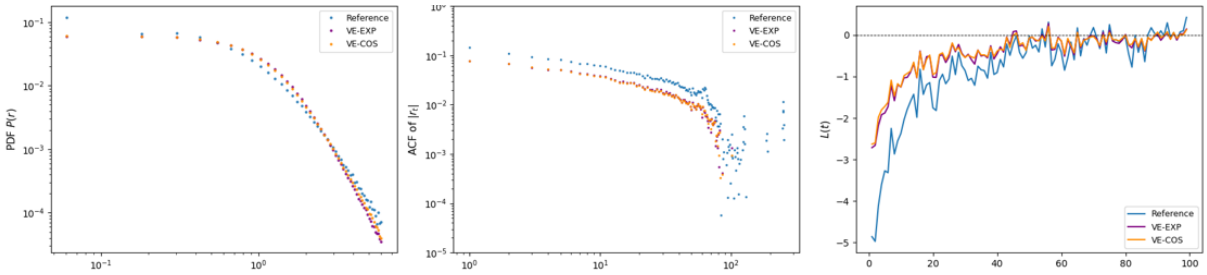


Figure 6.6: Stylized facts for Reference, VE+Exponential (purple), and VE+Cosine (orange) on windows ($L = 512$). From left to right: heavy-tail distribution, volatility clustering, leverage effect (cf. Fig. 6.4).

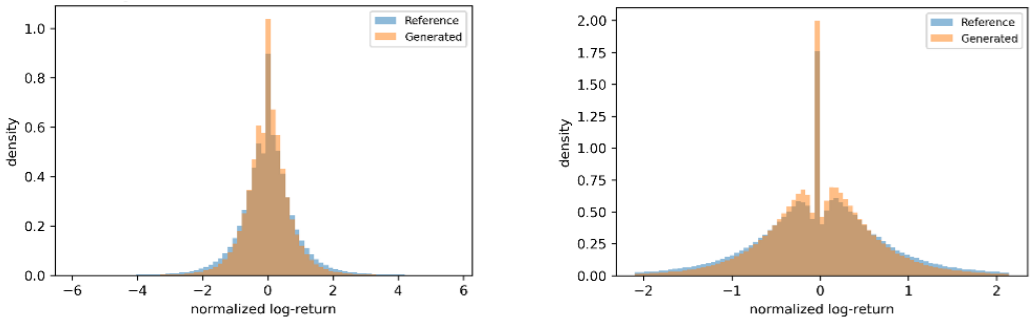
6.4.2 Phase 2 Unconditional EDM-CSDI

The next phase focused on understanding how the EDM framework impacts the generalization and learning of stylized facts of the CSDI model while unconditionally train. The model evaluated is EDM-CSDI (Unc.). A total of $20,000$ windows of length $L = 512$ were generated, and for each conditional set of information (i.e., each window), five sample paths were denoised with 200 steps ($NFEs = 400$).

Split	Configuration	Mean	Std	Skew.	Kur.	$q_{0.01}$	$q_{0.99}$	$D_{\mathrm{KS}} \downarrow$	$W_1 \downarrow$	$\hat{\alpha}$
Both	Reference	0.0000	1.0000	−0.5864	39.5	−2.7188	2.7746	—	—	4.418
Both	EDM-CSDI (Unc.)	0.0008	0.7523	−0.2292	20.0	−2.0292	2.0855	0.0418	0.0029	4.640

Table 6.4: Aggregate statistics for normalized log-returns, EDM-CSDI (Unc.), combining training and held out validation windows (*Both*). D_{KS} and W_1 are computed against the unnormalized training marginal.

Tab. 6.4 alongside Fig. 6.7 show that while the model is still deficient in its ability to learn higher moments, it visibly recovers a sharper density peak near zero, consistent with the zero-return peak. A plausible explanation is architectural: under the EDM parameterisation the network predicts x_0 directly (Sec. 2.5), so for observations exactly at $-\mu_{\mathrm{norm}}/\sigma_{\mathrm{norm}}$ (0% log-return when unnormalized) the denoiser can map directly onto the observed value, whereas in Phase 1 the network predicts the injected noise $\epsilon \sim \mathcal{N}(0, I)$, which carries no direct signal that the underlying clean value is exactly zero, particularly at small σ. While solid, this remains a hypothesis, as the conditioning information and architecture were not varied independently of the EDM reformulation. The overestimation of the zero-return effect is visible in Fig. 6.7, where the generated density peak exceeds the reference's.



(a) Density $q_{0.001} - q_{0.999}$ (b) Density $q_{0.02} - q_{0.98}$

Figure 6.7: Marginal density for EDM-CSDI (Unc.) against Reference (blue), validation and training windows combined.

Stylized Facts Through EDM preconditioning the model's ability to capture stylized facts improved, nevetheless the generated distribution remains less dispersed relative to the reference: standard deviation (0.7523 vs. 1.0) and the 1st/99th percentiles are each approximately 25% smaller in magnitude (Tab. 6.4). This underestimate the magnitude

of extreme daily moves, a gap that the conditional model (Sec. 6.4.3) substantially close. A regime-stratified analysis (App. C.2) shows this compression is not uniform: the generated/reference volatility ratio for EDM-CSDI (Unc.) decreases monotonically from 0.87 in low-volatility windows to 0.74 in high-volatility windows, meaning the underdispersion is most severe exactly where it matters most for risk.

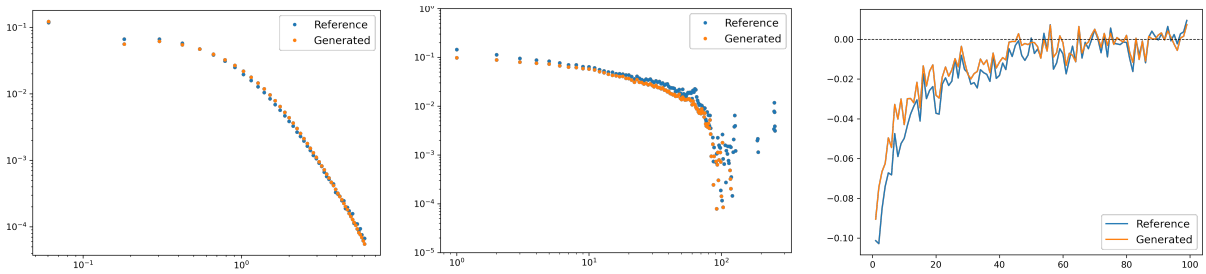


Figure 6.8: Stylized facts for Reference and EDM-CSDI (Unc.) on all generated windows. From left to right: heavy-tail distribution, volatility clustering, leverage effect.

6.4.3 Phase 2 Conditional EDM-CSDI

The next step involved the introduction of conditional information, specifically the transformed OHL (Open, High, Low) values. These settings are described in Tab. 4.1, Phase 2, and the model will be referred to as EDM-CSDI (Cond.).

Similarly to EDM-CSDI (Unc.), a total of $20,000$ windows of length $L = 512$ were generated with five sample paths each. In this case the validation windows are the same as those used for EDM-CSDI (Unc.). Complete results are in the appendix, Tab. B.6.

Aggregate Statistics

As reported in Tab. 6.5 and illustrated in Fig. 6.9, the new conditional information enabled the model to reach much higher accuracy in higher moments and distributional details. In particular, the conditional model narrows the tail-heaviness gap relative to the unconditional model: $\hat{\alpha} = 4.572$ (Cond.) vs. 4.640 (Unc.), against a reference of 4.418, and it matches the Reference quantiles more accurately overall (Tab. 6.5). Additionally, the model is better at matching the zero-return spike specific pattern, with lower yet fairly accurate performance for held-out validation windows. This suggests that the conditioning information provides useful global and local signal. Consistent with

Split	Configuration	Mean	Std	Skew.	Kur.	$q_{0.01}$	$q_{0.99}$	$D_{\mathrm{KS}} \downarrow$	$W_1 \downarrow$	$\hat{\alpha}$
Val	Ground Truth	−0.0024	0.8656	−0.4069	13.9	−2.3305	2.3723	—	—	4.072
Val	EDM-CSDI (Unc.)	0.0031	0.7314	−0.3558	19.9	−1.9595	2.0232	0.0331	0.0020	4.768
Val	EDM-CSDI (Cond.)	0.0033	0.8299	−0.4432	13.9	−2.2396	2.2635	0.0352	0.0005	4.278
Both	Reference	0.0000	1.0000	−0.5864	39.5	−2.7188	2.7746	—	—	4.418
Both	EDM-CSDI (Unc.)	0.0008	0.7523	−0.2292	20.0	−2.0292	2.0855	0.0418	0.0029	4.640
Both	EDM-CSDI (Cond.)	0.0029	0.9487	−0.5390	39.0	−2.5789	2.6306	0.0411	0.0006	4.572

Table 6.5: Aggregate statistics for normalized log-returns, EDM-CSDI (Unc.) and (Cond.). *Val*: validation windows vs. Ground Truth (GT) conditioning data; *Both*: validation and training combined vs. the full Reference. D_{KS} and W_1 are computed against the unnormalized training marginal.

this interpretation is the regime-stratified breakdown (App. C.2) that attributes much of this gain to crisis-regime windows, specifically: the generated/reference volatility ratio reaches 0.99 for EDM-CSDI (Cond.) in high-volatility windows, against 0.74 for EDM-CSDI (Unc.), suggesting the aggregate kurtosis-gap closure is driven primarily by better reproduction of crisis-regime extremes.

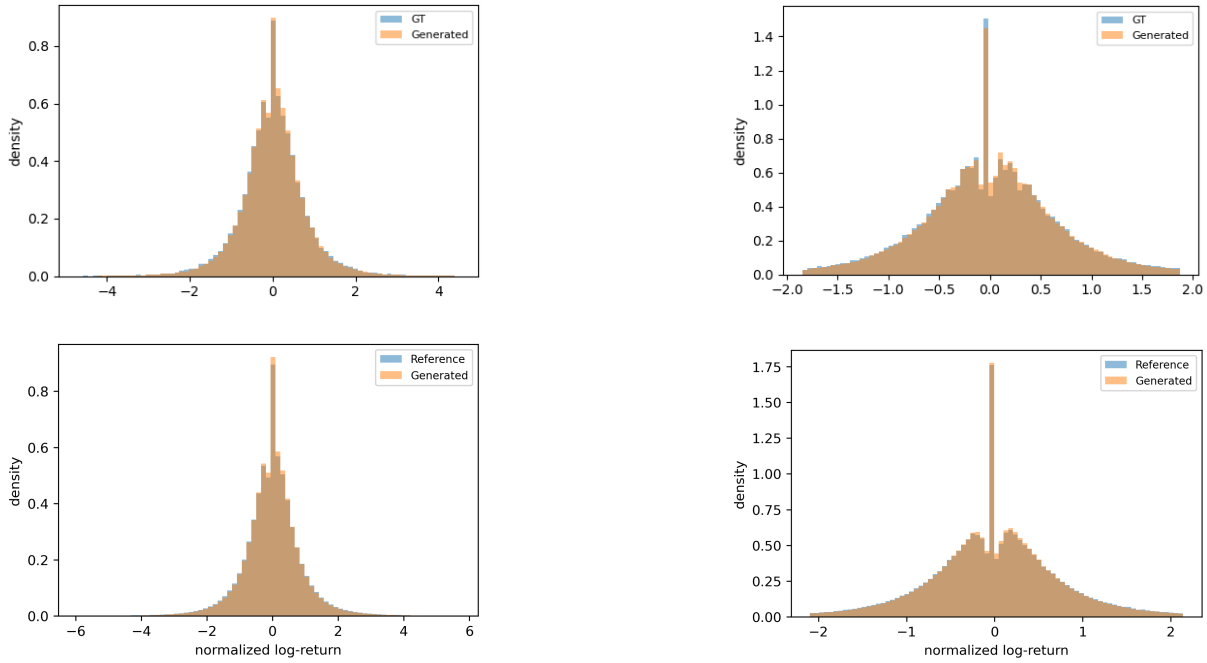


Figure 6.9: Marginal distribution against the empirical reference (blue) for the EDM-CSDI (Cond.) model. *Top row*: validation windows only (GT, because only windows used for conditioning were considered); *bottom row*: validation and training windows combined. *Left column*: full density range $q_{0.001}$–$q_{0.999}$; *right column*: central range $q_{0.02}$–$q_{0.98}$.

Stylized Facts Through the conditioning information, the EDM-CSDI (Cond.)

model matches the stylized facts much more closely, as visualized in Fig. 6.10, consistent with the model capturing the indirect relationships between log-returns at different lags. Of particular interest is that the leverage-effect intensity varies between the *Validation* and *Both* splits, pointing to a discriminative response to diverse conditioning information.

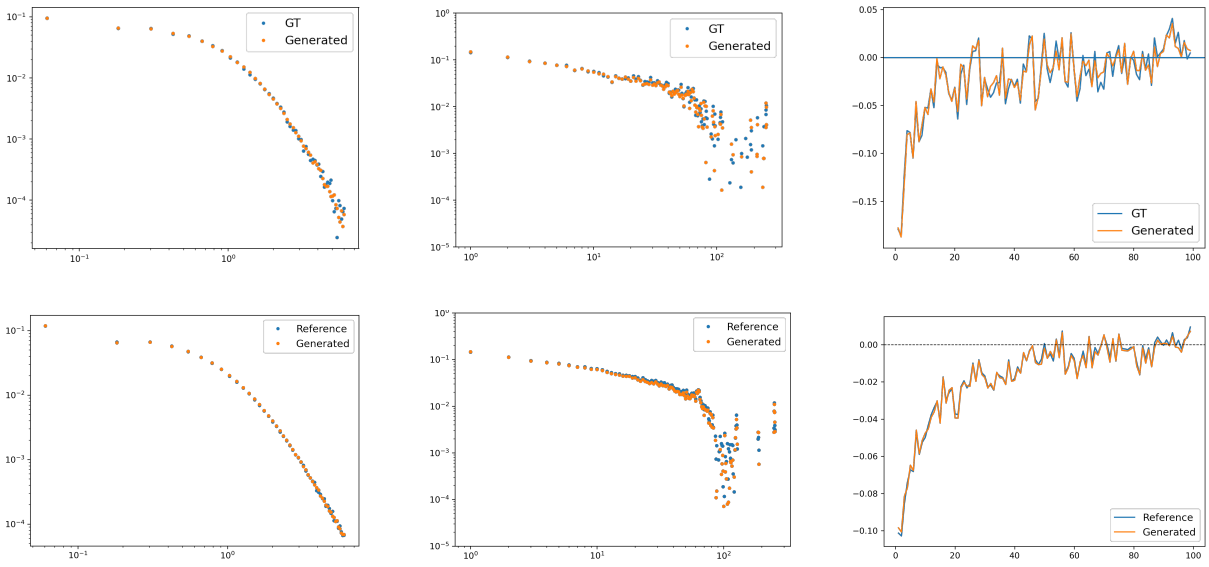


Figure 6.10: Stylized facts with Reference or Ground Truth (GT) (blue) versus Generated for the EDM-CSDI (Cond.) model. *Top row*: validation windows only; *first column*: Validation windows versus the ground truth (conditioning windows). *second column*: combined validation and training windows versus the entire Reference (dataset). *Left column*: heavy-tail distribution; *centre column*: volatility clustering; *right column*: leverage effect.

CRPS and Interval Coverage For this analysis the EDM-CSDI (Con.) was used to generate two datasets. One with 50 samples for each 2,000 windows, while the other only with 25 samples. Both with $L = 512$ and $s = 100$. Generating these samples for each of the $20,000$ windows was computationally unfeasible given the HPC constraints.

As observable in Tab. 6.6, the EDM-CSDI model achieve a CRPS skill score of 0.7376 (Both, $N = 50$), meaning a 73.76% improvement attributable to conditioning signal. This helps reduce the average MAE from approximately one standard deviation in daily log-returns to under a third, while decreasing the breadth of the prediction. This is true both for Validation and for combined (Both) windows. An OLS regression of Close on the same OHL conditioning already captures most of this signal, yet EDM-CSDI (Cond.) still improves on it with a CRPS skill score of 0.4044 (Both, $N = 50$), indicating

the conditional density carries information beyond a linear point forecast. Interestingly, validation windows show a lower CRPS and MAE, consistent with less exposure to rare and extreme market events, as expected given the smaller subset size.

Regarding the interval coverage (Tab. 6.7) higher nominal levels are correlated with more overconfident model's predictions (negative bias). Increasing the number of generated samples from 25 to 50 consistently increases empirical coverage at every nominal level, shifting the model from overconfident (negative bias) toward better-calibrated or underconfident (positive bias) predictions; this shift is sufficient to flip the sign of the bias at the 80% and 90% levels, while at 95% and 99% it narrows but does not eliminate the overconfidence. For financial markets, where coverage of extreme events is important, more samples (which provide broader coverage of extreme events and better CRPS) seem to be a more reasonable choice.

Configuration	**Split**	**N. Samples**	**MAE** ↓	**CRPS** ↓	$\mathbf{T_1^{norm}}$	$\mathbf{T_1}$	$\mathbf{T_2^{norm}}$
Climatology	Val	25	0.619 13	0.448 69	0.897 45	0.018 73	0.897 53
OLS	Val	25	0.253 62	0.186 92	0.373 85	0.007 80	0.373 86
EDM-CSDI (Cond.)	Val	25	0.186 00	0.121 44	0.244 38	0.005 10	0.245 88
Climatology	Both	25	0.662 88	0.482 32	0.964 66	0.020 13	0.964 29
OLS	Both	25	0.285 20	0.212 52	0.425 07	0.008 87	0.425 10
EDM-CSDI (Cond.)	Both	25	0.195 46	0.126 02	0.257 31	0.005 37	0.262 58
Climatology	Val	50	0.608 51	0.449 00	0.897 82	0.018 73	0.897 65
OLS	Val	50	0.253 62	0.186 90	0.373 85	0.007 80	0.373 90
EDM-CSDI (Cond.)	Val	50	0.184 41	0.121 41	0.244 31	0.005 10	0.245 84
Climatology	Both	50	0.650 65	0.482 35	0.964 61	0.020 13	0.964 52
OLS	Both	50	0.285 20	0.212 55	0.425 15	0.008 87	0.425 20
EDM-CSDI (Cond.)	Both	50	0.193 79	0.126 59	0.257 27	0.005 37	0.262 59

Table 6.6: CRPS decomposition for the EDM-CSDI models against the climatology and OLS baselines. CRPS is decomposed as $\text{CRPS} = \text{T}_1^{\text{norm}} - \frac{1}{2}\text{T}_2^{\text{norm}}$, where T_1 is the accuracy term unnormalised by $\sigma = 0.020867$. **N. Samples** is to be intended for each window. Lower CRPS is better.

Split	Nominal (%)	EDM-CSDI 25	EDM-CSDI 50
Val	50.0	50.59	52.23
Val	80.0	78.41	82.16
Val	90.0	86.66	90.35
Val	95.0	91.34	94.06
Val	99.0	93.46	96.89
Both	50.0	51.57	53.05
Both	80.0	78.95	82.52
Both	90.0	86.87	90.50
Both	95.0	91.46	94.02
Both	99.0	93.60	96.84

Table 6.7: Empirical coverage at nominal levels for EDM-CSDI with 25 samples, and EDM-CSDI with 50 samples. Values closer to the nominal level indicate better-calibrated prediction intervals.

Price Paths Samples It is also possible to observe the influence of conditioning directly by looking at the price paths (Figs. 6.11, 6.12). Conditional information visibly influences the model's generation of new windows, and hard features, such as jumps, appear to be reflected in the generated paths. An extended comparison of price paths is reported in Figs. B.9, B.8.

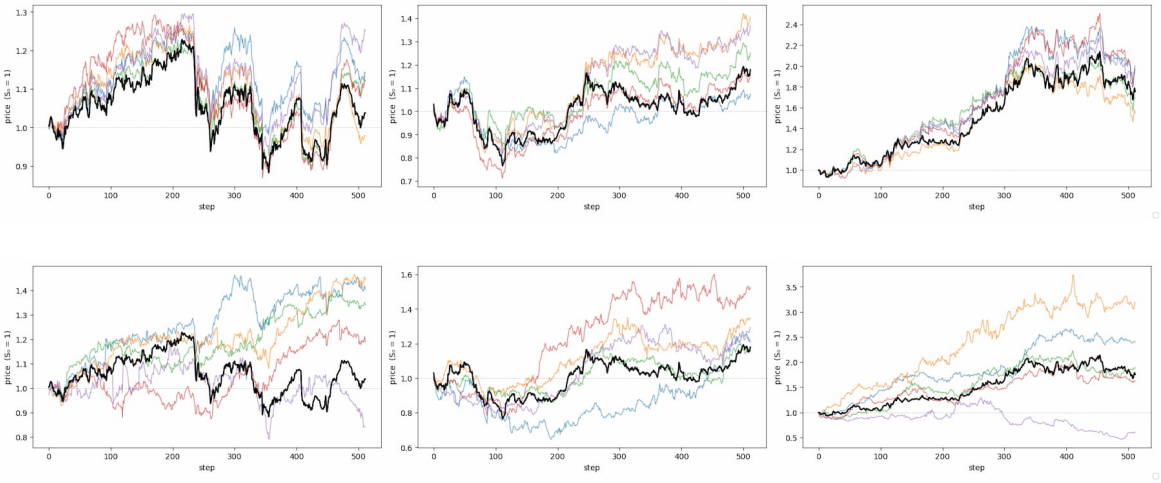


Figure 6.11: Samples of prices paths. *Top.* EDM-CSDI (Con.). *Bottom.* EDM-CSDI (Unc.). In black is drawn the Ground Truth, identical for both models.

Sampling Variations Additional sampling explorations for EDM-CSDI (Cond.) are reported in App. C. Replacing the deterministic Heun ODE sampler with a stochastic second-order Heun (SDE) variant ($S_{\text{churn}} = 10$) increases the standard deviation of gen-

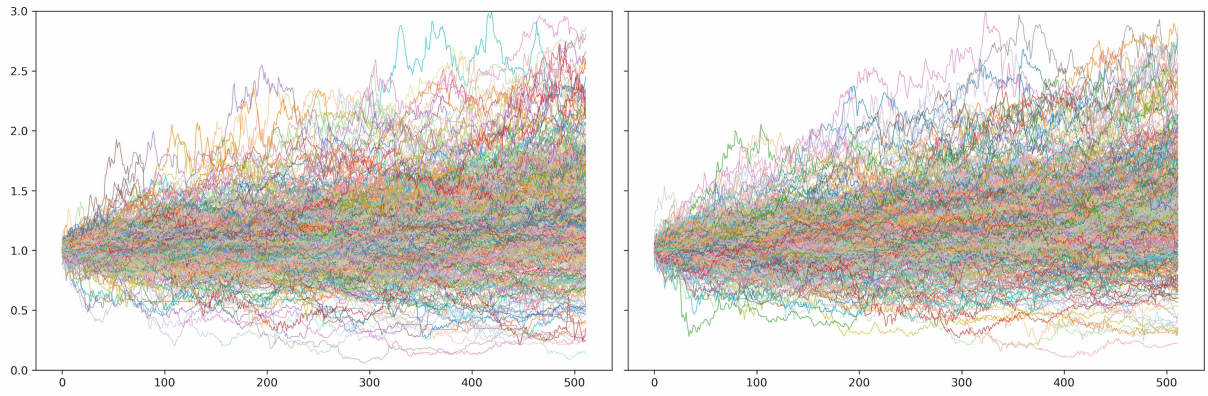


Figure 6.12: *Rigth.* Reference price samples path. *Left.* Generated price path by EDM+CSDI (Con.) aggregated.

erated normalized log-returns from 0.9487 to 1.0041 (reference: 1.0000) and improves W_1 by roughly an order of magnitude (0.000569 to 0.000077), but overshoots kurtosis (45.6 vs. a reference of 39.5) and skewness (Tab. C.1, Tab. C.2), suggesting that the injected stochasticity pushes the model into rarely-seen, poorly-learned regions of the distribution. Despite this, the Heun SDE sampler achieves a slightly lower CRPS and MAE than Heun ODE for the same number of samples ($N \in \{25, 50\}$), at the cost of marginally more overconfident coverage at the 90%, 95%, and 99% levels (App. C, Tab. C.3, Tab. C.4). Stylized facts are largely preserved under both samplers, though the SDE variant shows slightly more variability in the leverage effect (Fig. C.1). Separately, varying the number of function evaluations under the Heun ODE sampler ($NFEs \in \{200, 400, 600\}$) produced nearly indistinguishable aggregate statistics and distributional metrics (Tab. C.5, Tab. C.6), indicating that the default budget of $NFEs = 400$ used throughout this chapter is not a binding constraint, and that halving it to $NFEs = 200$ does not measurably degrade generation quality.

6.5 Summary

This chapter has explored the implementation and journey that led to the EDM-CSDI (Cond.) model and its ability to replicate both the distributional and the stylized-fact properties of S&P 500 close log-returns.

Regarding RQ1 (Phase 1), the VE and VP families partially reproduce the bulk of the

return distribution, quantiles, standard deviation, and the leverage effect at reduced scale, while underestimating or destabilising higher moments and undershooting the heavyness of the tails. The GBM family fails outright, generating near-Gaussian samples (Tab. 6.2). VE+Exponential achieves the best distributional similarity ($D_{KS} = 0.037$, $W_1 = 0.002$), while VP+Cosine attains the closest tail exponent ($\hat{\alpha} = 4.368$ vs. reference 4.378).

Regarding RQ2 (Phase 2), embedding the architecture within the EDM framework and conditioning on same-day Open, High, and Low log-returns yields a substantial improvement over the Phase 1 baseline: the excess-kurtosis gap relative to the reference narrows from a 49% deficit (EDM-CSDI Unc., 20.0 vs. 39.5) to a 1% deficit (EDM-CSDI Cond., 39.0 vs. 39.5), and W_1 improves roughly of five times (0.0029 to 0.0006, Both split; Tab. 6.5). Conditioning also yields a CRPS skill score of approximately 0.4044 over the OLS baseline and a substantial reduction in MAE (Tab. 6.6). Calibration remains imperfect: empirical coverage tracks the nominal level reasonably well at moderate confidence levels but the model remains overconfident at the 95% and 99% levels (Tab. 6.7).

Taken together, these results indicate that the CSDI-based architecture can learn the bulk of the FTS distribution under the SDE framework of Kim et al. [17], but that only the combination of EDM preconditioning and OHL conditioning substantially narrows the gap in higher-order moments and tail behaviour, properties that are well established as critical for risk-sensitive financial applications such as scenario analysis, stress testing, and portfolio risk management.

Chapter 7

Discussion

7.1 Discussion of Results

Across all the experiments reported in Ch. 6, EDM-CSDI (Cond.) consistently outperforms the other models, and several further patterns emerge from a cross-phase reading of the results.

Prediction Target and the Zero-return Spike As visible in Figs. 6.3 and 6.7, Phase 1 models predict the injected noise ϵ, and their generated ECDF passes through the central spike at zero returns without reproducing it. This indicates that the ϵ-target carries no information about the discreteness of the underlying price grid. EDM-CSDI (Unc.), which predicts x_0 directly, exhibits the opposite behaviour, slightly overshooting the same spike (Fig. 6.7). The zero-return spike therefore serves as a single-statistic check of whether a given prediction target captures the near-discrete nature of prices, which are quoted in cents, hidden within an otherwise continuous log-return variable.

Conditioning, Volatility Regimes, and Two Tail Failure Modes Every unconditional configuration in this thesis underestimates excess kurtosis: VE+Exponential reaches only 14.3 against a reference of 36.0–39.5 (Tabs. 6.2 and 6.3), and EDM-CSDI (Unc.) reaches 20.0 against 39.5, a 49% deficit (Tab. 6.4). Conditioning on same-day OHL closes this deficit (39.0 vs. 39.5, Tab. 6.5) and improves roughly five times W_1. A possible reading is that unconditional generation is implicitly a mixture problem: the training

pool combine calm and turbulent regimes, and a model sampling unconditionally must reproduce both without being told which one is which. Unless it can be inferred from the noised data alone, samples are pulled towards an average regime of that period. Because OHL are strongly correlated with realized volatility (App. A), they provide the missing regime indicator. This hypothesis is tested directly in App. C.2: stratifying generated windows by reference-volatility tertile, EDM-CSDI (Cond.) reproduces the reference's High/Low volatility ratio almost exactly (1.77 vs. 1.70, ≈104%), whereas EDM-CSDI (Unc.) compresses it to 1.45 (≈85%), confirming that OHL conditioning is what lets the model match regime-specific magnitudes rather than reverting to an average regime.

On the other hand, GARCH-X and Merton overstate kurtosis by two to three orders of magnitude (11021.7 and 3736.4 vs. 38.5, Tab. 6.1) due to a small fraction of paths escaping their working windows. The two model families fail in opposite ways: parametric models occasionally diverge, while diffusion models trained without conditioning information are pulled toward the mean and underrepresent rare extremes [28, 39].

Sharper but Not Better Calibrated The conditional model reduces MAE from ≈ 0.62 (climatology) to ≈ 0.19, against ≈ 0.29 for an OLS regression on the same OHL conditioning, and still reaches a CRPS skill score of 0.4044 relative to OLS (Tab. 6.6), yet remains overconfident at the 95% and 99% nominal levels even with 50 samples per window (Tab. 6.7). Part of this gap may simply be a finite sample artifact: for an interval built from the min/max of N i.i.d. draws (where $N = 25$ or 50 full-length paths sampled per window under the same OHL conditioning each contributing one value per position) there is an expected coverage provided by order statistics of $(N-1)/(N+1)$, i.e. $24/26 \approx 92.3\%$ for $N = 25$ and $49/51 \approx 96.1\%$ for $N = 50$, regardless of how many positions or windows the resulting intervals are averaged over [1]. Both bounds are close to the observed coverage at the 99% level (Val split): 92.3% against 93.46% for $N = 25$, and 96.1% against 96.89% for $N = 50$, aggregated across all 512 positions of the 2,000 evaluated windows. With

[1] This figure follows from treating the realized outcome as exchangeable with the N generated values under calibration: each of the $N+1$ values is equally likely to be the overall minimum or maximum, so the realized outcome lies inside $[\min, \max]$ with probability $1-2/(N+1) = (N-1)/(N+1)$; under-dispersion relative to the true distribution would push observed coverage below this level.

only 25 and 50 samples per position, a 99% interval's endpoints fall outside the range of the available order statistics. How much of the remaining gap reflects genuine under-dispersion of the learned density, once this finite-N effect is accounted for, is left open.

Champion Selection and Scale Transfer The Phase 1 champions carried into Phase 2, VE+Exponential and VE+Cosine, were selected on D_{KS}/W_1, the metrics on which they are indeed best (0.037/0.002 and 0.062/0.003, Tab. 6.2) and also by considering Kim et al. [17] findings. VP+Cosine, however, achieved the tail exponent closest to the reference ($\hat{\alpha} = 4.368$ vs. 4.378, error 0.010, against 0.083 for VE+Exponential) but was not retained due to computational and time limitation. VE also offers a direct comparison with EDM, since both build on the VE SDE formulation. It was found that regenerating the VE champions at $L = 512$ produced no systematic improvement over their $L = 2048$ behaviour (Tab. 6.3): the kurtosis deficit persistsed.

7.2 Limitations

Training, Sampler, and Architecture Ablation No formal convergence analysis was conducted, training stability was monitored empirically, and several hyperparameters were adopted from literature defaults rather than tuned specifically for FTS:

- σ_{data}: fixed to 1 by construction, with its window-wise variation and interaction with $c_{\text{skip}}/c_{\text{out}}$ not explored.
- $\sigma_{\max}$ was fixed rather than data-derived (Sec. 7.3 discusses an alternative), and $c_{\text{noise}}(\sigma)$ used the 1/4 scaling of Karras et al. [16].
- Sampler: $\rho = 7$ (EDM default) was used throughout, although $\rho = 3$ equalizes truncation error and may suit FTS better given that volatility clustering depends on mid-range σ. Stochastic churn ($S_{\text{churn}} > 0$) was not varied and NFEs values were only partially explored.
- Architecture: embedding dimensions and fixed sinusoidal positional encoding were inherited from Kim et al. [17] without ablation. Learnable positional encodings and

skip-connection weights were not tested.

Confounded EDM and Conditioning Effects The hypothesis that the sharpened zero-return peak of EDM-CSDI (Unc.) follows from x_0-prediction, rather than some other side effect of the EDM reformulation (Sec. 6.4), was not fully isolated experimentally. A 2×2 design (CSDI with x_0-prediction but without EDM preconditioning, and EDM-CSDI with ϵ-prediction) would be needed to differentiate the gains derived from Phase 2 from those related to the EDM reformulation and to OHL conditioning, which were introduced together.

Capacity-to-Data Ratio The conditional model's parameter count ($\approx$ 3.4M, Sec. 4.4) is of the same order as the dataset's unique observations ($\approx$ 2.7M, Sec. 4.1) once the substantial overlap between sliding windows ($L = 512$, $s = 100$) is accounted for, which raises the concern that part of the model's accuracy could reflect memorisation rather than genuine generalisation. To probe this, EDM-CSDI (Cond.) was retrained with its channels halved to 64, which reduces the parameter count to $\approx$ 1.2M, a capacity-to-data ratio below one half, while leaving the six residual blocks and every other setting unchanged so that capacity is the only varying factor (App. C.3). The outcome is hard to reconcile with memorisation: distributional similarity on the combined split is essentially preserved (D_{KS} moves from 0.0411 to 0.0390 and W_1 stays at $\approx$ 0.0006), and only the strictly held-out validation split worsens slightly (D_{KS} from 0.0352 to 0.0425), the opposite of the train-favouring gap a memorising model would exhibit. What the smaller model loses is instead concentrated in the structure that is hardest to learn, as skewness falls towards zero and the combined-split tail destabilises into a few outlier windows ($\hat{\alpha} = 4.94$ against a reference of 4.42), whereas on the held-out windows the tail exponent remains close to the ground truth ($\hat{\alpha} = 4.11$ against 4.07). The reduced model therefore underfits rather than overfits, supporting the reading that the strong validation metrics of EDM-CSDI (Cond.) follow from the informativeness of OHL conditioning and from genuine generalisation. The same robustness extends to probabilistic forecasting, where sampling fifty paths per window the smaller model retains a CRPS skill score close to 0.70 over

climatology and continues to outperform the OLS baseline, while its interval coverage does not deteriorate on the held-out split (App. C.3).

Computational Budget and Scope The CRPS and coverage analysis (Tabs. 6.6 and 6.7) used 2,000 windows, sampling 25–50 paths per window under the same conditioning set due to HPC constraints. As discussed above, the quantity that matters for interval calibration is this N (N. Samples) per-window and 25–50 remains too few to resolve fully 95–99% intervals. More broadly, all diffusion models were trained on a single corpus of S&P 500 close log-returns spanning both decimalization regimes as one undifferentiated dataset, and the only baselines were classical parametric models (GARCH-X, Merton-X). No comparison was made against other deep generative approaches to FTS modeling [25, 26, 55], so the standing of EDM-CSDI (Cond.) within the broader deep-learning literature remains partly unexplored.

7.3 Future Work

Random Masking The `predict_close` mask used in Phase 2 is a fixed channel-wise partition, so the model only ever learns $p(C \mid O, H, L)$. CSDI's `randmask`, which varies the conditioning set stochastically at each step, would let the model learn conditional distributions over arbitrary channel subsets, for instance predicting the intraday range from the open, or imputing any missing pattern. Whether this generality helps or hurts `predict_close` specifically remains open.

Principled $\sigma_{\max}$ Selection and Heavy-Tailed Noising Rather than abalting $\sigma_{\max}$ manually, Pandey et al. [40], propose to estimate $\sigma_{\max}$ by minimizing the mutual information between clean and maximally-noised data, which could remove this hyperparameter entirely. More broadly, t-EDM, replace the Gaussian noising distribution with a Student-t, preserving the EDM preconditioning structure while changing the tail behaviour of the forward process, directly targeting some of the heavy tails modeling difficulties discussed in Sec. 7.1.

Stochastic Sampling and Constraint-Preserving Conditioning This thesis

used the deterministic Heun ODE sampler ($S_{\text{churn}} = 0$) for the results of Sec. 6.4.3. A preliminary stochastic-churn run (Heun SDE, $S_{\text{churn}} = 10$, App. C, Tabs. C.1, C.2) cause the standard deviation, tail quantiles, and W_1 to move closer to the reference, partially addressing the heavy tail modeling problem of Sec. 7.1, but overshoots kurtosis and skewness, as noise injection pushes sampling supposedly into poorly-learned regions. A systematic exploration of S_{churn}, S_{tmin}, and S_{tmax} is needed to find a setting that improves tail coverage without causing overshooting.

Constraint-Preserving Conditioning Enforcing $H \geq C \geq L$ in generated samples would eliminate a set of structurally invalid paths, but such a hard constraint could also restrict the freedom of the diffusion process to explore and generate genuinely novel trajectories, particularly near the edges of the H–L range. The strength of this constraint could in principle be varied from a soft penalty to a strict reparameterisation, which would itself constitute a design choice affecting sample diversity. This remains untested.

Financial Metrics The evaluation in this thesis stayed at the level of distributional and stylized-fact metrics (D_{KS}, W_1, kurtosis, tail exponent, CRPS, coverage), which measure how faithfully the generated log-returns reproduce the reference but say little about the practical financial decisions such generated FTS could inform. A natural next step is to assess EDM-CSDI (Cond.) through risk measures that practitioners actually report, for instance Value at Risk (VaR) and Expected Shortfall (ES). This would ground the generator in a concrete risk-management use case and offer practically meaningful complement to the purely statistical metrics used here.

Regime-Stratified Training, Evaluation, and Reweighting The divergence in the intensity of some stylized facts between the *Validation* and *Both* splits, and across models, calls for further investigation and generalization, specifically, testing this thesis' findings under other regimes, such as the pre-/post-2002 decimalization process. A first regime-stratified evaluation of realized-volatility level is explored in App. C.2, but call for further investigation and testing. This would clarify whether accuracy is uniform across regimes or concentrated in those dominating the training windows. If tail erosion is per

se partly a consequence of rare windows contributing relatively to the average training loss, reweighting the training distribution toward certain windows, as in class-balancing diffusion [56], could offer a way to test this hypothesis.

Chapter 8

Conclusion

This thesis set out to address a long-standing challenge in Quantitative Finance: capturing the stylized facts of financial time series: heavy tails, volatility clustering, and the leverage effect, within a generative framework that is both expressive and tractable. Two classical baselines, GARCH-X and Merton-X, were first calibrated as a point of reference (Ch. 6): well understood, interpretable, and partially effective at reproducing the bulk of the distribution, but structurally unable to capture the leverage effect and other distributional properties. This established the gap that a deep generative approach would need to close.

Phase 1 replicated the CSDI-based diffusion architecture of Kim et al. [17], whose attention-based backbone descends from a lineage of gated residual architectures, PixelCNN, WaveNet, and DiffWave, across the VE, VP, and GBM SDE families and several noise schedules. This phase confirmed that the architecture can learn the bulk of the return distribution and recover, at reduced scale, key stylized facts such as the leverage effect, while also clear limitations arose in the tails and higher moments, most severely for the GBM family. These findings both validated the chosen architecture and motivated the extension pursued in Phase 2.

The central contribution of this thesis, EDM-CSDI, addressed these limitations directly by embedding the same CSDI backbone within the Elucidated Diffusion Model framework of Karras et al. [16] and introducing conditioning on same-day Open, High, and Low log-returns. As shown in Ch. 6 and discussed in Ch. 7, this combination substan-

tially narrowed the gap left open by Phase 1: heavy tails, volatility clustering, and the leverage effect were reproduced far more faithfully, and the conditioning information translated into sharper, better-informed probabilistic forecasts than an unconditional model could provide.

Taken together, these results answer both research questions posed at the beginning: a CSDI-based diffusion architecture is indeed able to learn the stylized facts of financial time series, and embedding it within the EDM framework while conditioning on readily available intraday information meaningfully improves its ability to do so. Beyond the empirical findings, this thesis delivers a clarified and generalized EDM-CSDI design that is mathematically transparent and replicable, offering a solid foundation for the open questions and extensions outlined in Ch. 7. Overall, the work presented here is an encouraging step toward generative models of financial time series that can be relied upon for future expansion in scenario analysis, stress testing, and risk-aware decision making. In this sense, the thesis returns to where it began: the enduring human desire not only to understand the mechanisms of financial markets, but to replicate them. With diffusion models, it seems we are learning to do both a little better.

Bibliography

[1] Cont, Rama. "Empirical properties of asset returns: Stylized facts and statistical issues". In: *Quantitative Finance* 1 (2) (2001): pp. 223–236.

[2] Box, George E. P. and Jenkins, Gwilym M. *Time Series Analysis: Forecasting and Control*. San Francisco: Holden-Day, 1970.

[3] Bollerslev, Tim. "Generalized autoregressive conditional heteroskedasticity". In: *Journal of Econometrics* 31 (3) (1986): pp. 307–327.

[4] Goodfellow, Ian, Pouget-Abadie, Jean, Mirza, Mehdi, Xu, Bing, Warde-Farley, David, Ozair, Sherjil, Courville, Aaron, and Bengio, Yoshua. "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems* 27 (2014).

[5] Creswell, Antonia, White, Tom, Dumoulin, Vincent, Arulkumaran, Kai, Sengupta, Biswa, and Bharath, Anil A. "Generative Adversarial Networks: An Overview". In: *IEEE Signal Processing Magazine* 35 (1) (2018): pp. 53–65.

[6] Kingma, Diederik P. and Welling, Max. "Auto-Encoding Variational Bayes". In: *International Conference on Learning Representations* (2014).

[7] Kingma, Diederik P. and Welling, Max. "An Introduction to Variational Autoencoders". In: *Foundations and Trends in Machine Learning* 12 (4) (2019): pp. 307–392.

[8] LeCun, Yann, Chopra, Sumit, Hadsell, Raia, Ranzato, Marc'Aurelio, and Huang, Fu Jie. "A tutorial on energy-based learning". In: *Predicting Structured Data*. Ed. by

G. Bakir, T. Hofmann, B. Schölkopf, A. Smola, and B. Taskar. MIT Press, 2006, pp. 191–246.

[9] Rezende, Danilo Jimenez and Mohamed, Shakir. "Variational Inference with Normalizing Flows". In: *International Conference on Machine Learning*. PMLR, 2015, pp. 1530–1538.

[10] Ho, Jonathan, Jain, Ajay, and Abbeel, Pieter. "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems* 33 (2020): pp. 6840–6851.

[11] Song, Yang, Sohl-Dickstein, Jascha, Kingma, Diederik P., Kumar, Abhishek, Ermon, Stefano, and Poole, Ben. "Score-Based Generative Modeling through Stochastic Differential Equations". In: *International Conference on Learning Representations* (2021).

[12] Song, Jiaming, Meng, Chenlin, and Ermon, Stefano. "Denoising Diffusion Implicit Models". In: *International Conference on Learning Representations*. 2021.

[13] Rombach, Robin, Blattmann, Andreas, Lorenz, Dominik, Esser, Patrick, and Ommer, Björn. "High-Resolution Image Synthesis with Latent Diffusion Models". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 10684–10695.

[14] Dhariwal, Prafulla and Nichol, Alexander. "Diffusion Models Beat GANs on Image Synthesis". In: *Advances in Neural Information Processing Systems* 34 (2021): pp. 8780–8794.

[15] Tashiro, Yusuke, Song, Jiaming, Song, Yang, and Ermon, Stefano. "CSDI: Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation". In: *Advances in Neural Information Processing Systems* 34 (2021): pp. 24804–24816.

[16] Karras, Tero, Aittala, Miika, Aila, Timo, and Laine, Samuli. "Elucidating the Design Space of Diffusion-Based Generative Models". In: *Advances in Neural Information Processing Systems* 35 (2022): pp. 26565–26577.

[17] Kim, Gihun, Choi, Sun-Yong, and Kim, Yeoneung. “A diffusion-based generative model for financial time series via geometric Brownian motion”. In: *arXiv preprint arXiv:2507.19003* (2025).

[18] Black, Fischer. “Studies of stock market volatility changes”. In: *Proceedings of the American Statistical Association, Business and Economic Statistics Section*. 1976, pp. 177–181.

[19] Mandelbrot, Benoit. “The Variation of Certain Speculative Prices”. In: *The Journal of Business* 36 (4) (1963): pp. 394–419.

[20] Fama, Eugene F. “Random Walks in Stock Market Prices”. In: *Financial Analysts Journal* 21 (5) (1965): pp. 55–59.

[21] Gopikrishnan, Parameswaran, Plerou, Vasiliki, Amaral, Luís A. Nunes, Meyer, Martin, and Stanley, H. Eugene. “Scaling of the distribution of fluctuations of financial market indices”. In: *Physical Review E* 60 (5) (1999): pp. 5305–5316.

[22] Mantegna, Rosario N. and Stanley, H. Eugene. “Scaling behaviour in the dynamics of an economic index”. In: *Nature* 376 (6535) (1995): pp. 46–49.

[23] Merton, Robert C. “Option pricing when underlying stock returns are discontinuous”. In: *Journal of Financial Economics* 3 (1-2) (1976): pp. 125–144.

[24] Black, Fischer and Scholes, Myron. “The Pricing of Options and Corporate Liabilities”. In: *Journal of Political Economy* 81 (3) (1973): pp. 637–654.

[25] Yoon, Jinsung, Jarrett, Daniel, and Schaar, Mihaela van der. “Time-series Generative Adversarial Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.

[26] Vuleti, Milena, Prenzel, Felix, and Cucuringu, Mihai. “Fin-GAN: forecasting and classifying financial time series via generative adversarial networks”. In: *Quantitative Finance* 24 (2) (2024): pp. 175–199.

[27] Hyvärinen, Aapo. “Estimation of Non-Normalized Statistical Models by Score Matching”. In: *Journal of Machine Learning Research* 6 (2005): pp. 695–709.

[28] Vincent, Pascal. “A connection between score matching and denoising autoencoders”. In: *Neural Computation* 23 (7) (2011): pp. 1661–1674.

[29] Parisi, Giorgio. “Correlation functions and computer simulations”. In: *Nuclear Physics B* 180 (3) (1981): pp. 378–384.

[30] Song, Yang and Ermon, Stefano. “Generative Modeling by Estimating Gradients of the Data Distribution”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019, pp. 11895–11907.

[31] Øksendal, Bernt. “Stochastic differential equations”. In: *Stochastic Differential Equations: An Introduction with Applications*. Springer, 2003.

[32] Anderson, Brian D. O. “Reverse-time diffusion equation models”. In: *Stochastic Processes and their Applications* 12 (3) (1982): pp. 313–326.

[33] Chen, Ricky T. Q., Rubanova, Yulia, Bettencourt, Jesse, and Duvenaud, David K. “Neural Ordinary Differential Equations”. In: *Advances in Neural Information Processing Systems*. 2018, pp. 6571–6583.

[34] Deng, Jia, Dong, Wei, Socher, Richard, Li, Li-Jia, Li, Kai, and Fei-Fei, Li. “ImageNet: A large-scale hierarchical image database”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255.

[35] Oord, Aaron van den, Kalchbrenner, Nal, and Kavukcuoglu, Koray. “Pixel Recurrent Neural Networks”. In: *International Conference on Machine Learning*. PMLR, 2016, pp. 1747–1756.

[36] Oord, Aaron van den, Dieleman, Sander, Zen, Heiga, Simonyan, Karen, Vinyals, Oriol, Graves, Alex, Kalchbrenner, Nal, Senior, Andrew, and Kavukcuoglu, Koray. “WaveNet: A Generative Model for Raw Audio”. In: *arXiv preprint arXiv:1609.03499* (2016).

[37] Kong, Zhifeng, Ping, Wei, Huang, Jiaji, Zhao, Kexin, and Catanzaro, Bryan. “DiffWave: A Versatile Diffusion Model for Audio Synthesis”. In: *International Conference on Learning Representations*. 2021.

[38] Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N, Kaiser, ukasz, and Polosukhin, Illia. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems* 30 (2017).

[39] Yu, Yifeng and Yu, Lu. “Diffusion Models with Heavy-Tailed Targets: Score Estimation and Sampling Guarantees”. In: *arXiv preprint arXiv:2601.06715* (2026).

[40] Pandey, Kushagra, Pathak, Jaideep, Xu, Yilun, Mandt, Stephan, Pritchard, Michael, Vahdat, Arash, and Mardani, Morteza. “Heavy-tailed diffusion models”. In: *arXiv preprint arXiv:2410.14171* (2024).

[41] Yoon, Eun Bi, Park, Keehun, Kim, Sungwoong, and Lim, Sungbin. “Score-based Generative Models with Lévy Processes”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.

[42] Parkinson, Michael. “The Extreme Value Method for Estimating the Variance of the Rate of Return”. In: *The Journal of Business* 53 (1) (1980): pp. 61–65.

[43] Byrd, Richard H., Lu, Peihuang, Nocedal, Jorge, and Zhu, Ciyou. “A Limited Memory Algorithm for Bound Constrained Optimization”. In: *SIAM Journal on Scientific Computing* 16 (5) (1995): pp. 1190–1208.

[44] Ronneberger, Olaf, Fischer, Philipp, and Brox, Thomas. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Vol. 9351. LNCS. Springer, 2015, pp. 234–241.

[45] Slok, Torsten. *Apollo Chief Economist: Average Tenure of Companies in the S&P 500 Index*. Apollo Global Management / Apollo Academy. 2026.

[46] Furfine, Craig H. “Decimalization and market liquidity”. In: *Economic Perspectives* 27 (4) (2003): pp. 2–12.

[47] Loshchilov, Ilya and Hutter, Frank. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations* (2019).

[48] Loshchilov, Ilya and Hutter, Frank. “SGDR: Stochastic Gradient Descent with Warm Restarts”. In: *International Conference on Learning Representations*. 2017.

[49] Arjovsky, Martin, Chintala, Soumith, and Bottou, Léon. “Wasserstein Generative Adversarial Networks”. In: *Proceedings of the 34th International Conference on Machine Learning*. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 214–223.

[50] Massey Jr., Frank J. “The Kolmogorov-Smirnov Test for Goodness of Fit”. In: *Journal of the American Statistical Association* 46 (253) (Mar. 1951): pp. 68–78.

[51] stakahashy. *fingan: The implementation of “Modeling financial time-series with generative adversarial networks”*. `https://github.com/stakahashy/fingan`. 2023.

[52] Alstott, Jeff, Bullmore, Ed, and Plenz, Dietmar. “powerlaw: A Python Package for Analysis of Heavy-Tailed Distributions”. In: *PLoS ONE* 9 (1) (2014): e85777.

[53] Matheson, James E. and Winkler, Robert L. “Scoring rules for continuous probability distributions”. In: *Management Science* 22 (10) (1976): pp. 1087–1096.

[54] Virtanen, Pauli et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020): pp. 261–272.

[55] Takahashi, Tomonori and Mizuno, Takayuki. “Generation of synthetic financial time series by diffusion models”. In: *Quantitative Finance* (2025).

[56] Qin, Yiming, Zheng, Huangjie, Yao, Jiangchao, Zhou, Mingyuan, and Zhang, Ya. “Class-Balancing Diffusion Models”. In: *CVF Conference on Computer Vision and Pattern Recognition*. 2023.

Appendix A

Data

In this chapter, additional information regarding the data used in this work are provided.

OHLC Distribution It is subsequently displayed in Tab. A.1 the aggregate statistics for the z-score normalized OHLC data and their mean and standard deviation (Tab. A.2).

Statistic	**Close**	**High**	**Low**	**Open**
Mean	0.0000	0.0000	0.0000	0.0000
Std	1.0000	1.0000	1.0000	1.0000
Min	−44.8905	−44.9547	−74.1453	−87.0546
$q_{0.01}$	−2.7247	−1.4785	−3.4485	−2.6747
$q_{0.05}$	−1.4216	−0.8963	−1.5955	−1.1570
$q_{0.25}$	−0.4350	−0.5050	−0.2970	−0.2697
Median	−0.0216	−0.1908	0.1894	−0.0244
$q_{0.75}$	0.4330	0.2699	0.5344	0.2814
$q_{0.95}$	1.4386	1.5226	0.9590	1.1723
$q_{0.99}$	2.7817	3.3728	1.6660	2.7358
Max	33.1965	57.0824	39.3966	63.8070
Skewness	−0.5773	7.1008	−4.3308	−2.1116
Ex. Kurtosis	38.5	261.2	106.4	254.7

Table A.1: Full distributional statistics for the normalized OHLC log-returns reference dataset (n = 2,623,489). Each channel is independently z-scored to zero mean and unit variance prior to training.

OHLC Correlation For a more comprehensive dataset overview it is reported also the correlation matrix in Fig. A.1.

Channel	Mean	Std
Close	0.000451	0.020867
Open	0.000265	0.010859
High	0.012179	0.019033
Low	−0.011306	0.017881

Table A.2: Per-channel mean and standard deviation used for z-score normalization of the OHLC log-return channels prior to training.

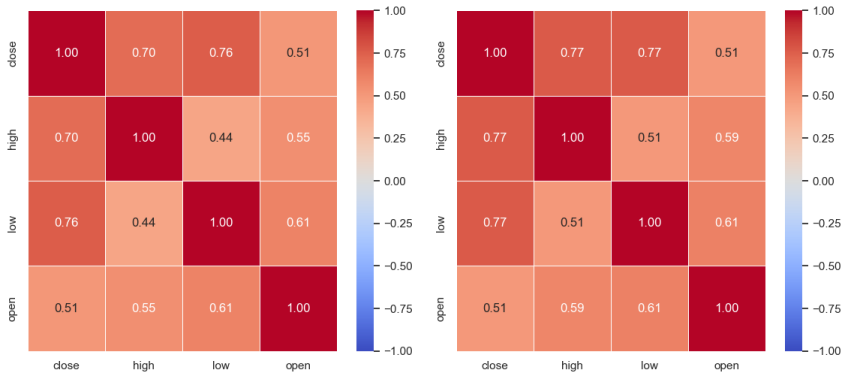


Figure A.1: Correlation Matrix for OHLC values. *Left.* Correlation for concatenated data. *Right.* Average correlation across files.

A.1 Decimalization

In this section it will be discussed more in depth the decimalization process briefly mentioned in Sec. 4.1, specifically what it is and how it is distributed in our training dataset.

Introduction The decimalization process was mandated by the Securities and Exchange Commission in January 2000 and completed by the 9th of April 2001 [46]. Before this period US stock prices were quoted in fractions, specifically 1/16th of a dollar, meaning that the minimum price movement needed to record an effective change in price was 0.0625 dollars. After decimalization this changed to 1 cent.

Characteristics 7.1928% of all our returns are exactly 0.0. The distribution of these zero-returns is not equal, specifically they are mostly concentrated around 1980s and 1990s as it is possible to see from the Fig. A.2.

The peak incidence of zero-returns occurred in 1984, with 10,600 observations, corresponding to 21.374% of that year's total returns.Interestingly, the frequency of 0% returns

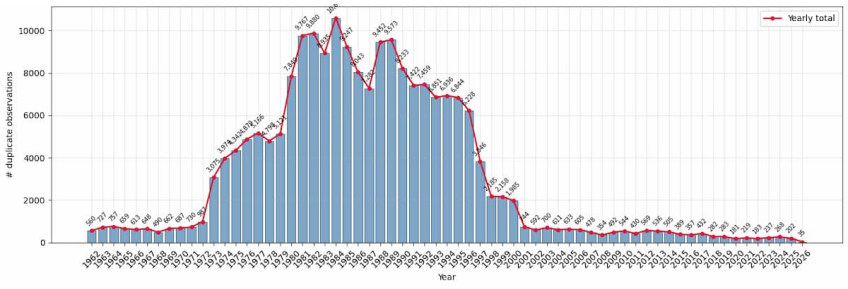


Figure A.2: Frequency count of absolute zero returns grouped by year.

drops before 1980. This is due to the 40-year historical minimum imposed on ticker selection (Sec. 4.1): all tickers have data from at least April 1986, but as one moves further back in time the number of available observations falls considerably. Specifically, the number of tickers decreases from 210 with data going back to 1986, to 91 in the 1970s, and to only 26 in the 1960s.

Decimalization Consequences Decimalization is responsible for the distinctive $q_{0.02}$–$q_{0.98}$ range shape discussed in the experimental chapter (Ch. 6), as further evidenced by the evolution of the log-return distribution shown in Fig. A.3.

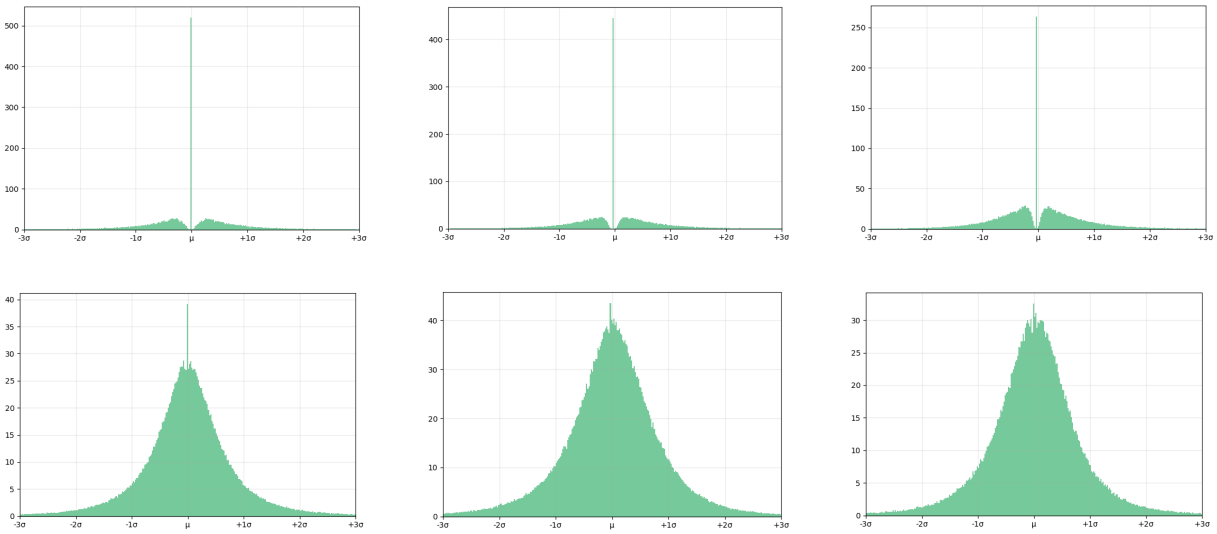


Figure A.3: Log-returns distribution by decade from 1970 (*top-left*) to 2020 (*bottom-right*). The decade 2020 refers to data till April 2026. Only data within the range of three standard deviations are displayed.

Appendix B

Experiments and Results

This appendix gathers the full quantitative evidence behind Ch. 6, reporting the complete distributional statistics, heavy-tail estimates, and stylized-facts figures only summarized there. It follows the same progression: GARCH-X and Merton-X baselines (Sec. B.1), the nine Phase1 configurations (Sec. B.2), and the Phase2 champion and EDM-CSDI modelsso each main-text claim can be traced to its underlying numbers.

B.1 Baseline Models

Full distributional statistics for the GARCH-X and Merton-X baseline models alongside the training reference are reported in Tab. B.1. GARCH-X was evaluated on $n =$ 50,000,000 generated samples and Merton-X likewise. From this pool a 10M-observation subsample was used for the aggregate statistics reported in the main chapter (Tab. 6.1). The reference comprises $n =$ 2,623,489 observations from the training dataset (see Sec. 4.1).

Aggregate Statistics Due to the generation constraints that have been applied to GARCH-X and Merton-X (Sec. D.4.1 and Sec. D.4.2), it was possible to bound the number of unlikely returns, meaning returns whose absolute value exceeds one, $|r_t| > 1$, to approximately 0% for GARCH-X and to 0.02% for Merton-X. Those few extreme generated values contribute to most of the kurtosis and skewness shifts.

Stylized Facts Here is a comprehensive table of the tail exponent estimation which

Statistic	Reference	GARCH-X	Merton-X
Mean	0.0005	0.0009	0.0005
Std	0.0232	0.0230	0.0364
Min	−0.9363	−17.5946	−8.2768
$q_{0.01}$	−0.0626	−0.0577	−0.0584
$q_{0.05}$	−0.0324	−0.0296	−0.0292
$q_{0.25}$	−0.0096	−0.0085	−0.0089
Median	0.0000	0.0009	0.0003
$q_{0.75}$	0.0105	0.0104	0.0098
$q_{0.95}$	0.0338	0.0316	0.0303
$q_{0.99}$	0.0649	0.0597	0.0598
Max	0.6931	10.4692	8.9261
Skewness	−0.5773	−7.5635	1.2102
Ex. Kurtosis	38.5	11021.7	3736.4

Table B.1: Full distributional statistics for the training reference, GARCH-X, and Merton-X. All values are reported to four decimal places. The reference sample is the raw training dataset ($n = 2{,}623{,}489$); each model was evaluated on $n = 50{,}000{,}000$ generated samples. Excess kurtosis is reported relative to the Gaussian (kurt − 3). The Reference median (−0.000019) rounds to 0.0000 at this precision.

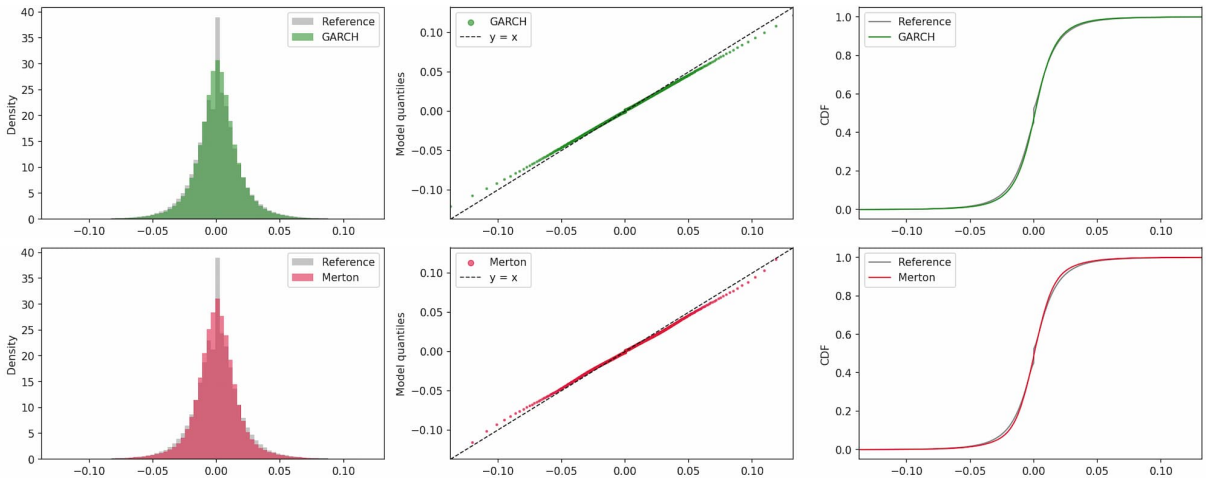


Figure B.1: From left to right, marginal distribution ($q_{0.001} - q_{0.999}$), QQ-plot, ECDF plot. *Top row*: GARCH-X. *Bottom row*: Merton-X.

make use of Alstott et al. [52] `powerlaw` python module on a subset estimation of $300,000$ values. The results are available in Tab. B.2.

Configuration	$\hat{\alpha}$	$x_{\min}$	D_{KS}	n**-tail**
Reference	4.532	0.1170	0.0138	901
GARCH-X	4.188	0.0725	0.0103	3266
Merton-X	3.344	0.0337	0.0284	23508

Table B.2: Heavy-tail (power-law) statistics for Reference, GARCH-X, and Merton-X. D_{KS} is the distance between the fitted power law and the empirical tail.

B.2 Phase–1

B.2.1 Aggregate Statistics

In this section the aggregate statistics for Sec. 6.3 are displayed in full in Tab. B.3. The configurations shared by all runs are: ODE sampler, n-windows = 512, sequence length = 2048, stride = 400, diffusion steps = 2000, observation-count = $1,048,576$. In this case, reference statistics are computed on the full set of training windows rather than on the raw dataset, so that the reference and generated samples are drawn from the same population. Since `stride` $< L$, consecutive windows overlap and the total number of observations exceeds that of the original dataset. The reference set comprises $n = 11{,}403{,}264$ observations across 5,568 windows of length $L = 2048$.

Images of all the marginal distribution metrics and analysis are displayed in Fig. B.2 for VE SDE, Fig. B.3 for GBM-style SDE, and Fig. B.4 for VP SDE.

B.2.2 Stylized Facts

In this sub-section we display the stylized-facts grids for the combinations of the VE, VP, and GBM SDEs with the different noise schedules. Specifically, positive heavy-tail distribution plots are visible in Fig. B.5, volatility clustering graphs in Fig. B.6, and the leverage effect in Fig. B.7.

	Ref.	VE			GBM			VP		
Statistic		**Cos.**	**Exp.**	**Lin.**	**Cos.**	**Exp.**	**Lin.**	**Cos.**	**Exp.**	**Lin.**
Mean	0.0005	0.0008	0.0005	0.0004	0.0000	0.0001	0.0001	0.0003	0.0011	0.0023
Std	0.0208	0.0221	0.0190	0.0270	0.1437	0.1453	0.2797	0.0235	0.0546	0.0446
Min	−1.0393	−0.8637	−0.7375	−1.0027	−1.1243	−1.2076	−1.4188	−0.9684	−1.4700	−3.7898
$q_{0.01}$	−0.0562	−0.0558	−0.0479	−0.0649	−0.3350	−0.3385	−0.6519	−0.0607	−0.1365	−0.0739
$q_{0.05}$	−0.0291	−0.0322	−0.0282	−0.0422	−0.2361	−0.2387	−0.4601	−0.0345	−0.0826	−0.0438
$q_{0.25}$	−0.0086	−0.0112	−0.0101	−0.0164	−0.0963	−0.0974	−0.1880	−0.0121	−0.0270	−0.0161
Median	0.0000	0.0008	0.0005	0.0004	0.0001	0.0001	0.0002	0.0003	−0.0010	0.0014
$q_{0.75}$	0.0095	0.0128	0.0112	0.0173	0.0969	0.0979	0.1886	0.0126	0.0269	0.0190
$q_{0.95}$	0.0304	0.0340	0.0293	0.0432	0.2361	0.2388	0.4600	0.0353	0.0935	0.0474
$q_{0.99}$	0.0583	0.0575	0.0492	0.0659	0.3342	0.3379	0.6516	0.0627	0.1539	0.0891
Max	0.7694	0.4840	0.6426	0.6988	0.7719	0.8477	1.3203	0.5055	0.8569	5.2992
Skewness	−0.5387	−0.5979	−0.0860	−0.1712	−0.0130	−0.0122	−0.0023	−0.2985	0.1530	6.3270
Ex. Kurt.	36.0	21.5	14.3	7.1	0.1	0.1	0.0	15.8	5.6	529.8

Table B.3: Full window-level distributional statistics for the training reference and all nine Phase 1 configurations ($L = 2048$). Reference comprises 5568 windows; each configuration is evaluated on 512 generated windows. Columns are grouped by SDE family (VE, GBM, VP) and noise schedule (Cos., Exp., Lin.).

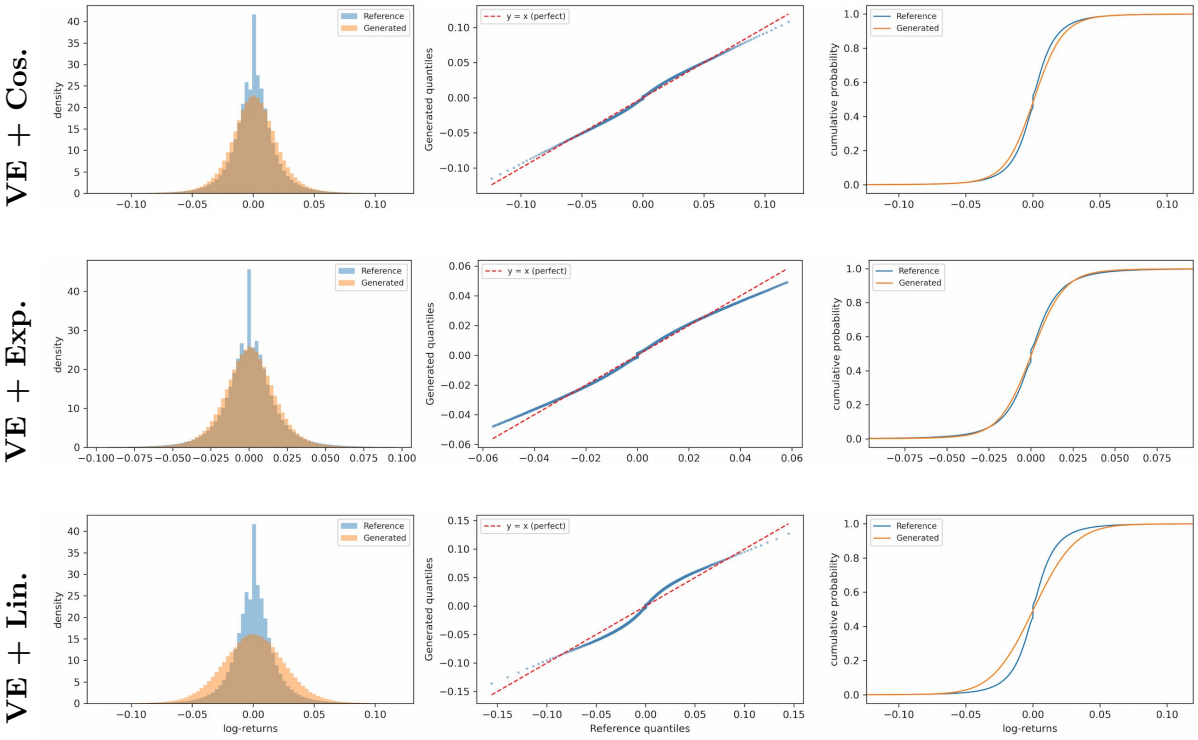


Figure B.2: Marginal distribution for the VE SDE family across all three noise schedules (cosine, exponential, linear). From left to right: full marginal distribution ($q_{0.001}$–$q_{0.999}$), QQ-plot, ECDF.

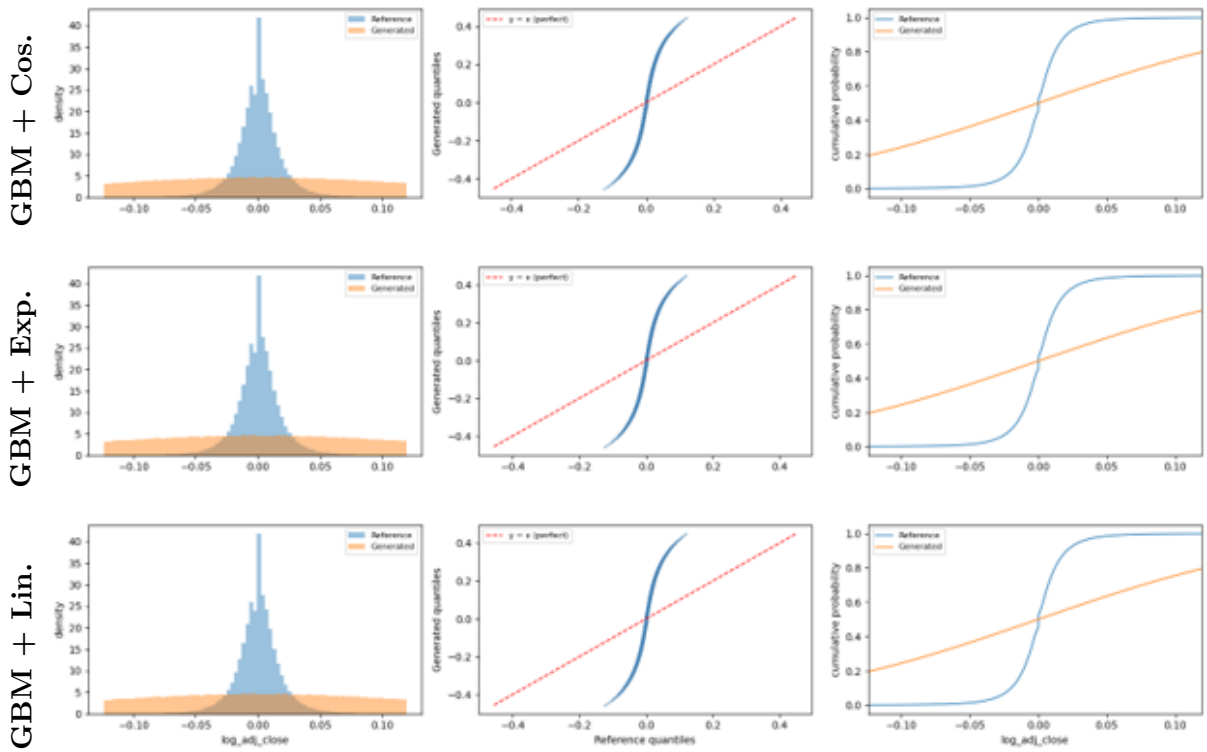


Figure B.3: Marginal distribution for the GBM SDE family across all three noise schedules (cosine, exponential, linear). From left to right: full marginal distribution ($q_{0.001}$–$q_{0.999}$), QQ-plot, ECDF.

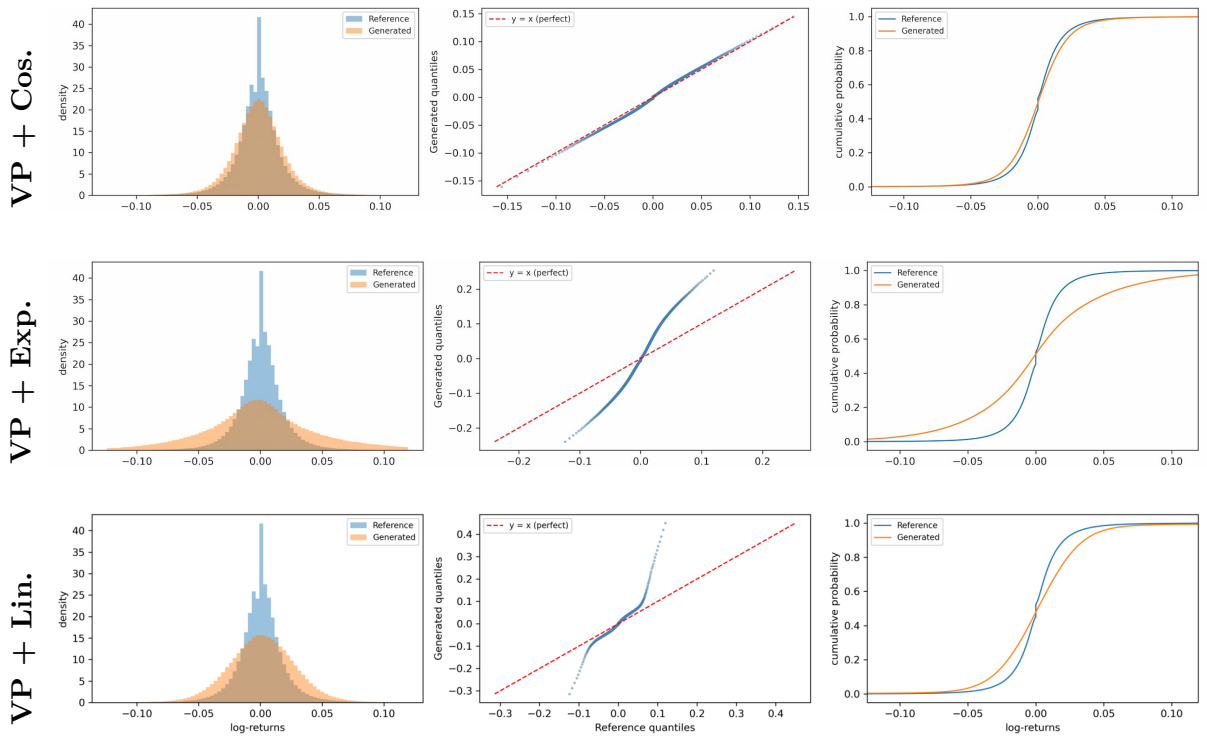


Figure B.4: Marginal distribution for the VP SDE family across all three noise schedules (cosine, exponential, linear). From left to right: full marginal distribution ($q_{0.001}$–$q_{0.999}$), QQ-plot, ECDF.

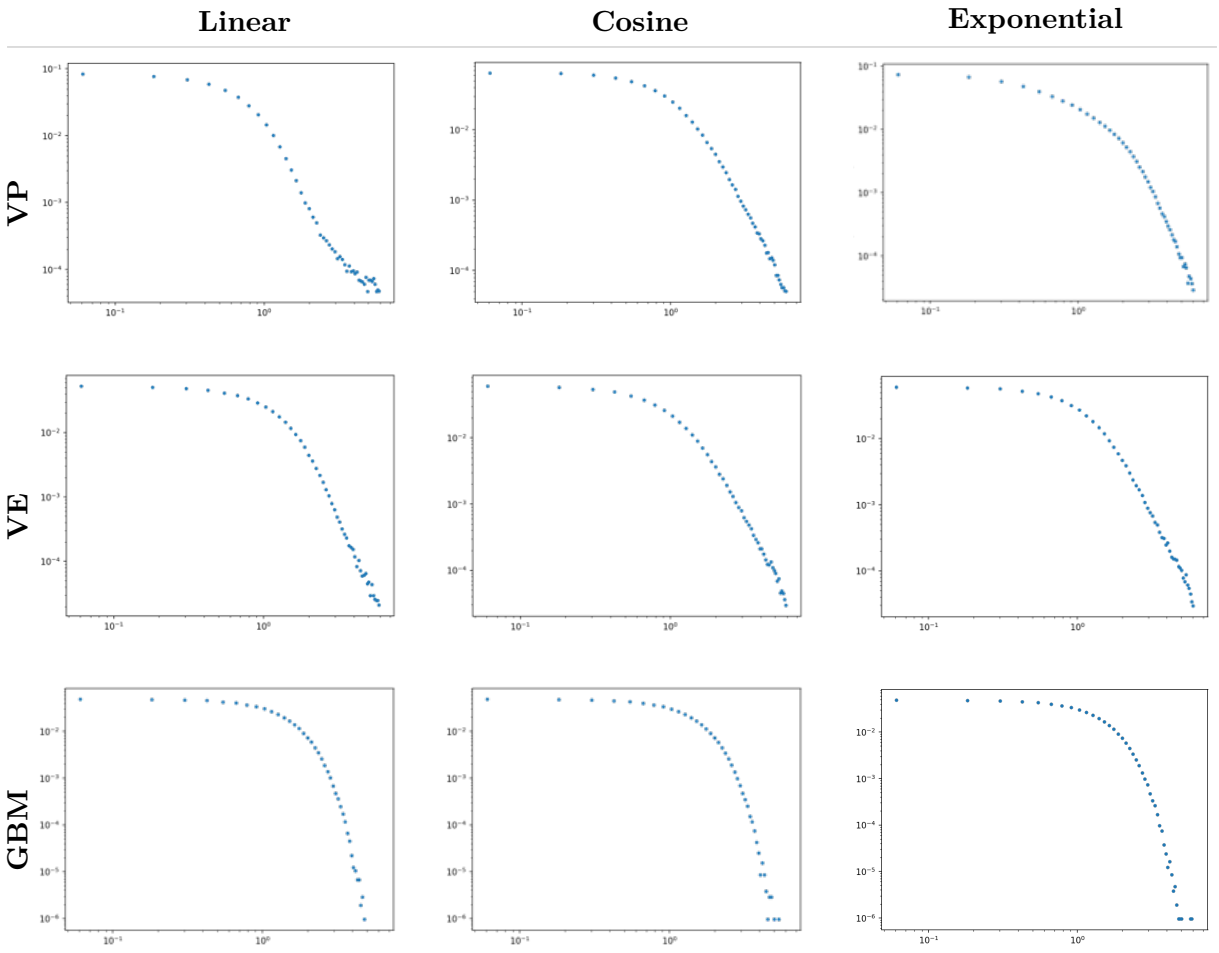


Figure B.5: Heavy-tail distribution (log–log scale, positive tail) for all nine Phase 1 configurations. *Rows* (top to bottom): VP, VE, and GBM SDE families. *Columns* (left to right): linear, cosine, and exponential noise schedules. The estimated power-law tail exponent $\hat{\alpha}$ is reported in Tab. 6.2; the empirical reference is $\hat{\alpha} = 4.378$.

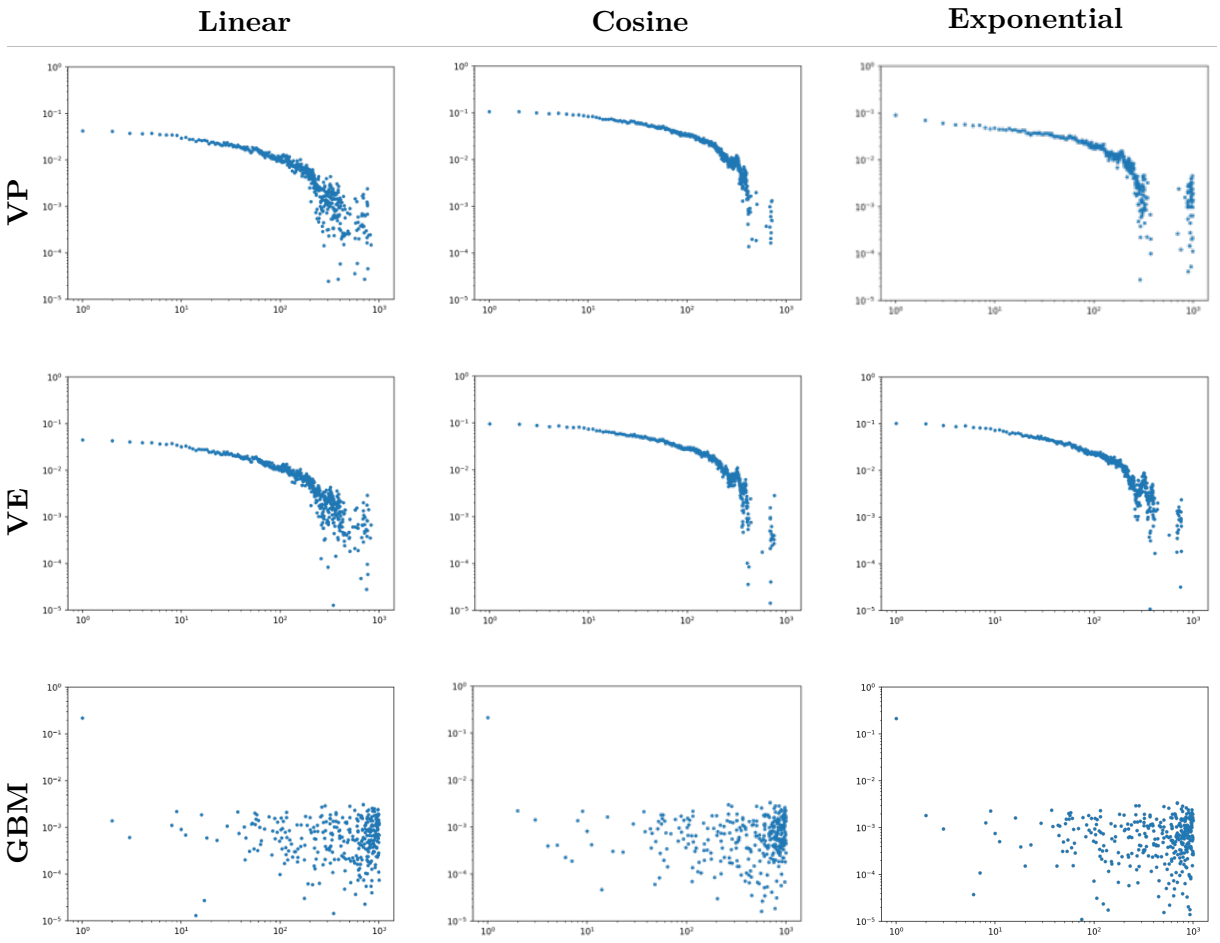


Figure B.6: Volatility clustering (autocorrelation of absolute returns on a log–log scale) for all nine Phase 1 configurations. Rows (top to bottom): VP, VE, and GBM SDE families. Columns (left to right): linear, cosine, and exponential noise schedules. Slow power-law decay indicates successful reproduction of long-range volatility dependence; rapid collapse or flattening indicates failure.

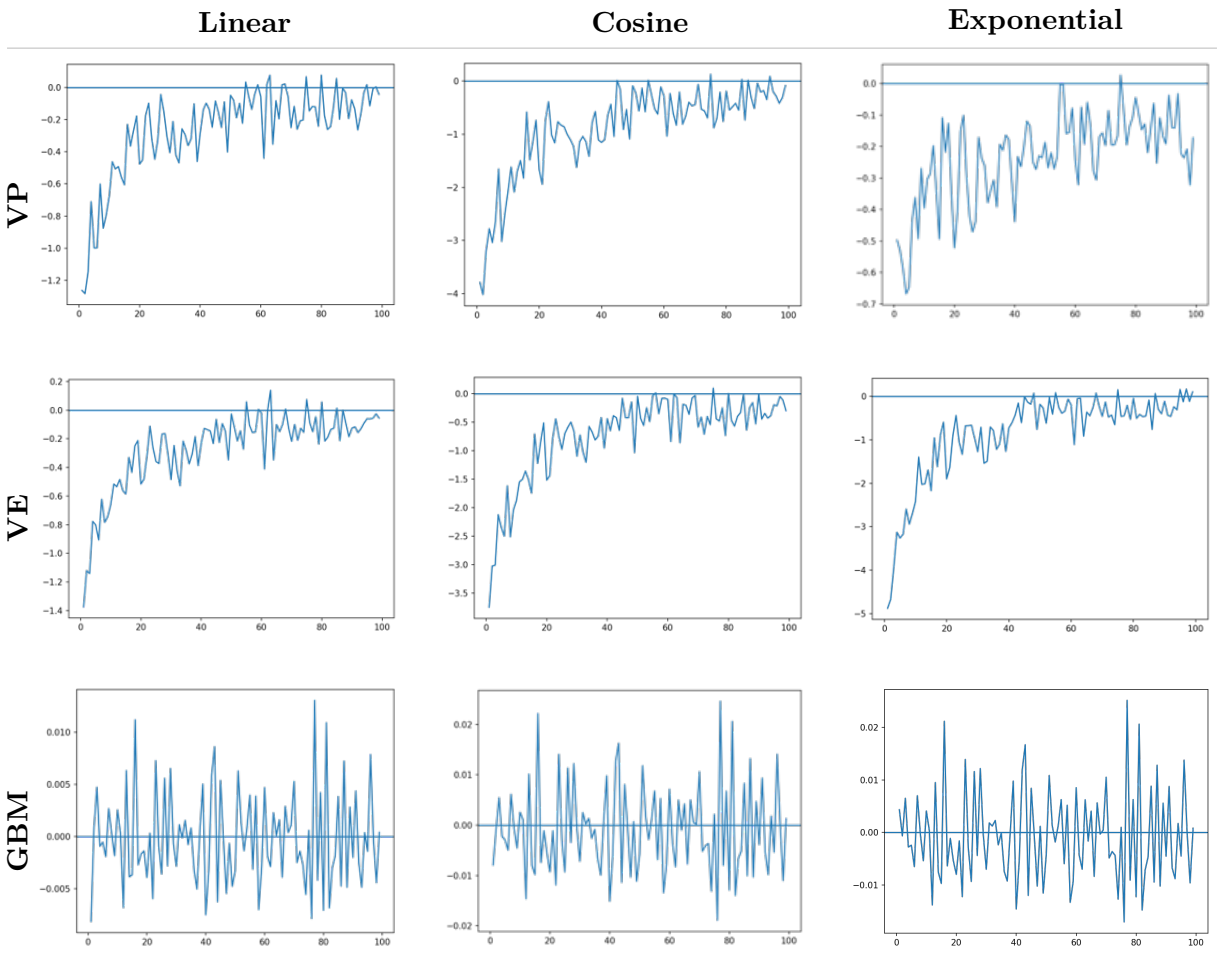


Figure B.7: Leverage effect (lead–lag correlation $L(k)$ between returns and future squared returns) for all nine Phase 1 configurations. Rows (top to bottom): VP, VE, and GBM SDE families. Columns (left to right): linear, cosine, and exponential noise schedules. Persistent negative values at small positive lags indicate successful reproduction of the leverage effect; oscillation around zero indicates failure.

B.3 Phase 2

B.3.1 Phase 1 Champions

In this section are presented the extended versions of the results concerning the two generalized models VE SDE with exponential and cosine schedules of Phase 1.

Aggregate Statistics The full aggregate-statistics table is displayed in Tab. B.4.

Statistic	**Reference**	**VE+Exp.**	**VE+Cos.**
Mean	0.0005	0.0003	0.0004
Std	0.0209	0.0199	0.0209
Min	−0.9363	−1.0095	−1.0032
$q_{0.01}$	−0.0563	−0.0508	−0.0535
$q_{0.05}$	−0.0292	−0.0300	−0.0311
$q_{0.25}$	−0.0086	−0.0108	−0.0110
Median	0.0000	0.0004	0.0004
$q_{0.75}$	0.0095	0.0115	0.0118
$q_{0.95}$	0.0305	0.0307	0.0321
$q_{0.99}$	0.0583	0.0514	0.0553
Max	0.6931	0.9388	1.2113
Skewness	−0.5864	−0.2382	−0.2368
Ex. Kurt.	39.5	14.3	16.6
$D_{\mathrm{KS}} \downarrow$	—	0.0436	0.0458
$W_1 \downarrow$	—	0.0019	0.0020

Table B.4: Full aggregate statistics for the Phase 1 champion models ($L = 512$, stride= 100), computed over all windows (training and validation combined).

Stylized Facts Subsequently, it is possible to observe the stylized facts: heavy-tail distribution, volatility clustering, and leverage effect for the models VE+Exponential and VE+Cosine against the reference, reported in Ch. 6, Fig. 6.6. Details about the heavy-tail distribution are provided in Tab. B.5.

Statistic	Reference	VE+Exp.	VE+Cos.
$\hat{\alpha}$	4.418	4.634	4.577
$x_{\min}$	0.0680	0.0529	0.0579
D_{KS}	0.0134	0.0090	0.0145
n-**tail**	3994	5423	4929

Table B.5: Heavy-tail (power-law) statistics for VE+Exponential and VE+Cosine ($L = 512$), computed over all windows (training and validation combined). D_{KS} is the distance between the fitted power law and the empirical tail.

B.3.2 Unconditional and Conditional EDM-CSDI

In this sub-section the detailed data regarding the EDM-CSDI (Unc.) and EDM-CSDI (Cond.) models are reported.

Aggregate Statistics The aggregate statistics that show distributional characteristics for EDM-CSDI (Unc.) and EDM-CSDI (Con.) are reported in Tab. B.6.

	Validation			Both		
Statistic	**GT**	**Unc.**	**Con.**	**Ref.**	**Unc.**	**Con.**
Mean	-0.0024	0.0031	0.0033	0.0000	0.0008	0.0029
Std	0.8656	0.7314	0.8299	1.0000	0.7523	0.9487
Min	−13.8337	−32.0912	−14.6549	−44.8905	−45.0383	−60.237
$q_{0.01}$	−2.3305	−1.9595	−2.2396	−2.7188	−2.0292	−2.5789
$q_{0.05}$	−1.2871	−1.0868	−1.2335	−1.4207	−1.117	−1.3620
Median	−0.0216	−0.0214	−0.0172	−0.0216	−0.0210	−0.0189
$q_{0.95}$	1.3035	1.11312	1.2566	1.4381	1.132	1.3797
$q_{0.99}$	2.3723	2.0232	2.2635	2.7746	2.0855	2.6306
Max	10.4889	14.2408	11.9020	33.1964	82.230	34.4689
Skewness	−0.4069	−0.3558	−0.4432	−0.5864	−0.2292	−0.5390
Ex. Kurt.	13.9	19.9	13.9	39.5	20.0	39.0
$D_{\mathrm{KS}} \downarrow$	—	0.0331	0.0352	—	0.0418	0.0411
$W_1 \downarrow$	—	0.0020	0.0005	—	0.0029	0.0006

Table B.6: Full aggregate statistics for EDM-CSDI (Unc.) and (Cond.) on normalized log-returns. *Validation*: GT vs. generated; *Both*: full Reference vs. generated. D_{KS} and W_1 are computed against the unnormalized training marginal.

Stylized Facts The stylized-facts figures for both EDM-CSDI (Unc.) and EDM-CSDI (Cond.) are reported in Ch. 6, respectively Fig. 6.8 and Fig. 6.10, while statistics about the heavy-tail distribution are made available in Tab. B.7.

Sample Paths Additional sample paths generated by EDM-CSDI (Unc.) and

EDM-CSDI (Cond.) are illustrated in Fig. B.9 and Fig. B.8, respectively, while the aggregate sample-path distribution is shown in Fig. B.10 for the unconditional and Fig. 6.12 for the conditional model.

	Validation			**Both**		
Statistic	**GT**	**Unc.**	**Con.**	**Ref.**	**Unc.**	**Con.**
$\hat{\alpha}$	4.072	4.768	4.278	4.418	4.640	4.572
$x_{\min}$	1.6504	2.4861	2.1686	3.6664	2.4406	4.2274
D_{KS}	0.0235	0.0226	0.0180	0.0208	0.0276	0.0086
n-**tail**	3486	3494	1429	2869	4003	3137

Table B.7: Heavy-tail (power-law) statistics for EDM-CSDI (Unc.) and (Cond.) ($L = 512$), Validation and Both splits. *Validation* is computed against the Ground Truth (GT) and *Both* against the full Reference. D_{KS} is the distance between the fitted power law and the empirical tail.

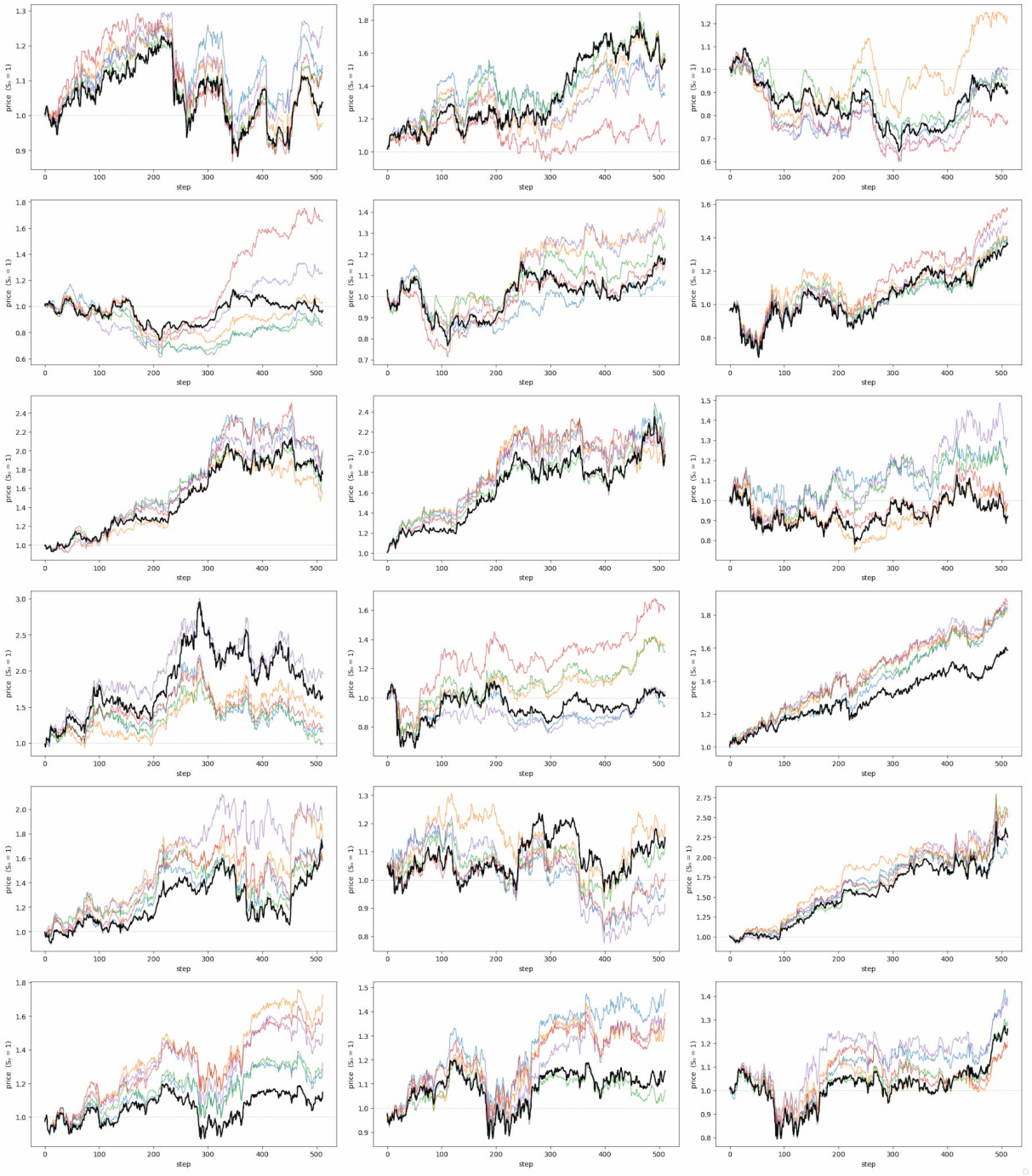


Figure B.8: Sample price paths for EDM-CSDI (Con.).

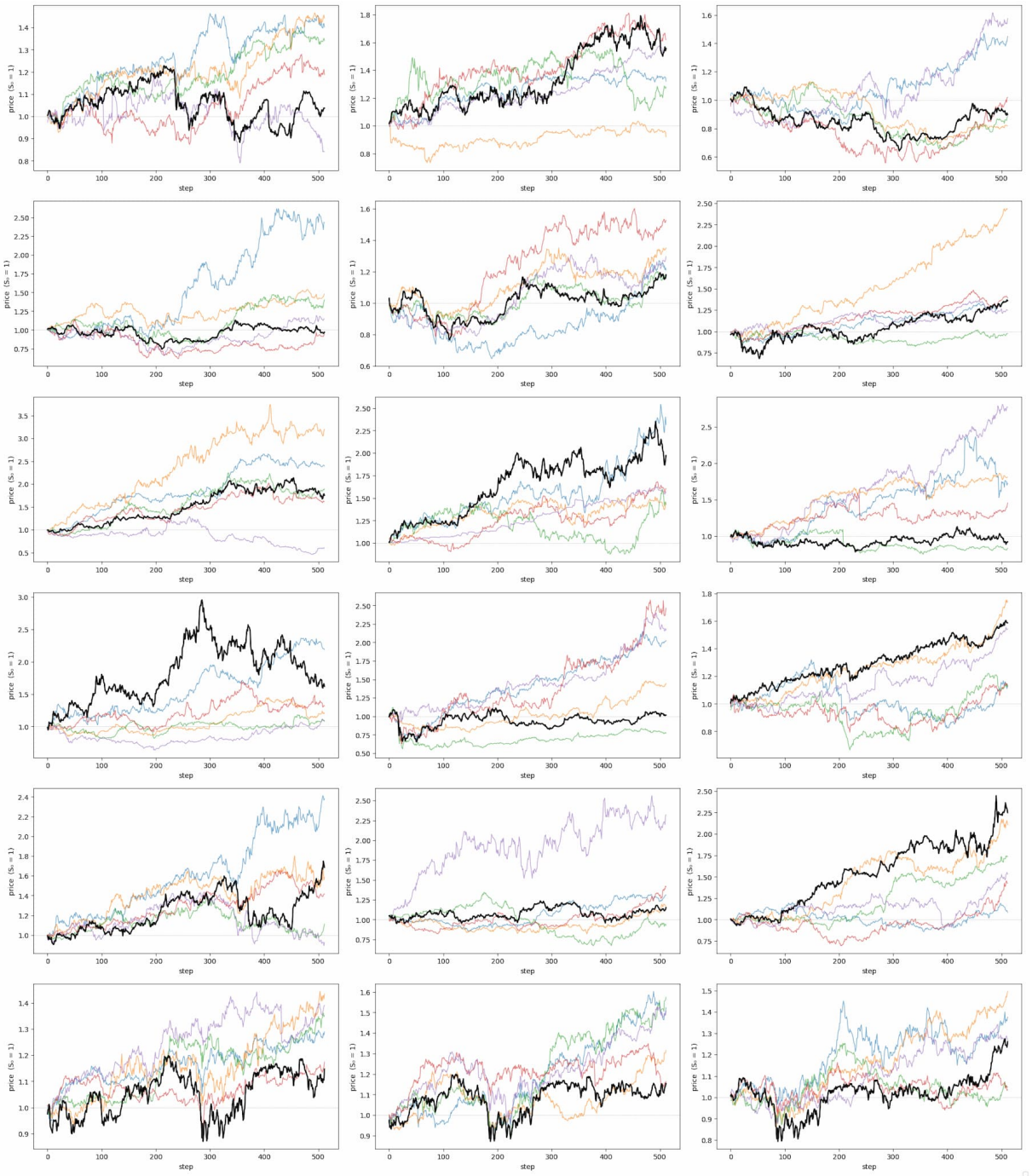


Figure B.9: Sample price paths for EDM-CSDI (Unc.).

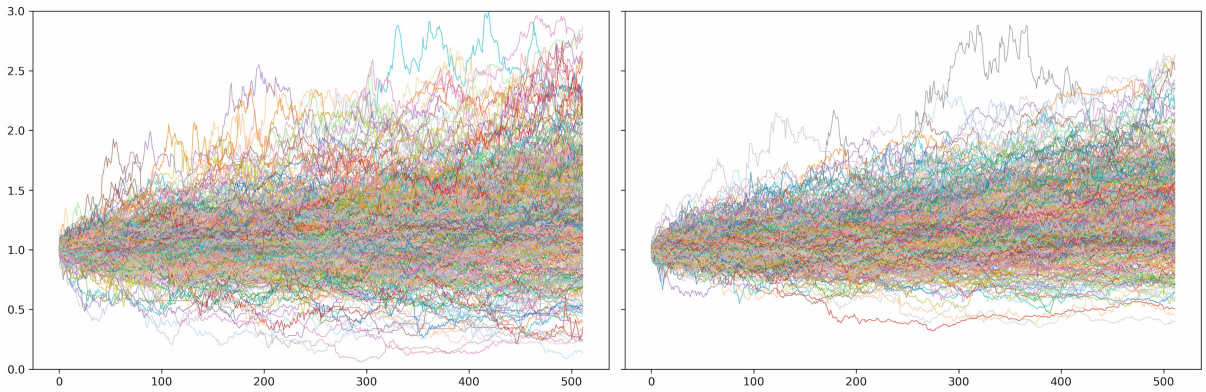


Figure B.10: Samples of generated price path by EDM+CSDI (Unc.) aggregated.

Appendix C

Additional Experiments and Analysis

In this chapter, we collect the additional explorations and analyses that complement the main results, grouped along two lines: how the model behaves when the sampling procedure is varied, and how it responds when two of the interpretive claims made earlier are stressed directly.

The first line concerns the generation phase and asks whether sample improvements are possible, in particular when stochasticity is re-introduced. Two sampling explorations are covered: the second-order Heun stochastic sampler (Heun SDE for brevity) and an NFEs exploration. Specifically, the first evaluates whether the model can recover some peculiarities of FTS paths through the injection of stochasticity, while the second checks whether the reconstruction error is already under control with the current settings ($NFEs = 400$ or $steps = 200$), or whether there is margin for improvement or reduction, given that this was an upper bound.

The second line revisits two open questions left by the main text. Sec. C.2 tests the conditioning hypothesis of Sec. 7.1 (Ch. 7) through a regime-stratified analysis of generated volatility, checking whether same-day OHL conditioning lets EDM-CSDI (Cond.) reproduce period-specific volatility levels that the unconditional model averages away. Finally, Sec. C.3 reports a capacity ablation that halves the model's channel width to probe the capacity-to-data concern raised in Sec. 7.2, asking whether the model's accuracy reflects genuine generalisation rather than partial memorisation of the training

windows.

C.1 EDM-CSDI Conditional Sampling Variations

C.1.1 Heun Stochastic Sampler

In the first phase, we generated data under the same conditions as Sec. 6.4.3, using EDM-CSDI (Cond.) with a stochastic sampler instead. Specifically, we experimented with a budget of $S_{\text{churn}} = 10$, $S_{\text{tmin}} = 0.05$, $S_{\text{tmax}} = \infty$, and $S_{\text{noise}} = 1.0$. Over $20,000$ windows, the distributional properties remained largely unchanged.

Sampler	**Mean**	**Std**	**Skew.**	**Kur.**	$q_{0.01}$	$q_{0.99}$
Reference	0.0000	1.0000	−0.5864	39.5	−2.7188	2.7746
Heun ODE	0.0029	0.9487	−0.5390	39.0	−2.5789	2.6306
Heun SDE	0.0012	1.0041	−0.6876	45.6	−2.7224	2.7662

Table C.1: Aggregate statistics for EDM-CSDI (Cond.) under the deterministic Heun ODE and stochastic Heun (SDE, $S_{\text{churn}} = 10$) samplers, normalized log-returns. This is for combined windows.

Sampler	D_{KS}	W_1	$\hat{\alpha}$
Reference	—	—	4.418
Heun ODE	0.0411	0.000569	4.572
Heun SDE	0.0377	0.000077	4.599

Table C.2: Distributional similarity metrics (D_{KS}, W_1, $\hat{\alpha}$) for EDM-CSDI (Cond.) under the Heun ODE/SDE samplers (cf. Tab. C.1).

As shown in Tab. C.1 and Tab. C.2, the advantage of using SDE is that the injection of noise brings new possible trajectories, increasing the standard deviation. While this is positive, as it brings us closer to the original distribution, it also causes kurtosis to increase and skewness to grow beyond the reference. This is probably due to the stochasticity pushing the model into regions that are seldomly seen, hence poorly learned.

Stylized Facts Fig. C.1 compares the heavy-tail distribution, volatility clustering, and leverage effect obtained with the Heun ODE and Heun SDE samplers against the Reference. The Heun SDE sampler generated paths display somewhat more variability in leverage effects, while matching Heun ODE and Reference properties.

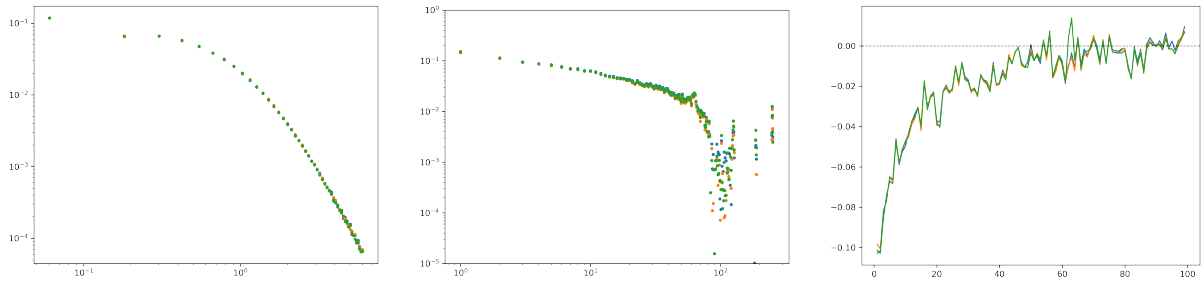


Figure C.1: Stylized facts for EDM-CSDI (Cond.) under the Heun ODE (orange) and Heun SDE (green) samplers, against the Reference (blue). From left to right: heavy-tail distribution, volatility clustering, leverage effect.

CRPS and Interval Coverage To assess whether the gains in distributional similarity reported in Tab. C.1 and Tab. C.2 translate into probabilistic forecasting performance, the CRPS decomposition and coverage analysis of Sec. 6.4.3 were repeated for the Heun SDE sampler, using the same $N = 25$ samples per window as for Heun ODE to allow a direct comparison.

Configuration	**Split**	**Sampler**	**N. Samples**	**MAE** ↓	**CRPS** ↓	$\mathbf{T_1^{norm}}$	$\mathbf{T_1}$	$\mathbf{T_2^{norm}}$
Climatology	Val	—	25	0.619 13	0.448 69	0.897 45	0.018 73	0.897 53
EDM-CSDI (Cond.)	Val	Heun ODE	25	0.186 00	0.121 44	0.244 38	0.005 10	0.245 88
EDM-CSDI (Cond.)	Val	Heun SDE	25	0.181 00	0.120 40	0.238 33	0.004 97	0.235 87
Climatology	Both	—	25	0.662 88	0.482 32	0.964 66	0.020 13	0.964 29
EDM-CSDI (Cond.)	Both	Heun ODE	25	0.195 46	0.126 02	0.257 31	0.005 37	0.262 58
EDM-CSDI (Cond.)	Both	Heun SDE	25	0.188 75	0.123 89	0.249 81	0.005 21	0.251 83

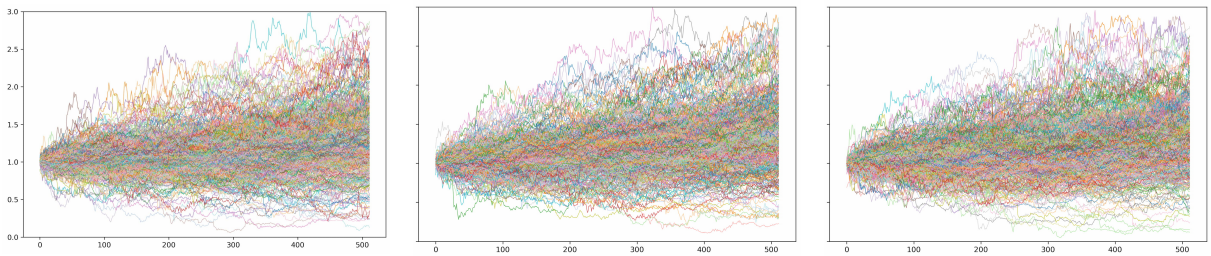
Table C.3: CRPS decomposition for the EDM-CSDI models against the climatology baseline. CRPS is decomposed as $\mathrm{CRPS} = \mathrm{T}_1^{\mathrm{norm}} - \frac{1}{2}\mathrm{T}_2^{\mathrm{norm}}$, where T_1 is the accuracy term unnormalised by $\sigma = 0.020867$. **N. Samples** is to be intended for each window. Lower CRPS is better.

Split	Nominal (%)	EDM-CSDI 25, Heun ODE	EDM-CSDI 25, Heun SDE
Val	50.0	50.59	50.95
Val	80.0	78.41	78.41
Val	90.0	86.66	86.47
Val	95.0	91.34	91.06
Val	99.0	93.46	93.23
Both	50.0	51.57	51.53
Both	80.0	78.95	78.75
Both	90.0	86.87	86.58
Both	95.0	91.46	91.11
Both	99.0	93.60	93.26

Table C.4: Empirical coverage at nominal levels for EDM-CSDI with 25 samples, and EDM-CSDI with 50 samples. Values closer to the nominal level indicate better-calibrated prediction intervals.

For the same number of samples, Heun SDE achieves a lower CRPS and MAE than Heun ODE in both splits (Both: CRPS 0.12389 vs. 0.12602; Val: 0.12040 vs. 0.12144), consistent with the closer match to the reference standard deviation and the gain in W_1 reported previously. However, Tab. C.4 shows that coverage at the 90%, 95%, and 99% levels is consistently lower of 0.2–0.35 percentage points than Heun ODE, which is already overconfident at these levels: the extra variability injected by the SDE sampler improves the overall fit but does not widen the predictive intervals enough to fix the upper-tail miscalibration.

Price Paths Samples Fig. C.2 illustrates sample price paths generated under the Heun ODE and Heun SDE samplers against the Ground Truth were it is possible to observe that SDE carries more variability and breadth in its generated paths.



Ground Truth Heun ODE Heun SDE

Figure C.2: Sample price paths for EDM-CSDI (Cond.). *Left*: Ground Truth. *Centre*: Heun ODE. *Right*: Heun SDE.

C.1.2 NFEs Variations

Another set of experiments focused on sampling with EDM-CSDI (Cond.) under the Heun ODE sampler with a different number of function evaluations (NFEs), specifically $NFEs = 200$ ($steps = 100$) and $NFEs = 600$ ($steps = 300$), to evaluate whether any discernible difference would be observed with respect to the default configuration $NFEs = 400$ ($steps = 200$) used in Sec. 6.4.3. For computational-speed reasons given the HPC constraints, the generated set for this exploration was limited to $10,000$ windows with 5 samples each (except for Heun ODE 400), rather than the full $20,000$-window set used elsewhere in this chapter.

Sampler	**NFEs**	**Mean**	**Std**	**Skew.**	**Kur.**	$q_{0.01}$	$q_{0.99}$
Reference	—	0.0000	1.0000	−0.5864	39.5	−2.7188	2.7746
Heun ODE	200	0.0028	0.9490	−0.5280	39.8	−2.5800	2.6300
Heun ODE	400	0.0029	0.9487	−0.5390	39.0	−2.5789	2.6306
Heun ODE	600	0.0028	0.9490	−0.5280	39.8	−2.5800	2.6300

Table C.5: Aggregate statistics for EDM-CSDI (Cond.) under the Heun ODE sampler with varying numbers of function evaluations (NFEs), normalized log-returns, computed over all (training and validation) windows. $NFEs = 400$ ($steps = 200$) is the default configuration used throughout this chapter and reproduces the Heun ODE row of Tab. C.1 ($20,000$ windows, 5 samples each); $NFEs = 200$ ($steps = 100$) and $NFEs = 600$ ($steps = 300$) were generated on the reduced $10,000$-window, 5-samples-per-window set described above.

Sampler	**NFEs**	D_{KS}	W_1	$\hat{\alpha}$
Reference	—	—	—	4.418
Heun ODE	200	0.0402	0.000558	4.554
Heun ODE	400	0.0411	0.000569	4.572
Heun ODE	600	0.0403	0.000561	5.001

Table C.6: Distributional similarity metrics (D_{KS}, W_1, $\hat{\alpha}$) for EDM-CSDI (Cond.) under the Heun ODE sampler with varying NFEs (cf. Tab. C.5). D_{KS} and W_1 are computed against the unnormalized training marginal. Cells marked — were not computed for this exploration.

As shown in Tab. C.5, all three NFE configurations are nearly indistinguishable from one another and from the Reference, with standard deviation, skewness, kurtosis, and tail quantiles all within a narrow band around the $NFEs = 400$ baseline.

Tab. C.6 indicates that $NFEs = 200$ and $NFEs = 600$ yield marginally lower D_{KS} and W_1 than $NFEs = 400$. Overall, departing from $NFEs = 400$ in either direction does not yield a clear, consistent improvement; part of the observed differences may also be attributable to the reduced $10,000$-window evaluation set used for $NFEs = 200$ and $NFEs = 600$ rather than to the NFE budget itself. Nevertheless, decreasing the number of steps by 50% does not seem to be causing lower generative performances.

C.2 Volatility-Regime Analysis

In Sec. 7.1 it was hypothesised that same-day OHL conditioning acts as a "missing regime indicator", allowing EDM-CSDI (Cond.) to avoid the regime-averaging behaviour of the unconditional model. This analysis address this statement and test it directly for EDM-CSDI (Cond.) and EDM-CSDI (Unc.). It is shown that the models can reproduce to a different extent the period-specific level of volatility associated with historical episodes (the dot-com unwind, the 2008 Global Financial Crisis, the 2020 COVID shock).

Method The standard deviation of the $L = 512$ Close log-returns of a window is used as a volatility proxy. Over the 25,226 reference windows (210 tickers, stride 100), overlapping windows covering the same calendar date are averaged into a date-level signal spanning the 16,175 trading dates of the dataset (1962–2026). Its peaks align with real market shocks, such as the GFC and COVID. Reference windows are split into Low/Medium/High regimes by the tertiles of this proxy ($q_{1/3} = 0.0145$, $q_{2/3} = 0.0203$, unnormalised log-return units), with thresholds computed only on reference data to avoid leakage. Each of the 20,000 generated windows (5 samples each, Sec. 6.4.3) is matched via the tracked `(file, window_start)` keys, to its reference period. The generated proxy (std of the 512 steps, averaged over the 5 samples) is tested for monotonic ordering across regimes with a Kruskal–Wallis omnibus test, Bonferroni-corrected pairwise Mann–Whitney U tests, and an OLS correlation against the reference proxy. A 20-day rolling-std variant repeats the analysis at finer time resolution, exposing short-lived crisis bursts that the window-level statistic dilutes.

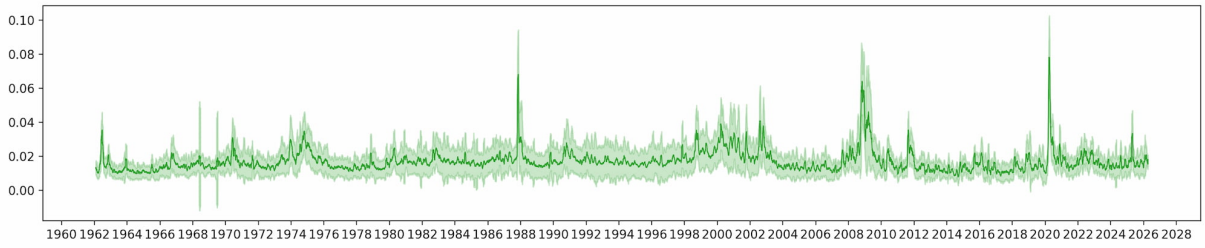


Figure C.3: Reference volatility proxy vs Mean across tickers of 20-day standard deviation rolling average with ±1 standard deviation bands.

Results For both models the Kruskal–Wallis test rejects equal distributions across regimes ($H = 5204$ for EDM-CSDI (Cond.), $H = 7408$ for EDM-CSDI (Unc.); both $p \approx 0$), and every pairwise Mann–Whitney comparison[1] orders the regimes correctly (High > Medium > Low): both variants encode period-specific volatility. They differ in how well they reproduce its *magnitude* (Tab. C.7). The reference High/Low ratio is 1.70. EDM-CSDI (Cond.) reproduces it almost exactly (1.77, ≈ 104%), with near-unity Gen/Ref ratios across all regimes and individual windows reaching crisis-level dispersion above 0.10. EDM-CSDI (Unc.) compresses the range to 1.45 (≈ 85%), with Gen/Ref ratios decreasing monotonically from 0.87 to 0.74 and generated dispersion capped near 0.033, nevertheless its regime ordering correlates more tightly with the reference ($r = 0.69$ vs. $r = 0.50$ for Cond.).

Mean volatility proxy	Low	Medium	High
Reference	0.0138	0.0177	0.0235
EDM-CSDI (Unc.)	0.0120	0.0143	0.0173
EDM-CSDI (Cond.)	0.0131	0.0166	0.0232
Gen/Ref (Unc.)	0.87	0.81	0.74
Gen/Ref (Cond.)	0.95	0.94	0.99

Table C.7: Mean volatility proxy (flat 512-step std, unnormalised log-return units) by reference regime, and the corresponding generated/reference ratios for EDM-CSDI (Cond.) and (Unc.).

Temporal Embedding as an Implicit Regime Indicator EDM-CSDI (Unc.) recovers the regime ordering because as discussed in Sec. 3.7 the `unconditional` mask retain the absolute temporal embedding of each window's calendar date, so the model still

[1]Scipy's functions (`scipy.stats`) [54] were used to compute Kruskal–Wallis (`kruskal`) test and Mann–Whitney (`mannwhitneyu`) comparison.

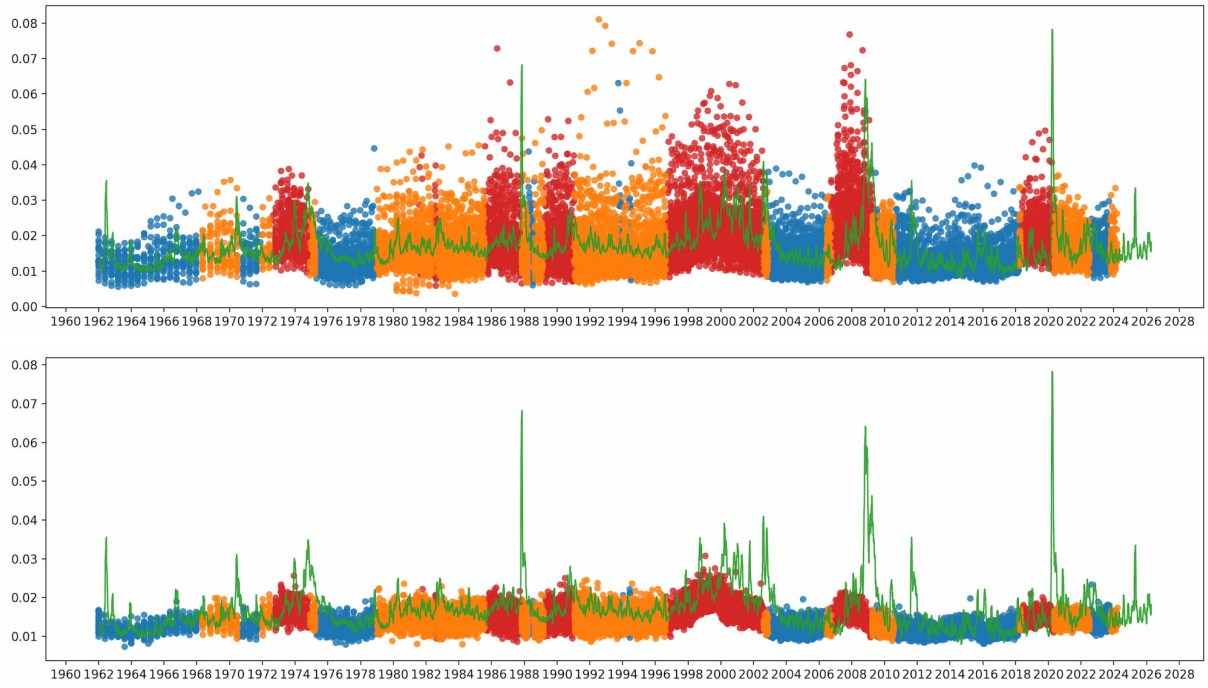


Figure C.4: Reference line (green) overlay Generated volatility proxy: mean 20-day standard deviation rolling average. *Top.* EDM-CSDI (Con.). *Bottom.* EDM-CSDI (Unc.). High (red dots), Medium (orange dots), and Low (blue dots).

learns $p_\theta(x \mid \tau)$ rather than the fully marginal $p(x)$. This embedding alone is sufficient to recover the historical ordering of volatility regimes, while OHL conditioning is what enable the model to match the magnitude and crisis-scale of extremes, consistent with the excess-kurtosis gap of Tab. 6.5.

Time-Resolved View The 20-day rolling-std overlay (Fig. C.4) sharpens this contrast: EDM-CSDI (Cond.) produces rolling-volatility bursts located with the 2008 and 2020 reference spikes, while EDM-CSDI (Unc.) remains within 0.02–0.03 and misses the ~ 0.08 peaks. Consistently, the across-sample spread per window is small for (Cond.) ($\approx 2.5\%$ of the mean) but large for (Unc.) ($\approx 25\%$). Therefore, the OHL conditioning binds the conditional samples to the realized window, while the unconditional samples, driven only by the temporal embedding, revert toward the regime average.

Summary These tests confirms that the regime-indicator hypothesis of Sec. 7.1: EDM-CSDI (Cond.) reproduces both the ordering and the magnitude of historical volatility regimes, including crisis-scale extremes, while EDM-CSDI (Unc.) recovers only the ordering, via its retained temporal embedding. As a caveat, the proxy summarises the

volatility level of a window rather than its persistence (the autocorrelation structure was already examined via the stylized facts of Sec. 6.4).

C.3 Capacity Ablation and Analysis

In Sec. 7.2 it was noted that EDM-CSDI (Cond.) carries ≈ 3.4M parameters against only ≈ 2.7M unique observations, which raises the question of whether its accuracy reflects genuine generalisation or partial memorisation of the training windows. This analysis address this concern directly by retraining the model with the number of channels halved from 128 to 64. The change lowers the parameters of each ResidualBlock from 545,152 to 182,592, and the total from 3,419,777 to 1,244,417, bringing the capacity-to-data ratio below one half. The number of residual layers (six) and every other setting of Sec. 6.4.3 were left unchanged, so that capacity is the only varying factor. The rationale is that a model relying on memorisation should lose the most when capacity is removed, and primarily on the held-out windows, whereas a model that has learned a generalisable conditional law should degrade gracefully and not asymmetrically across splits.

Marginal Distribution As shown in Tab. C.8, the bulk of the distribution is robust to the capacity reduction: on the *Both* split D_{KS} and W_1 are essentially unchanged with respect to the 128-channel model (0.0390 vs. 0.0411, and ≈ 0.0006 for both), and the central quantiles remain close to the reference. The degradation concentrates instead in the higher moments, where capacity matters most. On *Both*, skewness collapses towards zero (-0.0024 against a reference -0.5864) and excess kurtosis inflates to 114.4; this is however driven by a handful of unstable windows (normalised min/max of $-70.5/123.4$), analogous to the VP+Linear behaviour of Sec. 6.3, rather than by a genuinely heavier tail. Consistently, the power-law exponent rises to $\hat{\alpha} = 4.94$ (against 4.42), i.e. a *lighter* bulk-tail accompanied by sparse extreme outliers. The held-out *Validation* split is by contrast well behaved, with a moderate kurtosis of 21.4 and a tail exponent ($\hat{\alpha} = 4.11$) that is actually closer to its ground truth (4.07) than the 128-channel model (4.28), indicating that the smaller model still generalises the tail shape on unseen conditioning

windows. The zero-return spike, as in every other configuration, is not recovered, and is in fact slightly more oversmoothed than at 128 channels, more markedly on *Validation* than on *Both*, while the QQ-plot confirms that despite the high kurtosis the tails remain underestimated (Fig. C.5).

Split	**Configuration**	**Mean**	**Std**	**Skew.**	**Kur.**	$q_{0.01}$	$q_{0.99}$	$D_{\mathrm{KS}} \downarrow$	$W_1 \downarrow$	$\hat{\alpha}$
Val	Ground Truth	−0.0024	0.8656	−0.4069	13.9	−2.3305	2.3723	—	—	4.072
Val	EDM-CSDI (Cond.), 128 ch	0.0033	0.8299	−0.4432	13.9	−2.2396	2.2635	0.0352	0.0005	4.278
Val	EDM-CSDI (Cond.), 64 ch	0.0071	0.9190	−0.1553	21.4	−2.5076	2.5359	0.0425	0.0007	4.110
Both	Reference	0.0000	1.0000	−0.5864	39.5	−2.7188	2.7746	—	—	4.418
Both	EDM-CSDI (Cond.), 128 ch	0.0029	0.9487	−0.5390	39.0	−2.5789	2.6306	0.0411	0.0006	4.572
Both	EDM-CSDI (Cond.), 64 ch	0.0056	0.9394	−0.0024	114.4	−2.5408	2.5999	0.0390	0.0006	4.936

Table C.8: Aggregate statistics for normalized log-returns of EDM-CSDI (Cond.) at 128 and 64 channels. *Val*: validation windows vs. Ground Truth (GT); *Both*: validation and training combined vs. the full Reference. D_{KS} and W_1 are computed against the unnormalized training marginal. The 128-channel rows reproduce Tab. 6.5.

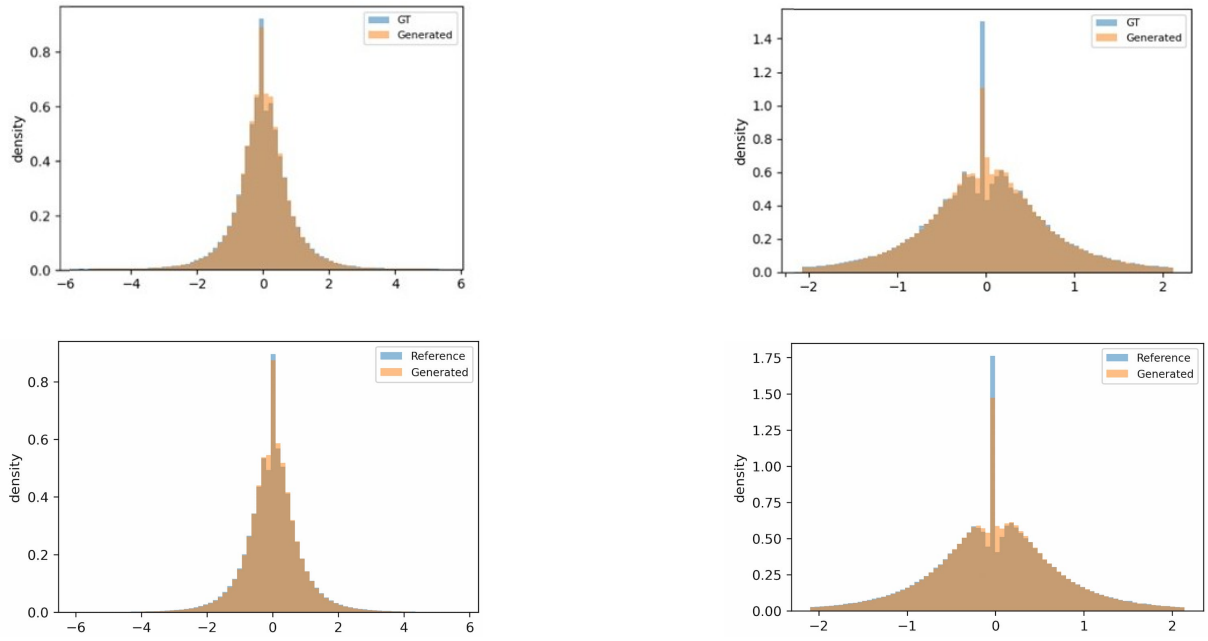


Figure C.5: Marginal distribution against the empirical reference (blue) for the 64-channel EDM-CSDI (Cond.) model. *Top.* validation windows only (GT). *Bottom.* combined validation and training windows. *Left.* full density range $q_{0.001}$–$q_{0.999}$. *Right.* central range $q_{0.02}$–$q_{0.98}$.

Stylized Facts The stylized facts remain fairly well reproduced, although visibly less sharply than at 128 channels (Fig. C.6). The heavy-tail distribution and volatility clustering stay reasonably close to the reference on both splits, while the leverage effect

and the magnitude of the tails are matched less accurately, consistent with the reduced capacity available to encode the higher-order dependence structure of the windows.

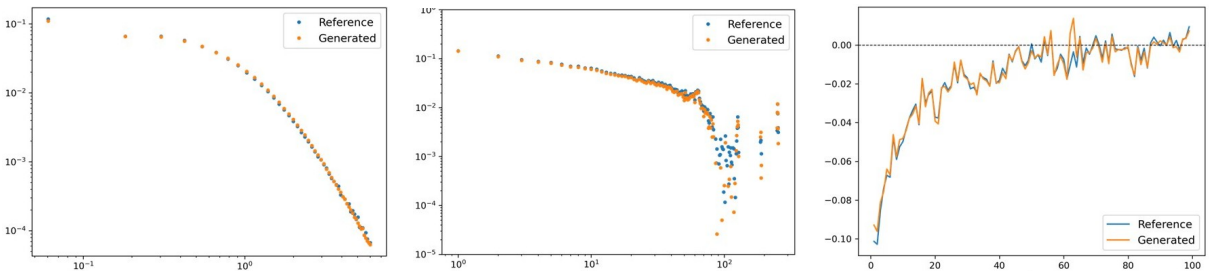


Figure C.6: Stylized facts for the 64-channel EDM-CSDI (Cond.) model against the Reference or Ground Truth (blue), evaluated on combined validation and training windows. From left to right: heavy-tail distribution, volatility clustering, leverage effect.

CRPS and Interval Coverage The CRPS decomposition and coverage analysis of Sec. 6.4.3 were repeated for the 64-channel model with $N = 50$ samples per window over the same 2,000-window protocol (Tab. C.9, Tab. C.10). Probabilistic skill is largely retained: the CRPS skill score over climatology stays at 0.71 (Both) and 0.70 (Val), against 0.74 and 0.73 at 128 channels, and the model still improves on the OLS linear forecast (CRPS skill 0.36 Both, 0.28 Val), confirming that the conditional density keeps information beyond a linear point forecast even at half capacity. The coverage shifts in a way consistent with the heavier, less controlled dispersion of the smaller model: at the 95% and 99% levels the 64-channel intervals are slightly less overconfident than the 128-channel ones (e.g. 95.11% vs. 94.06% at the 95% Val level), whereas at the 50%–90% levels they now over-cover the nominal value, as the extra dispersion, partly the outlier windows of Tab. C.8, widens the min/max intervals rather than reflecting genuinely better calibration. No held-out collapse appears, with Validation tracking the Both split closely, consistent with the underfitting reading of the marginal results above.

Configuration	Split	N. Samples	MAE ↓	CRPS ↓	T_1^{norm}	T_1	T_2^{norm}
EDM-CSDI (Cond.), 128 ch	Val	50	0.184 41	0.121 41	0.244 31	0.005 10	0.245 84
EDM-CSDI (Cond.), 64 ch	Val	50	0.220 41	0.145 08	0.292 99	0.006 11	0.295 82
EDM-CSDI (Cond.), 128 ch	Both	50	0.193 79	0.126 59	0.257 27	0.005 37	0.262 59
EDM-CSDI (Cond.), 64 ch	Both	50	0.216 96	0.142 32	0.289 71	0.006 05	0.294 79

Table C.9: CRPS decomposition for EDM-CSDI (Cond.) at 128 and 64 channels, with $N = 50$ samples per window. CRPS is decomposed as $\text{CRPS} = \text{T}_1^{\text{norm}} - \frac{1}{2}\text{T}_2^{\text{norm}}$, where T_1 is the accuracy term unnormalised by $\sigma = 0.020867$. **N. Samples** is to be intended for each window. Lower CRPS is better. The 128-channel rows reproduce the $N = 50$ values of Tab. 6.6.

Split	Nominal (%)	EDM-CSDI 50, 128 ch	EDM-CSDI 50, 64 ch
Val	50.0	52.23	54.32
Val	80.0	82.16	83.91
Val	90.0	90.35	91.75
Val	95.0	94.06	95.11
Val	99.0	96.89	97.61
Both	50.0	53.05	54.76
Both	80.0	82.52	84.04
Both	90.0	90.50	91.55
Both	95.0	94.02	94.78
Both	99.0	96.84	97.26

Table C.10: Empirical coverage at nominal levels for EDM-CSDI (Cond.) with $N = 50$ samples per window, at 128 and 64 channels. The 128-channel column reproduces the $N = 50$ values of Tab. 6.7. Values closer to the nominal level indicate better-calibrated prediction intervals.

Appendix D

Theoretical Background and Models

This appendix provides expanded mathematical background and explanation for the mathematical foundations and literature review in Ch. 2.

D.1 Brownian Motion

Brownian motion is the continuous-time stochastic process. It was first formally constructed by Wiener, which is why is also called the *Wiener process*.

Definition A stochastic process $\{W_t\}_{t\geq 0}$ defined on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ is called a *standard Brownian motion* if it satisfies the following four properties:

1. *Initialisation*: $W_0 = 0$ almost surely.

2. *Independent increments*: for any $0 \leq t_0 < t_1 < \cdots < t_n$, the increments $W_{t_1} - W_{t_0}, W_{t_2} - W_{t_1}, \ldots, W_{t_n} - W_{t_{n-1}}$ are mutually independent random variables.

3. *Stationary Gaussian increments*: for any $0 \leq s < t$,

$$W_t - W_s \sim \mathcal{N}(0, t-s).$$

4. *Continuous paths*: the map $t \mapsto W_t$ is almost surely continuous.

These four axioms are minimal and mutually consistent. [31].

D.1.1 Key Statistical Properties

Setting $s = 0$ in property 3, we immediately obtain:

$$\mathbb{E}[W_t] = 0, \qquad \mathrm{Var}(W_t) = t, \qquad W_t \sim \mathcal{N}(0,\, t).$$

The variance grows linearly in time: uncertainty accumulates at rate $\sqrt{t}$, not t. This sub-linear growth is the signature of diffusion and contrasts with a deterministic path (variance 0) or with a purely random variable with no time structure.

Covariance and correlation For $0 \le s \le t$, write $W_t = W_s + (W_t - W_s)$. Since $W_t - W_s$ is independent of W_s (property 2) and has mean zero: $\mathrm{Cov}(W_s, W_t) = s = \min(s, t)$. Consequently $\mathrm{Corr}(W_s, W_t) = \sqrt{s/t}$ for $s \le t$. Two observations far apart in time are weakly correlated, so there is no long-range memory.

Markov property The future distribution of W_t given all past information $\mathcal{F}_s = \sigma(W_u : u \le s)$ depends on the past only through the current value W_s:

$$P(W_t \in A \mid \mathcal{F}_s) = P(W_t \in A \mid W_s), \qquad s \le t.$$

Martingale property With respect to $\{\mathcal{F}_t\}$, W_t is a martingale:

$$\mathbb{E}[W_t \mid \mathcal{F}_s] = W_s, \qquad s \le t.$$

Proof: $\mathbb{E}[W_t \mid \mathcal{F}_s] = \mathbb{E}[W_s + (W_t - W_s) \mid \mathcal{F}_s] = W_s + \mathbb{E}[W_t - W_s] = W_s$, using independence and zero mean of the increment, where the martingale property encodes the absence of predictable drift in the noise.

D.2 Itô's Lemma and the Black-Scholes Model

Ordinary calculus provides the chain rule: if f is smooth and $x(t)$ is differentiable, then $df = f'(x)\, dx$. Because BM paths are not differentiable, this rule fails when x is driven by a BM. Itô's Lemma provides the correct generalisation.

D.2.1 Statement of Itô's Lemma

Let $\{X_t\}$ be an Itô process satisfying:

$$dX_t = a_t\,dt + b_t\,dW_t,$$

where a_t (the drift) and b_t (the diffusion coefficient) are stochastic processes satisfying standard integrability conditions [31]. Let $f(t,x)$ be a function that is once continuously differentiable in t and twice continuously differentiable in x. Then:

$$df(t,X_t) = \left(\frac{\partial f}{\partial t} + a_t\,\frac{\partial f}{\partial x} + \frac{1}{2}\,b_t^2\,\frac{\partial^2 f}{\partial x^2}\right)dt + b_t\,\frac{\partial f}{\partial x}\,dW_t.$$

The term $\frac{1}{2}b_t^2\frac{\partial^2 f}{\partial x^2}\,dt$ is the Itô correction. It is absent in ordinary calculus and appear because the stochastic increment has a non-negligible second-order contribution. A proof is provided in [31], chapter 4, *The Itô Formula and the Martingale Representation Theorem.*

D.2.2 Proof of the Black-Scholes Price Equation

Ch. 2 states that applying Itô's Lemma to $\log S_t$ under GBM (Eq. 2.2) yields the log-normal price formula. It was derived in full here. The GBM is the Itô process with drift coefficient $a_t = \mu S_t$ and diffusion coefficient $b_t = \nu S_t$:

$$dS_t = \mu\,S_t\,dt + \nu\,S_t\,dW_t.$$

Apply Itô's Lemma (Eq. D.2.1) to $f(S_t) = \log S_t$:

$$\frac{\partial f}{\partial t} = 0, \qquad \frac{\partial f}{\partial S} = \frac{1}{S_t}, \qquad \frac{\partial^2 f}{\partial S^2} = -\frac{1}{S_t^2}.$$

Substituting $a_t = \mu S_t$ and $b_t = \nu S_t$:

$$\begin{aligned} d(\log S_t) &= \left(0 + \mu S_t \cdot \frac{1}{S_t} + \frac{1}{2}(\nu S_t)^2 \cdot \left(-\frac{1}{S_t^2}\right)\right) dt + \nu S_t \cdot \frac{1}{S_t}\, dW_t \\ &= \left(\mu - \frac{1}{2}\nu^2\right) dt + \nu\, dW_t. \end{aligned}$$

This is a Brownian motion with constant drift $\mu - \frac{1}{2}\nu^2$ and constant diffusion coefficient ν. Integrating over $[0, t]$:

$$\log S_t - \log S_0 = \left(\mu - \frac{1}{2}\nu^2\right) t + \nu W_t,$$

and exponentiating:

$$S_t = S_0 \exp\left[\left(\mu - \frac{1}{2}\nu^2\right) t + \nu W_t\right].$$

Since $W_t \sim \mathcal{N}(0, t)$, the log-return over $[0, t]$ satisfies:

$$\log\left(\frac{S_t}{S_0}\right) \;\sim\; \mathcal{N}\left(\left(\mu - \frac{1}{2}\nu^2\right) t,\; \nu^2 t\right).$$

Log-returns are normally distributed with mean proportional to t and variance proportional to t, confirming the statement in Sec. 2.2 and the foundation of the Black–Scholes option pricing model [24].

Interpreting the Itô correction The term $-\frac{1}{2}\nu^2$ is not an approximation error, instead it is present because the exponential map from log-price to price is convex, so randomness in log-space increases the arithmetic expectation in price-space, therefore it compensates for this convexity effect so that the price process still has drift μ in expectation, so that $\mathbb{E}[S_t] = S_0\, e^{\mu t}$.

D.3 Poisson Process and Merton's Jump-Diffusion Model

The Poisson process is the canonical model for events that arrive randomly, infrequently, and with no predictable timing, these in the financial context are sudden discontinuous jumps in asset prices.

D.3.1 The Poisson Process

Definition A stochastic process $\{P_t\}_{t\geq 0}$ taking values in $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$ is a *Poisson process with intensity* $\lambda > 0$ if:

1. $P_0 = 0$ almost surely.

2. Independent increments: for any $0 \leq t_0 < t_1 < \cdots < t_n$, the increments $P_{t_1} - P_{t_0}$, $\ldots$, $P_{t_n} - P_{t_{n-1}}$ are mutually independent.

3. Stationary Poisson increments: for any $0 \leq s < t$,

$$P_t - P_s \ \sim \ \text{Poisson}(\lambda(t-s)), \tag{D.1}$$

   i.e., $P(P_t - P_s = k) = \frac{e^{-\lambda(t-s)}[\lambda(t-s)]^k}{k!}$ for $k \in \mathbb{N}_0$.

4. At most one event per instant: $P(P_{t+h} - P_t \geq 2) = o(h)$ as $h \to 0^+$.

The process P_t counts the number of jumps that have occurred by time t. From Eq. (D.1):

$$\mathbb{E}[P_t] = \lambda t, \qquad \text{Var}(P_t) = \lambda t.$$

The intensity of λ simultaneously controls the average jump rate and the variability of the jump count. Because the waiting time between jumps are independent then we are dealing with a memoryless process, where the time until the next jump does not depend on how long one has already waited, see limitations in Sec. D.4).

D.3.2 Merton's Jump-Diffusion Model

Merton [23] extended the GBM (Eq. 2.2) by superimposing a compound Poisson jump component onto the continuous diffusion:$dS_t = \mu\, S_t\, dt + \nu\, S_t\, dW_t + S_{t^-}(J-1)\, dP_t$.. As discussed in Sec. 2.2 the log-return distribution under Merton's model is a Poisson-weighted mixture of Gaussians. This result requires $\log J$ to be normally distributed; consistent with Merton's [23] original formulation, J is therefore *log-normally* distributed:

$$\log J \;\sim\; \mathcal{N}(\mu_j,\, \sigma_J^2). \tag{D.2}$$

The parameter μ_j is the mean log-jump size, while the parameter σ_J^2 controls the dispersion of jump sizes.

D.3.3 Log-Return Distribution under Merton's Model

Applying the jump-diffusion analogue of Itô's Lemma to $f(S_t) = \log S_t$ under Eq. D.3.2, the continuous part is handled exactly as in App. D.2, while each jump at time t contributes $\log(J)$ to the log-price. The resulting log-price dynamics are:

$$d(\log S_t) = \left(\mu - \frac{1}{2}\nu^2\right)dt + \nu\, dW_t + \log J\, dP_t.$$

Integrating over $[0, t]$, conditional on exactly n jumps occurring in $[0, t]$, the log-return is a sum of the continuous Gaussian component and n independent log-normal jump contributions:

$$\log\left(\frac{S_t}{S_0}\right) \;\Bigg|\; P_t = n \;\sim\; \mathcal{N}\big(\big(\mu - \tfrac{1}{2}\nu^2\big)\, t + n\mu_j,\;\; \nu^2 t + n\sigma_J^2\big)\,.$$

The unconditional log-return density is obtained by summing over all possible jump counts, weighted by their Poisson probabilities $p_n = e^{-\lambda t}(\lambda t)^n / n!$:

$$p\Big(\log \tfrac{S_t}{S_0} = x\Big) = \sum_{n=0}^{\infty} \frac{e^{-\lambda t}(\lambda t)^n}{n!}\, \phi\big(x;\; \big(\mu - \tfrac{1}{2}\nu^2\big)\, t + n\mu_j,\; \nu^2 t + n\sigma_J^2\big)\,, \tag{D.3}$$

where $\phi(\cdot;\, m, v)$ denotes the Gaussian density with mean m and variance v. This is the Poisson-weighted mixture of Gaussians stated in Sec. 2.2.

D.4 Further Limitations of GARCH and Merton's Model

Sec. 2.2 identifies the core limitation shared by both models, which is further expanded here.

D.4.1 GARCH-X

Symmetric shock response and the leverage effect The GARCH(1,1) variance equation $\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$ depends on ϵ_{t-1}^2, which is symmetric in the sign of the innovation: a negative shock $\epsilon_{t-1} = -c$ and a positive shock $\epsilon_{t-1} = +c$ of the same magnitude produce identical increases in σ_t^2, contradicting the leverage effect.

Tail inadequacy Even with Student-t innovations, the GARCH conditional tail remains insufficiently heavy. Sec. 2.1 reported power-law tails with exponents $\alpha \in [3, 5]$; standard GARCH with Gaussian innovations has finite moments of all orders, and heavier-tailed innovations do not reproduce the empirical tail decay.

Short-memory volatility autocorrelation In GARCH(1,1), squared-return autocorrelation decays exponentially at rate $(\alpha+\beta)^k$. Empirically, volatility autocorrelation decays hyperbolically, $\mathrm{Corr}(|r_t|, |r_{t+k}|) \sim k^{-\beta}$ with $\beta \in [0.1, 0.5]$ [1], causing underestimation of volatility persistence at long horizons even when $\alpha + \beta$ is close to one.

Absence of jumps GARCH generates smooth conditional variance paths and cannot produce discontinuous price gaps or isolated large moves.

Re-estimation requirement The model must be re-fitted from scratch whenever a new regime arises, with no native mechanism to detect regime transitions.

D.4.2 Merton-X

Constant jump intensity The Poisson intensity λ in Eq. D.3.2 is fixed, whereas in reality jump activity is time-varying and clustered: large moves arrive in bursts during

market stress and are rare during calm periods. A constant λ cannot reproduce this self-reinforcing behaviour.

Tail inadequacy despite fat tails Although the Poisson mixture (Eq. D.3) has heavier tails than a single Gaussian, all moments remain finite. The n-th component has variance $\nu^2 t + n\sigma_J^2$ growing linearly in n, but the Poisson weights $e^{-\lambda t}(\lambda t)^n/n!$ decay super-exponentially, so the tail of Eq. D.3 decays faster than any power law.

No volatility clustering The diffusion component $\nu\, S_t\, dW_t$ has constant volatility ν. Jumps introduce excess kurtosis, but not persistent time-varying conditional variance: between jump events the model reverts to a constant-volatility GBM, and a jump does not alter the volatility of subsequent diffusion increments.

No leverage effect Both the diffusion noise $\nu\, dW_t$ and the jump multiplier J are specified symmetrically with respect to the sign of the return, so Merton's model cannot reproduce the negative lead-lag correlation $L(k) < 0$ defined in Sec. 2.1.

Calibration difficulty The likelihood surface of Eq. D.3 has multiple local optima, and the jump parameters $(\lambda, \mu_j, \sigma_J^2)$ are poorly identified unless the sample is long and contains clearly distinguishable events. With limited data, a small number of outliers can dominate the estimation of λ and σ_J^2, leading to unstable parameter estimates, which is why we had to constrain the generation process.

D.5 Likelihoods and Derivations

D.5.1 Log-GARCH-X Likelihood

Given the variance recursion (Eq. 3.1), the standardised residual is $z_t = \varepsilon_t/\sigma_t$. The negative log-likelihood is:

$$\mathcal{L}_{\text{GARCH-X}} = -\sum_{t=2}^{T}\Big[\log p_{t_\nu}(z_t) - \tfrac{1}{2}\log\sigma_t^2\Big], \tag{D.4}$$

where p_{t_ν} is the standardised Student-t log-pdf. The recursion is initialised at $\sigma_1^2 = \mathrm{Var}(r)$, and the stability constant $c = 10^{-8}$ prevents $\log(\varepsilon_{t-1}^2 + c)$ from becoming $-\infty$ on zero

residuals.

D.5.2 Merton-X Likelihood

Conditional on $P_t = k$ jumps, $r_t \sim \mathcal{N}(\mu + k\mu_J,\, \nu_t^2 + k\sigma_J^2)$. Marginalising over the Poisson counts:

$$\log p(r_t) = \log \sum_{k=0}^{K_{\max}} \frac{e^{-\lambda}\lambda^k}{k!}\, \mathcal{N}\big(r_t;\ \mu + k\mu_J,\ \nu_t^2 + k\sigma_J^2\big)\,, \tag{D.5}$$

evaluated via the log-sum-exp identity for numerical stability. The truncation $K_{\max} = 20$ covers all relevant probability mass for $\lambda \lesssim 15$; at that intensity, $P(P_t > 30) < 10^{-6}$. The full-series log-likelihood is a sum of T independent terms (no sequential dependency).

D.5.3 Estimation Procedure

Both models are fitted per ticker by minimising the negative log-likelihood with L-BFGS-B [43]. To reduce sensitivity to initialisation, two structurally distinct starting points are tried and the one with the lower NLL is retained. For GARCH-X, $(\alpha_0, \beta_0) \in \{(0.05, 0.90),\ (0.03, 0.90)\}$; for Merton-X, $(\lambda_0, \sigma_{J,0}) \in \{(0.10, 0.01),\ (0.50, 0.05)\}$.

D.5.4 Implementation Constraints and Design Choices

The following model-specific choices were made to ensure numerical stability and physical plausibility during fitting and simulation. Their value were adjusted iteratively to ensure the best possible outcome and distribution fidelity.

GARCH-X The model is formulated in log-variance space: the recursion propagates $\log \sigma_t^2$ rather than σ_t^2 directly, keeping variance strictly positive by construction. Two exogenous regressors enter the variance equation at each time step t, both observable before the day's close: (i) the squared overnight log-return $\texttt{open}_t^2$, and (ii) the Parkinson intra-day variance estimator $(\log H_t/L_t)^2/(4\log 2)$. The mean equation includes an AR(1) term whose coefficient ϕ is hard-bounded to $(-0.5,\ 0.5)$ to prevent simulated paths from diverging. Degrees of freedom are constrained to $\nu > 2.01$ via $\nu = 2.01 + e^{\vartheta}$, guaranteeing finite variance of the innovation distribution. Covariance stationarity is enforced by a

persistence cap $\alpha + \beta < 0.98$; any parameter vector violating this bound is penalised with NLL $= 10^{12}$. The log-squared lagged residual is regularised as $\log(\varepsilon_{t-1}^2 + 10^{-8})$ to prevent $\log 0$, and the log-variance is clipped to $[-30, 30]$ to prevent overflow. The recursion is initialised at $\log \mathrm{Var}(r)$.

Merton-X The diffusion variance is time-varying with log-linear dependence on two intra-day regressors: $\log \nu_t^2 = a_0 + \mathbf{q}_t^\top \mathbf{a}$, where $\mathbf{q}_t = [\texttt{open}_t^2,\ (\log H_t/L_t)^2]^\top$. Unlike GARCH-X, both regressors are winsorised at their 99th-percentile training value before entering the likelihood, limiting the influence of extreme intra-day moves on the fitted variance surface. Hard bounds are imposed on the jump parameters: $\log \lambda \in [-6, 3]$ (i.e. $\lambda \in (0.002, 20)$), $\mu_J \in [-0.5, 0.5]$, and $\log \sigma_J \in [-8, 0]$ (i.e. $\sigma_J \leq 1$), preventing degenerate jump configurations during optimisation. *Correction to Eq. D.5*: the Poisson series is truncated at $K_{\max} = 20$, not $K_{\max} = 30$ as stated in that equation. For S&P 500 daily returns, fitted λ values are consistently below 5, for which $P(N > 20 \mid \lambda) < 8 \times 10^{-8}$; the remaining tail mass is numerically negligible and the tighter truncation has no effect on results.

Appendix E

Mathematical Proofs

E.1 Score-Based Diffusion Models

E.1.1 Score Matching

Ch. 2 introduced the Fisher divergence as the natural objective for learning the score function, noting that it is intractable because it requires $\nabla_x \log p(x)$. Hyvärinen [27] showed that the Fisher divergence can be rewritten in a form computable without the true score. Starting from:

$$J_{SM}(\theta) = \frac{1}{2}\,\mathbb{E}_{p(x)}\left[\|s_\theta(x) - \nabla_x \log p(x)\|_2^2\right],$$

expand the square and drop the term $\frac{1}{2}\mathbb{E}[\|\nabla_x \log p(x)\|^2]$, because constant w.r.t. θ:

$$J_{SM}(\theta) = \frac{1}{2}\,\mathbb{E}_{p(x)}\left[\|s_\theta(x)\|_2^2\right] - \mathbb{E}_{p(x)}[s_\theta(x) \cdot \nabla_x \log p(x)] + C.$$

The cross-term equals:

$$\int p(x)\, s_\theta(x) \cdot \nabla_x \log p(x)\, dx = \int s_\theta(x) \cdot \nabla_x p(x)\, dx.$$

Applying integration by parts, with the boundary terms vanishing since $p(x) \to 0$ at infinity:

$$\int s_\theta(x) \cdot \nabla_x p(x)\, dx = -\int p(x)\, \nabla_x \cdot s_\theta(x)\, dx = -\mathbb{E}_{p(x)}[\nabla_x \cdot s_\theta(x)]\,,$$

where $\nabla_x \cdot s_\theta(x) = \mathrm{Tr}(\nabla_x s_\theta(x))$ is the trace of the Jacobian. Substituting back:

$$J_{SM}(\theta) = \mathbb{E}_{p(x)}\left[\frac{1}{2}\|s_\theta(x)\|_2^2 + \nabla_x \cdot s_\theta(x)\right] + C. \tag{E.1}$$

This objective is tractable: it requires only samples from $p(x)$ and network evaluations, with no ground-truth score. Nevertheless, the cost of computing the Jacobian trace, which requires $O(d)$ vector-Jacobian products, where d is the data dimension, is prohibitively expensive in high dimensions. Denoising Score Matching eliminates this cost entirely.

E.1.2 Denoising Score Matching

Vincent [28] showed that minimising the Fisher divergence over the corrupted marginal $p_\sigma(\tilde{x}) = \int p(x)\, q_\sigma(\tilde{x}|x)\, dx$ is equivalent, up to an additive constant independent of θ, to minimising the Denoising Score Matching (DSM) objective:

$$\mathcal{L}_{\mathrm{DSM}}(\theta) = \mathbb{E}_{x\sim p(x),\, \tilde{x}\sim q_\sigma(\tilde{x}|x)}\left[\|s_\theta(\tilde{x}, \sigma) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)\|^2\right].$$

Proof Both objectives share the term $\mathbb{E}_{p_\sigma}[\|s_\theta(\tilde{x})\|^2]$, because $p_\sigma(\tilde{x}) = \int p(x) q_\sigma(\tilde{x}|x) dx$. The difference lies in the cross-terms. The SM cross-term is:

$$\int p_\sigma(\tilde{x})\, s_\theta(\tilde{x}) \cdot \nabla_{\tilde{x}} \log p_\sigma(\tilde{x})\, d\tilde{x} = \int s_\theta(\tilde{x}) \cdot \nabla_{\tilde{x}} p_\sigma(\tilde{x})\, d\tilde{x}.$$

Since $\nabla_{\tilde{x}} p_\sigma(\tilde{x}) = \int p(x)\, q_\sigma(\tilde{x}|x)\, \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)\, dx$:

$$= \iint p(x)\, q_\sigma(\tilde{x}|x)\, s_\theta(\tilde{x}) \cdot \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)\, dx\, d\tilde{x} = \mathbb{E}_{x,\tilde{x}}[s_\theta(\tilde{x}) \cdot \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)]\,,$$

which is exactly the DSM cross-term. The residual $\mathbb{E}[\|\nabla_{\tilde{x}} \log p_\sigma\|^2] - \mathbb{E}[\|\nabla_{\tilde{x}} \log q_\sigma\|^2]$ is constant in θ, so the two objectives have identical gradients.

For a Gaussian kernel $q_\sigma(\tilde{x}|x) = \mathcal{N}(\tilde{x};\, x, \sigma^2 I)$, the conditional score is analytic:

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x) = -\frac{\tilde{x} - x}{\sigma^2} = -\frac{\epsilon}{\sigma}, \qquad \epsilon = \tilde{x} - x \sim \mathcal{N}(0, I).$$

The network thus learns to predict the normalised noise direction from the corrupted sample, requiring only paired $(x_0, \tilde{x})$ samples and no Jacobian computation. This is the foundation for the DSM loss used in both DDPM (Eq. 2.3) and the CSDI objective (Eq. 2.14).

E.1.3 Probability Flow ODE via the Fokker–Planck Equation

The Fokker–Planck (FP) equation is the mass-conservation law governing how the density $p_t(x)$ evolves under an Itô SDE. Here we derive the probability flow ODE from it.

Intuition Treat $p_t(x)$ as a cloud of probability mass. Two forces reshape it: *drift* f acts like a fluid velocity, transporting mass along deterministic trajectories and *diffusion* $g\, dW_t$ which injects randomness, spreading and smoothing the cloud like heat diffusing from a peak. The FP equation encodes both effects as a single continuity equation.

The FP equation For the forward SDE (Eq. 2.4) with scalar $g(t)$, the marginal density satisfies:

$$\frac{\partial p_t}{\partial t} = -\nabla_x \cdot \big(f(x_t, t)\, p_t\big) + \frac{g(t)^2}{2}\, \Delta_x p_t, \tag{E.2}$$

where $\Delta_x = \sum_i \partial^2 / \partial x_i^2$ is the Laplacian. The first term is advection (transport) by the drift; the second is isotropic diffusion that flattens peaks and fills troughs.

Deriving the probability flow ODE We seek a deterministic ODE $dx_t = v(x_t, t)\, dt$ whose marginals match those of the SDE. Such an ODE satisfies the continuity equation

$\partial_t p_t = -\nabla_x \cdot (v\, p_t)$. Setting this equal to the FP equation requires:

$$-\nabla_x \cdot (v\, p_t) = -\nabla_x \cdot (f\, p_t) + \frac{g(t)^2}{2}\, \Delta_x p_t.$$

Using the identity $\Delta_x p_t = \nabla_x \cdot (p_t\, \nabla_x \log p_t)$, the right-hand side becomes $-\nabla_x \cdot \Big(f\, p_t - \frac{g(t)^2}{2}\, p_t\, \nabla_x \log p_t \Big)$. Matching the divergence terms:

$$v(x_t, t) = f(x_t, t) - \frac{g(t)^2}{2}\, \nabla_x \log p_t(x_t), \tag{E.3}$$

which is the probability flow ODE stated in Sec. 2.4. Replacing the intractable score with $s_\theta(x_t, t)$ yields a Neural ODE with approximately identical marginals, enabling deterministic sampling and exact likelihood computation via the instantaneous change-of-variables formula.

E.2 The Wiener Filter

E.2.1 Wiener Filter and the Skip Connection Coefficient

The EDM preconditioning (Eq. 2.9) includes a skip connection $c_{\text{skip}}(\sigma) \cdot x_t$ that bypasses the neural network. Here c_{skip} is derived as the optimal linear estimator of x_0 given x_t, known as the *Wiener filter coefficient*.

Under the VE forward process $x_t = x_0 + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$, we seek the scalar c minimising the mean squared error of the linear predictor $c\, x_t$:

$$\min_c\ \mathbb{E}\big[\|c\, x_t - x_0\|^2\big]\,.$$

Setting the derivative with respect to c to zero (per component):

$$\frac{d}{dc}\, \mathbb{E}\big[\|c\, x_t - x_0\|^2\big] = 2\, \mathbb{E}[x_t(c\, x_t - x_0)] = 0 \implies c^* = \frac{\mathbb{E}[x_0 \cdot x_t]}{\mathbb{E}[x_t^2]}.$$

Computing the second moments under $x_t = x_0 + \sigma\epsilon$, with x_0 and ϵ independent and $\sigma^2_{\text{data}} = \mathbb{E}[x^2_{0,i}]$ the per-component data variance:

$$\mathbb{E}[x_0 \cdot x_t] = \mathbb{E}[x_0(x_0 + \sigma\epsilon)] = \mathbb{E}[x_0^2] + \sigma\,\mathbb{E}[x_0 \cdot \epsilon] = \sigma^2_{\text{data}},$$

$$\mathbb{E}[x_t^2] = \mathbb{E}[(x_0 + \sigma\epsilon)^2] = \sigma^2_{\text{data}} + \sigma^2.$$

Therefore:

$$c^* = \frac{\sigma^2_{\text{data}}}{\sigma^2_{\text{data}} + \sigma^2} = c_{\text{skip}}(\sigma).$$

This is precisely the EDM formula. The interpretation is direct: at low noise $\sigma \ll \sigma_{\text{data}}$, $c_{\text{skip}} \approx 1$ and the output is mostly the noisy input (already a good estimate of x_0); at high noise $\sigma \gg \sigma_{\text{data}}$, $c_{\text{skip}} \approx 0$ and the network F_θ must reconstruct x_0 almost entirely from scratch. The non-linear residual $c_{\text{out}}(\sigma) \cdot F_\theta(\cdot)$ accounts for what the Wiener filter cannot: the non-Gaussian, non-linear structure of the data distribution beyond its second-order statistics.

E.3 Bounds on the CRPS

Sec. 2.1 introduced the CRPS as a calibration-adjusted MAE. Here it is shown that, for X, X' i.i.d. draws from $\hat{F}$ and observation y,

$$0 \;\le\; \text{CRPS}(\hat{F}, y) \;\le\; \mathbb{E}_{\hat{F}}[|X - y|] = \text{MAE}.$$

Upper bound. Since $|X - X'| \geq 0$, its expectation is non-negative, so

$$\text{CRPS}(\hat{F}, y) = \mathbb{E}[|X - y|] - \tfrac{1}{2}\mathbb{E}[|X - X'|] \;\le\; \mathbb{E}[|X - y|] = \text{MAE}.$$

Lower bound. By the triangle inequality, $|X - X'| \leq |X - y| + |X' - y|$. Taking expectations and using that X and X' are identically distributed, so $\mathbb{E}[|X - y|] = \mathbb{E}[|X' -$

$y|]$:

$$\mathbb{E}[|X - X'|] \leq \mathbb{E}[|X - y|] + \mathbb{E}[|X' - y|] = 2\,\mathbb{E}[|X - y|]\,.$$

Dividing by two and rearranging gives $\mathbb{E}[|X-y|]-\frac{1}{2}\mathbb{E}[|X-X'|] \geq 0$, i.e. $\mathrm{CRPS}(\hat{F}, y) \geq 0$.

Together, the two bounds confirm that CRPS is non-negative and never exceeds the MAE of the ensemble mean prediction error, with equality in the upper bound when $\hat{F}$ collapses to a point mass (so $X = X'$ almost surely and the sharpness term vanishes).

Appendix F

Training Configuration

F.1 Optimiser and Learning Rate Schedule

AdamW At each step, the bias-corrected first and second moment estimates $\hat{m}_t$ and $\hat{v}_t$ are computed with decay coefficients β_1 and β_2, and the parameter update is:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta\lambda\,\theta_{t-1},$$

where η is the learning rate, λ the weight-decay coefficient, and ε a small constant for numerical stability.

Cosine annealing The learning rate at step t of a schedule of total length T is:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t}{T}\pi\right)\right).$$

A known risk of this schedule is entrapment in a local minimum if the learning rate decays below the threshold needed to escape it before convergence is reached.

F.2 Model Dimension and Parameter Count

This section derives the trainable parameter count of the CSDI-based diffusion backbone for the two experimental configurations, which differ only in the number of target features K.

F.2.1 Notation and Hyperparameters

Here is some notation: K is `target_dim`, E_f is `featureemb`, E_t is `timeemb`, C is `channels`, D is `diffusion_embedding_dim`, R is `num_layers`.

The side-information width per (feature, time) location is $S = E_t + E_f + 1$, where the $+1$ is the conditioning mask channel. This is independent of K because E_f is the output dimension of the feature embedding lookup, not the vocabulary size. The Transformer feed-forward width is $d_{\text{ff}} = 4C$. Fixed values:

$$E_f = 64, \quad E_t = 128, \quad C = 128, \quad D = 256, \quad S = 193, \quad I = 2, \quad R = 6.$$

F.2.2 Shared Parameters

Non-backbone

- **Time-embedding projection**: Linear(E_t, E_t): $E_t^2 + E_t = 16{,}512$.
- **Outside residual blocks (`diff_CSDI`)**: input projection Conv1d$(I, C, 1)$, output projection Conv1d$(C, 1, 1)$, and the two-layer diffusion embedding $2\,\text{Linear}(D, D)$

The total count of Non-backbone element is:

$$N_{\text{outside}} = (IC + C) + (C + 1) + 2(D^2 + D) = 384 + 129 + 131{,}584 = 132{,}097.$$

Residual Block (`ResidualBlock`)

- **Four projection layers**: diffusion Linear(D, C), conditioning Conv1d$(S, 2C, 1)$, mid and output Conv1d$(C, 2C, 1)$ for each residual block:

$$N_{\text{proj}} = DC + 2CS + 4C^2 + 7C = 148{,}608.$$

- **one Transformer encoder layer**: $d_{\text{model}} = C$, $d_{\text{ff}} = 4C$, contributions from multi-head attention $4C^2 + 4C$, feed-forward network $8C^2 + 5C$, and two layer-norm layers

$4C$ for each residual block:

$$N_{\text{trans}} = 12C^2 + 13C = 198{,}272.$$

F.2.3 K-dependent Parameters

Two components depend on K.

- **Feature embedding**: Embedding(K, E_f): contributes $K \cdot E_f$ parameters.
- **Feature Transformer**: when $K > 1$, each residual block contains a second Transformer encoder layer of identical architecture to the time Transformer, operating across the feature axis (N_{trans} additional parameters per block). For $K = 1$ this layer is instantiated, but never updated. The reason is just code compatability across phases.

Component	$K = 1$	$K = 4$
Feature embedding KE_f	64	256
Feature Transformer per block	0	198,272
Parameters per residual block N_{res}	346,880	545,152

F.2.4 Total Parameter Count

$$N_{\text{total}} = \underbrace{\left(KE_f + E_t^2 + E_t\right)}_{\text{non-backbone}} + \underbrace{N_{\text{outside}}}_{\text{outside blocks}} + R \cdot N_{\text{res}}.$$

	$K = 1$	$K = 4$
Non-backbone	16,576	16,768
Outside residual blocks	132,097	
$R \cdot N_{\text{res}}$	$6 \times 346{,}880 = 2{,}081{,}280$	$6 \times 545{,}152 = 3{,}270{,}912$
Total	**2,229,953**	**3,419,777**

The two configurations differ by 1,189,824 parameters ($6 \times N_{\text{trans}} + (K_4 - K_1) \cdot E_f = 1{,}189{,}632 + 192$), entirely attributable to the six feature Transformer layers absent in the univariate case and the larger feature embedding vocabulary.

F.3 Attention Memory and Sequence Length Constraint

The CSDI self-attention computes a score matrix of shape ($B \times K$, H, L, L), and PyTorch selects among three SDPA kernels at runtime. Flash SDPA avoids materialising the full matrix but requires more CUDA cores than the available MIG slices provide. Memory-Efficient SDPA encounters an internal limit of 65,535 on the batch $\times$ heads dimension: with $K = 4$, $B = 64$, and $L = 2048$, this product reaches 524,288. The only remaining fallback, Math SDPA, materialises the full $L \times L$ attention matrix, costing approximately 17 GB for attention alone at $L = 2048$, which exceeds the 40 GB budget once model parameters and activations are included. Reducing to $L = 512$ brings the attention cost within budget while still covering approximately two years of daily returns; the stride of 100 maintains the same ratio of overlap as 2048/400.